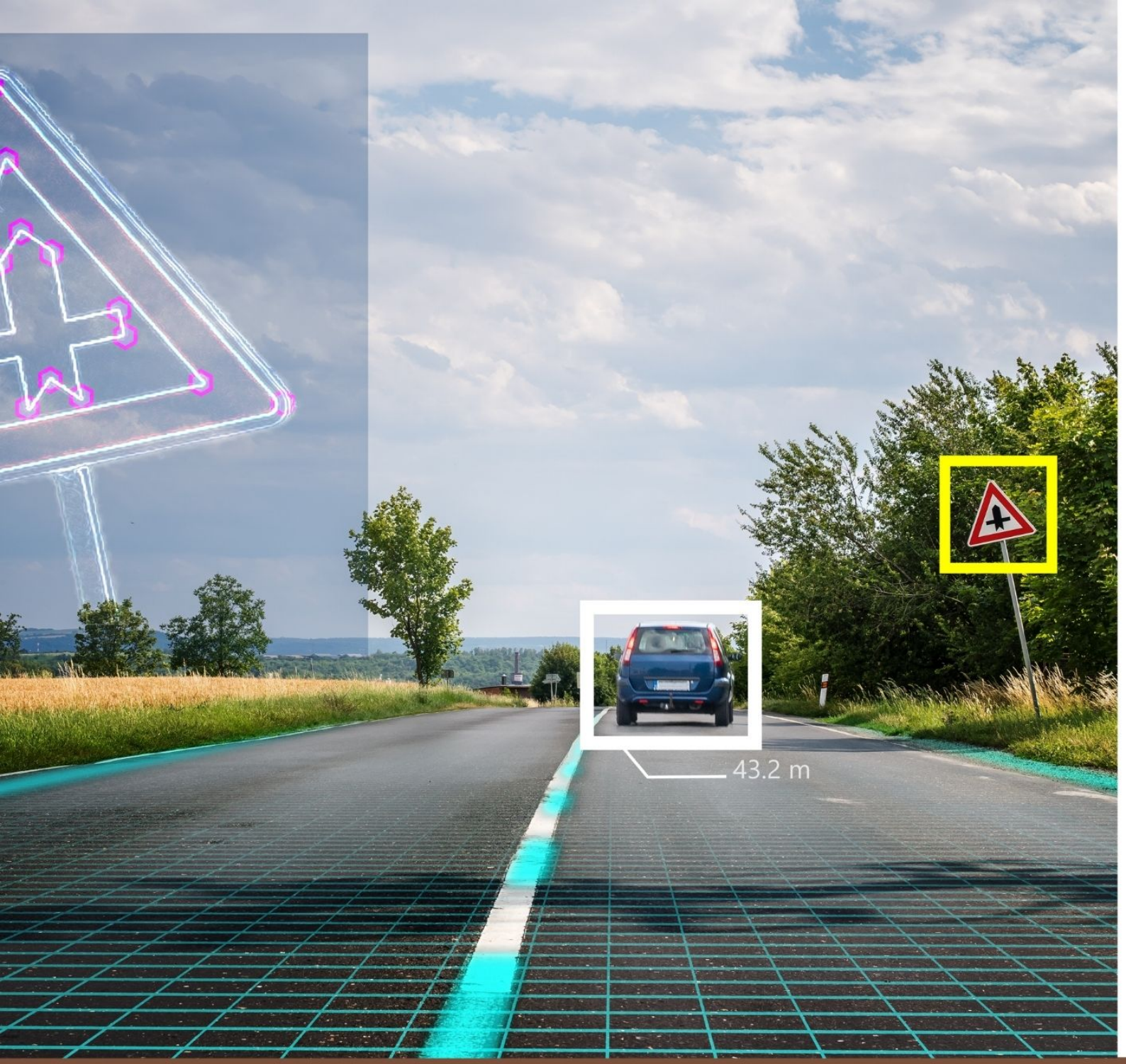


COMPUTER VISION

Prof. Jayanta Mukhopadhyay
Computer Science and Engineering
IIT Kharagpur

INDEX

S. No	Topic	Page No.
	Week 1	
1	Fundamentals of Image Processing Part I	1
2	Fundamentals of Imagr Processing Part II	21
3	Image Transform Part I	39
4	Image Transform Part II	58
	Week 2	
5	Projective Geometry â€“ Part I	79
6	Projective Geometry â€“ Part Ii	94
7	Projective Transformation	114
8	Homography:Properties â€“ Part I	140
9	Homography:Properties â€“ Part Ii	155
10	Homography:Properties â€“ Part Iii	172
	Week 3	
11	Camera Geometry â€“ Part I	189
12	Camera Geometry â€“ Part Ii	208
13	Camera Geometry â€“ Part Iii	223
14	Camera Geometry â€“ Part Iv	240
15	Camera Geometry â€“ Part V	261
	Week 4	
16	Stereo Geometry â€“ Part I	279
17	Stereo Geometry â€“ Part Ii	295
18	Stereo Geometry â€“ Part Iii	310
19	Stereo Geometry â€“ Part Iv	322
	Week 5	
20	Stereo Geometry â€“ Part Vi	334
21	Stereo Geometry â€“ Part V	346
22	Stereo Geometry â€“ Part Vii	363
23	Stereo Geometry â€“ Part Viii	380
	Week 6	
24	Feature Detection And Description â€“ Part I	394
25	Feature Detection And Description â€“ Part II	413
26	Feature Detection And Description â€“ Part III	432
27	Feature Detection And Description â€“ Part IV	449

28	Feature Detection And Description â€“ Part V	474
	Week 7	
29	Feature Matching And Model Fitting- Part I	485
30	Feature Matching And Model Fitting- Part Ii	502
31	Feature Matching And Model Fitting- Part Iii	514
32	Feature Matching And Model Fitting- Part Iv	527
33	Feature Matching And Model Fitting- Part V	540
	Week 8	
34	Color Fundamentals And Processing-Part I	558
35	Color Fundamentals And Processing-Part Ii	574
36	Color Fundamentals And Processing-Part Iii	591
37	Color Fundamentals And Processing-Part Iv	613
38	Color Fundamentals And Processing-Part V	633
39	Color Fundamentals And Processing-Part Vi	651
40	Color Fundamentals And Processing-Part Vii	665
	Week 9	
41	Range Image Processing â€“ Part I	679
42	Range Image Processing â€“ Part Ii	693
43	Range Image Processing â€“ Part Iii	707
44	Range Image Processing â€“ Part IV	720
45	Range Image Processing â€“ Part V	735
	Week 10	
46	Clustering And Classification Â€“ Part I	757
47	Clustering And Classification Â€“ Part Ii	780
48	Clustering And Classification Â€“ Part Iii	799
49	Clustering And Classification Â€“ Part Iv	816
50	Clustering And Classification Â€“ Part V	832
	Week 11	
51	Dimensional Reduction And Sparse Representation â€“ Part	855
52	Dimensional Reduction And Sparse Representation â€“ Part	871
53	Dimensional Reduction And Sparse Representation â€“ Part	891
54	Dimensional Reduction And Sparse Representation â€“ Part	909
	Week 12	
55	Deep Neural Architecture And Applications â€“ Part I	927
56	Deep Neural Architecture And Applications â€“ Part II	950
57	Deep Neural Architecture And Applications â€“ Part III	970

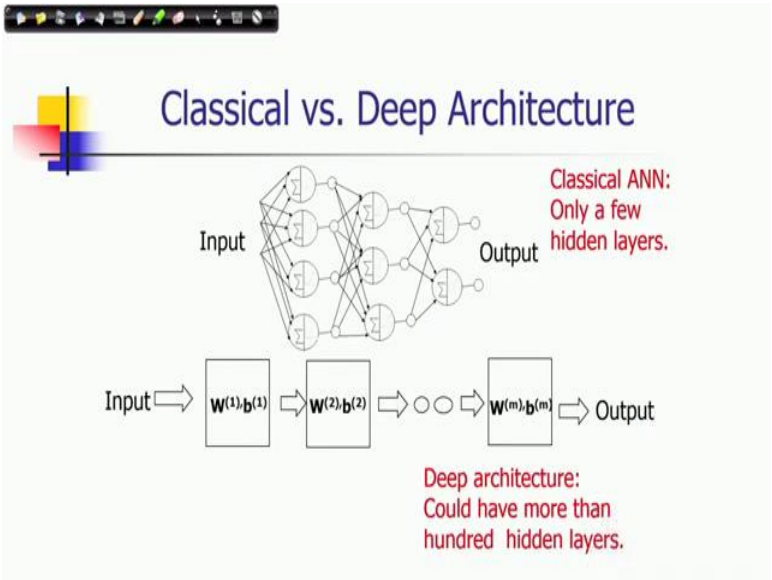
58	Deep Neural Architecture And Applications â€“ Part IV	988
59	Deep Neural Architecture And Applications â€“ Part V	1007
60	Deep Neural Architecture And Applications â€“ Part VI	1028

Computer Vision
Prof. Jayanta Mukhopadhyay
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

Lecture - 55
Deep Neural Architecture and Applications Part – I

We will discuss a new topic today and that is on Deep Neural Architecture and Applications.

(Refer Slide Time: 00:21)



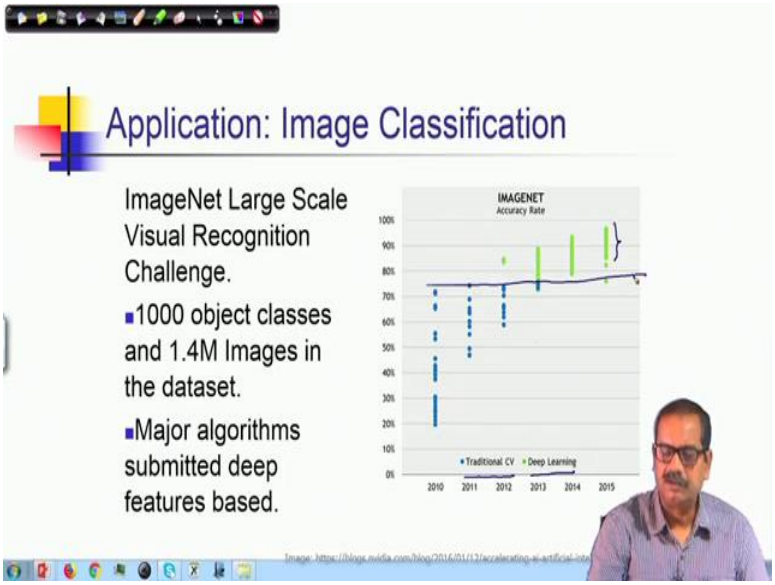
So, let us understand what is the difference between a classical neural architecture and a deep neural architecture. We have already discussed about the artificial neural network where you can see that it is a network of different neural nodes or perceptual nodes where you have the input at one end.

And it is the feed forward network, each node takes the weighted input vector and then the net sum of those weighted vectors are passed through a non-linear functions and the output is the response of that non-linear functions. And network of all these nodes that gives you the artificial neural network. And finally, there is an input and output layers of this particular architecture.

Now, the fact is that so far for classical artificial neural networks we had only a few hidden layers that is what we are used to do maximum three layers that we could see in many of

the applications, but 1 or 2 layers are more common. Whereas, for deep neural architecture you have more number of layers, you can have more than 100 hidden layers and that is one of the major difference.

(Refer Slide Time: 01:41)



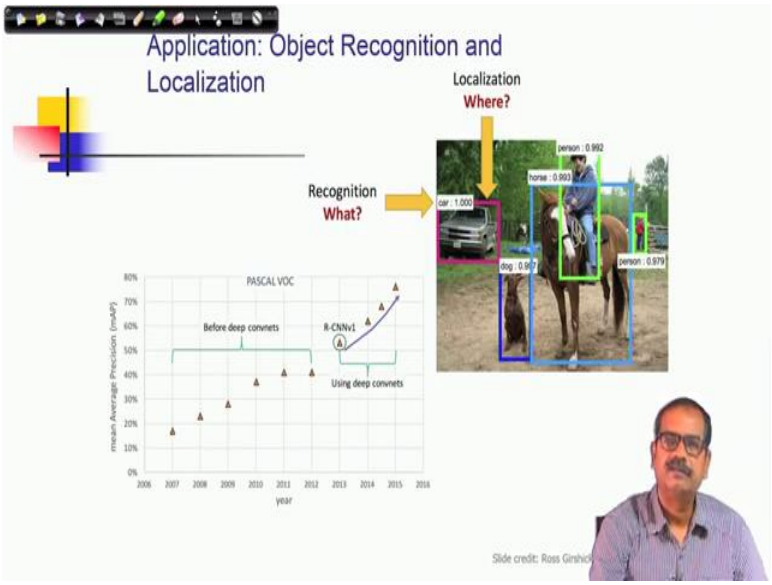
So, there are different applications of this deep neural architecture. So, before describing the features of this architecture let me give a brief overview what are the applications which are reported by this kind of architecture. So, one of the applications is an image classification and there is a data set image net large scale visual recognition challenge data set which was there for improving the classification algorithms for testing. And for a long time there are researchers they are working on these data sets and they are reporting their performance on this data set.

So, in this data set you have 1000 object classes and there are about 1.4 million images the design in the data set. And major algorithms which have been found to provide good performances they are deep features based which means they have used deep neural architecture while classifying this data.

So, here is a brief summary that what are the performances of different algorithms. As you can see that you have these traditional competition algorithms, so this is a traditional algorithm, and this is a deep learning algorithms. So, all the performance indexes of these deep learning algorithms they are very high where the traditional algorithms and as we are moving ahead with time. So, it is still 2050 it was shown after that also the performance

has been improved. So, the strain is increasing and you are getting better and better deep architecture for giving this kind of classification performance.

(Refer Slide Time: 03:35)



Another kind of application that we can discuss that is object recognition and localization. So, this is one example how this problem has been solved what is the kind of response. You can see that in this scene the system tries to localize an object for example, it has been shown car and there is a pink box rectangular box where the car has been shown so, this is the localization. And also it recognizes that what objects it is the car with certain confidence level or some probability value.

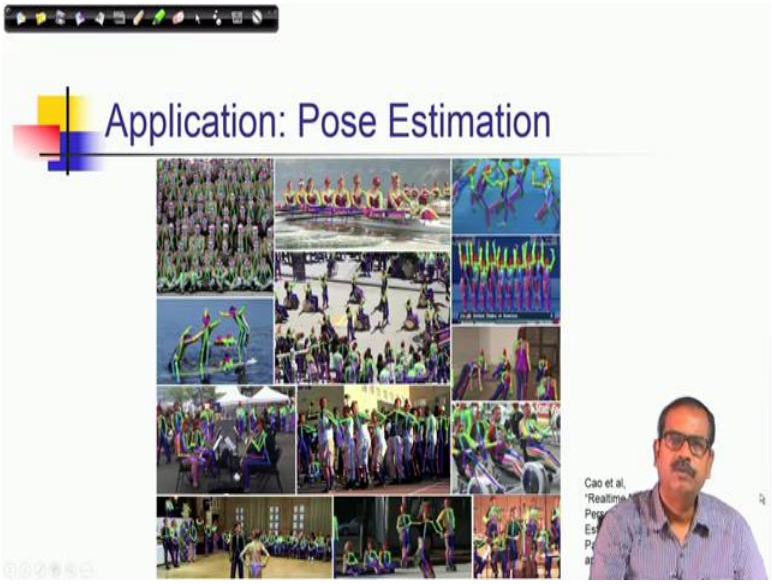
So, it is definitely car because it is 1.00 whereas, you can see that horse is 0.993, so there is some confusion with horse like this. And this kind of there is also another data set where these recognition and localization tasks are performed. And you can see that using deep conclusion neural networks or we will discuss what is a conclusion neural network in this under this topic, so they have provided they have given better performance in this case.

(Refer Slide Time: 04:51)



Object segmentation is another task in fact, the segmentation with respect to deep neural architecture it is called semantic segmentation. Because you are not only segmenting the objects you are also telling that what is that object, it is like localization almost similar thing. But, now you are doing pixel level declarations that is a segment of which type of object. So, this is one kind of application,

(Refer Slide Time: 05:19)



Pose estimation is another, you can see in this diagram that poses of different persons are shown by a skeleton figure. So, skeletons are 3-dimensional information, 3 dimensional it

is a description of a human body, human limbs with the graphical descriptions with 3D coordinate vertices. And connected by their edges and with our joints at different limbs those are representing our human skeleton.

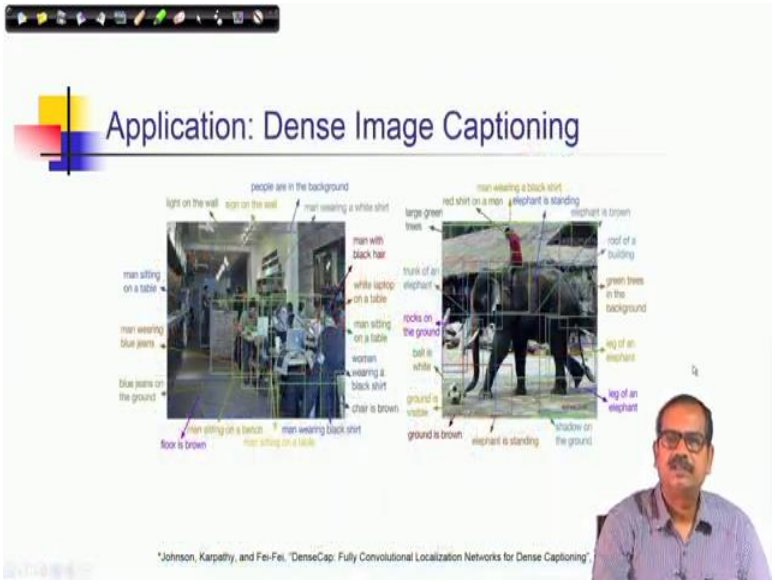
And depending upon the image of a different person, we try to guess at what kind of skeleton configurations could provide that kind of stance or that kind of pose of a person. So, this is estimation and that gives you idea that in what way the person is standing or doing some activity.

(Refer Slide Time: 06:17)



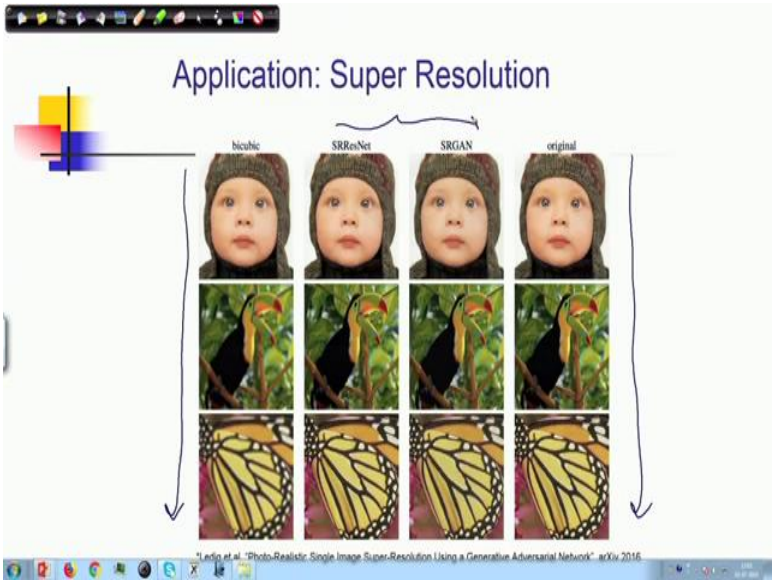
Another interesting application is image captioning it is a cross disciplinary problem as you can see that not only image, understanding of images and videos we are trying to describe using natural languages. So, you can see that with respect to different scenes there is some language, some description like you consider this particular image where a child where a girl is jumping. So, it is written here girl in pink dress is jumping in air, so this kind of descriptions those are generated by this kind of system.

(Refer Slide Time: 06:55)



Similarly, dense image captioning, so it is further complex. So, you would like to have not only you are describing the main motto of the same. But also you are describing every part of the image that is a elephant task of an elephant it is a football, there is a man sitting on top of an elephant like that the man is wearing a red shirt. So, this kind of description and this kind of captioning could be there using this kind of architecture.

(Refer Slide Time: 07:31)



Another application is super resolution, so here you can see that examples are given the original image is shown in the right most column. So, this is the original image and this is

the classical algorithm by cubic interpolation algorithm which produces the super resolutions or interpolated images. Whereas, using deep neural architecture you have this middle two columns those are the results and you can choose you can actually see the visual quality of these two interpolated images or super reserved images they are much better than what we have obtained by cubic interpolation.

(Refer Slide Time: 08:15)

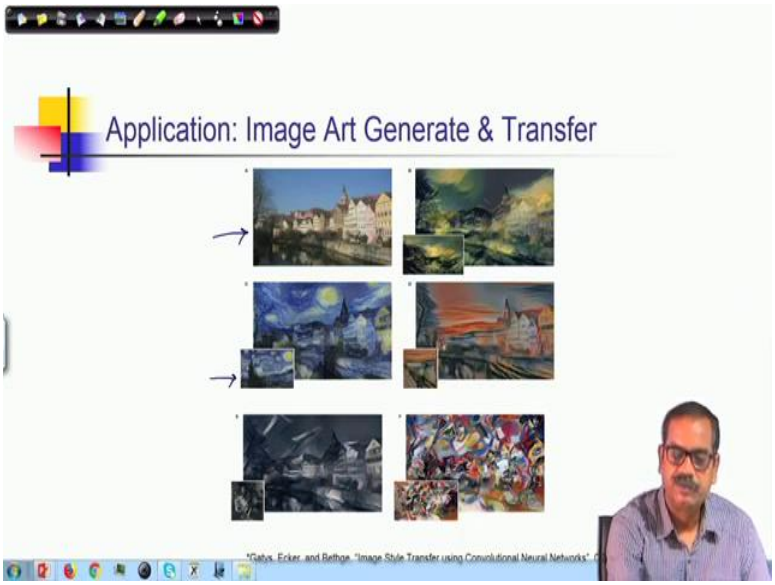
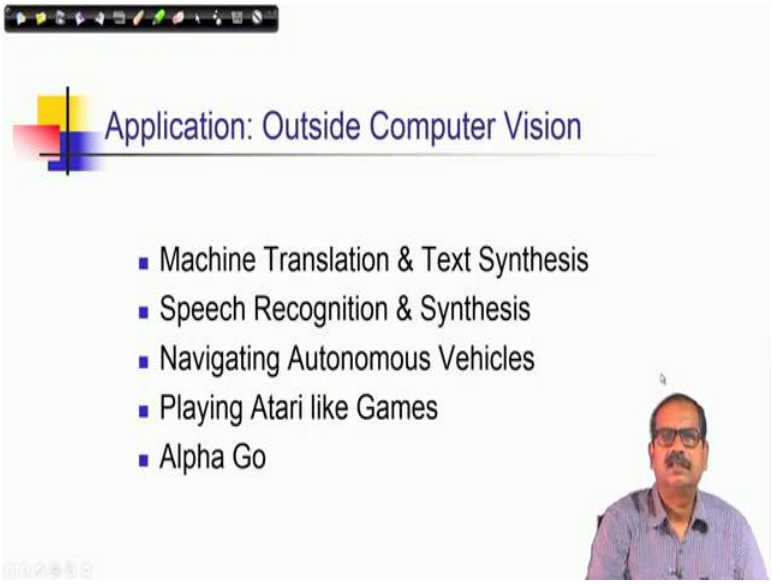


Image art generation and transfer this is also another very interesting applications. In fact, using deep neural architecture you can synthesize images, you can render images and here in particular you can see that there is a photograph. So, this is the original photograph which is taken by a camera and you consider the style of painter say this is a Hancock painting and I think this is starry night or that is a name.

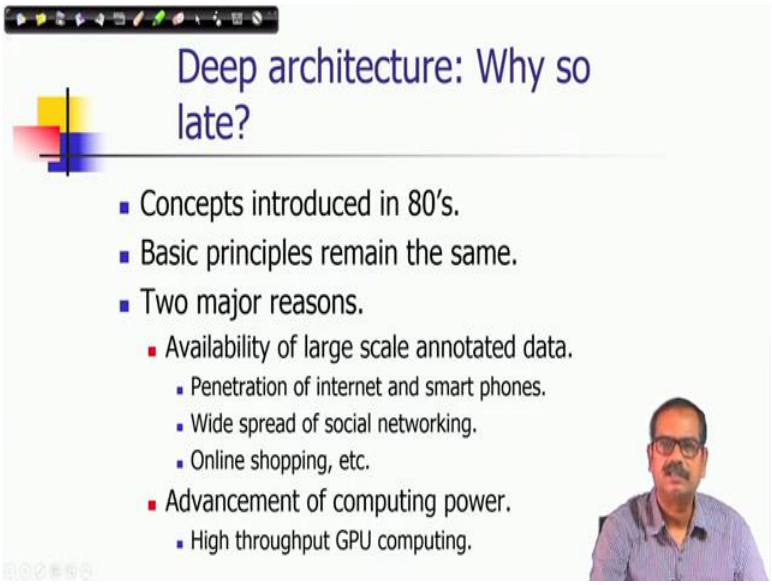
And using Hancock painting, so this photograph is with that style itself painting style this photograph has been generated. So, all these buildings etcetera you can observe here, but they have very high similarity with this style with which it has been painted. And similarly you can get the examples that same scene is rendered by different styles of painters.

(Refer Slide Time: 09:11)



Many other applications are there not only in computer vision deep learning deep neural architecture is used for in other areas. So, machine translation and text synthesis, speech recognition and synthesis, navigating autonomous vehicles, playing different kinds of games, even playing chess alpha go these are things.

(Refer Slide Time: 09:31)



So, one of the very one of the questions will come will naturally this question will arise to us that why this architecture why when it is giving. So, higher performance compared to classical techniques, but why did it take so much of time that to come to the stage. Because

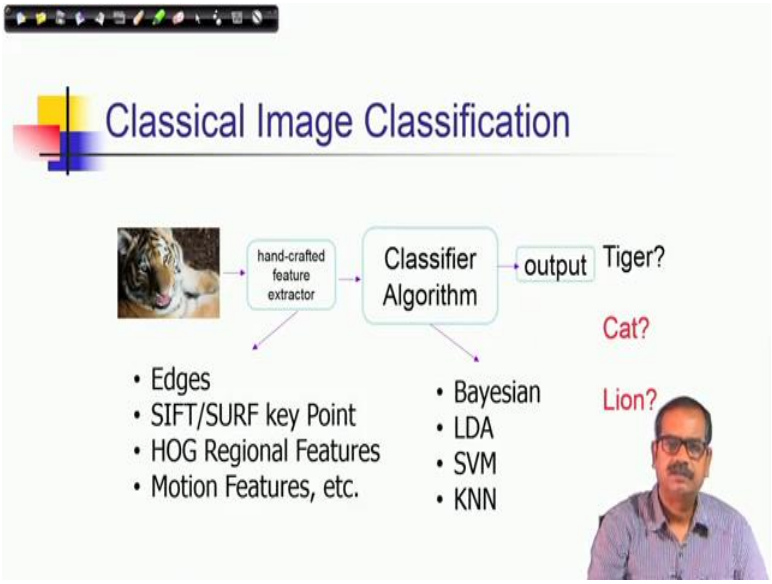
the concepts what you are using in deep neural competitions they are very old they are the same neural network competition, they are the same neural network model. So, they are introduced in 80's and they are basic principles they remain the same.

So, there are two major reasons that is why it has arrived today it is an historical reason; that means, the progress of technology and science that is what led to every kind of new technological advancement. So, it is a new technological advancement because, we have; now we are have the capacity of generating large scale annotated data and because of that availability of the data.

So, you can train large networks to develop any kind of model which can work in a very general, to solve any general and difficult problems. And then the advancement of computing power because this training also requires high computations and it has to be done within a feasible time. So, because, so there are say there are gpu computing which has come with a you know which has been which has given this advancement and which has been very much used for these kinds of competitions.

So, availability of large scale annotated data is possible because of penetration of internet and smartphones, widespread of social networking, online shopping etcetera and advancement of computing power as I mentioned high throughput GPU computing like this is one such example.

(Refer Slide Time: 11:35)



Now, let us understand that what is the difference between classical image classification techniques and the deep features based classification techniques or deep neural competition based classification techniques. In classical image classification you consider you have an image and then you would like to represent that this image by some handcrafted features. Which means, we have studied what could be the distinct features are characterized in this image and we developed computational algorithms to extract those features that is what is called handcrafted features. So, it is designed by as the algorithm is also designed accordingly and then we extract them this features.

For example edges, SIFT, SURF key points, HOG hog regional features, motion features we have discussed some of these features in my previous lectures. And then this feature representation is input to our classification algorithm, so use the classifier. And there are different classifier like bayesian classifier linear discriminate analysis, support vector machine, KNN algorithms. So, different such you know classifier could be there and then we produce the output. Output is something like whether it is tiger, cat, lion it is a standard classification problem, so this is the classical approach.

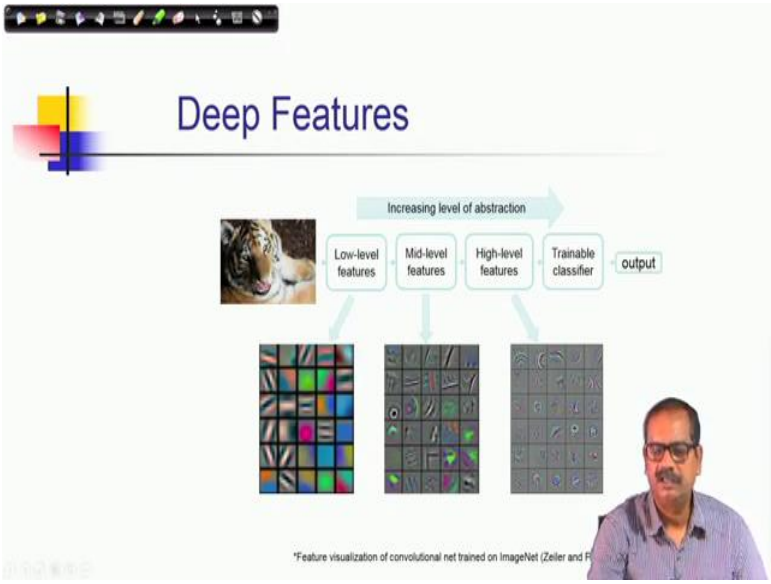
(Refer Slide Time: 13:07)



Whereas, so the problem here is that it is very tedious and costly to develop handcrafted features to handle various challenges, because there is a large variance of input image of the same object. So, these variance causes due to different kind of variations, you have viewpoint variation deformation, occlusion, interclass variation, illumination, clutter,

instances, scale; some nice examples are shown here with respect to that tiger. Say, you can see one by one that what kind of variations are there and, so that is why this is very difficult to develop a very general algorithm, it is dependent on one application and it is not easily transferable to other applications.

(Refer Slide Time: 13:59)

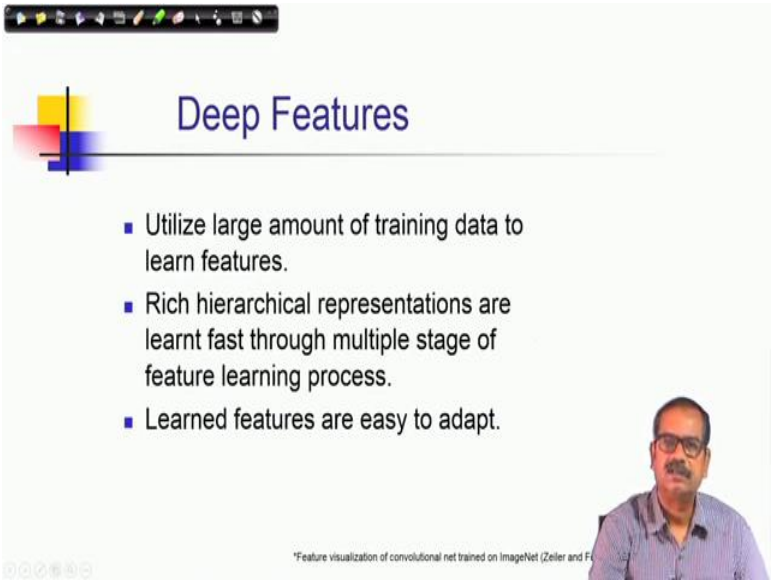


In deep feature based algorithm we learn the feature representation first using the it is a automated learning it is unsupervised learning. So, your mechanism is such that they extract the features depending upon the you know problems statement depending upon the applications.

So, here usually it has been shown we will discuss that part that how you are learning even the feature extractions that is not handcrafted, your neural network itself is learning to represent the features. So, there are different levels of features representations when you are doing deep feature representation.

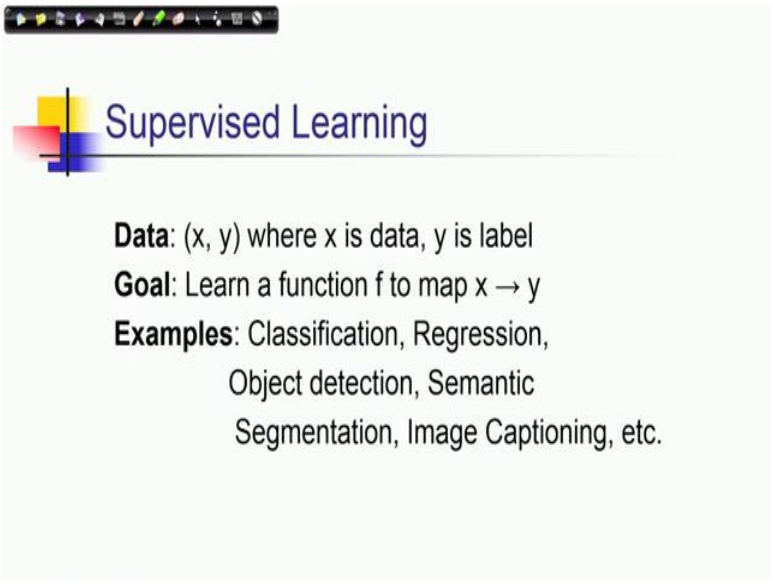
So, youwill have low level features and as you increase the level of abstraction you can have mid level features, you can have high level features and then you train the classifier using this features. The whole thing you are doing in one shot; that means, it is augmented it is coupled in the previous case classification scheme is independent of feature representation scheme. But now feature representation and classification they go hand in hand they go in one shot and then you get the output.

(Refer Slide Time: 15:23)



So, it utilizes large amount of training data to learn features and it has reached hierarchical representations as I have mentioned that from low level, mid level to high level feature representation. So, multiple stages of feature learning processes involved and it learns features adoptively, so in the context of your application this features are learnt.

(Refer Slide Time: 15:47)

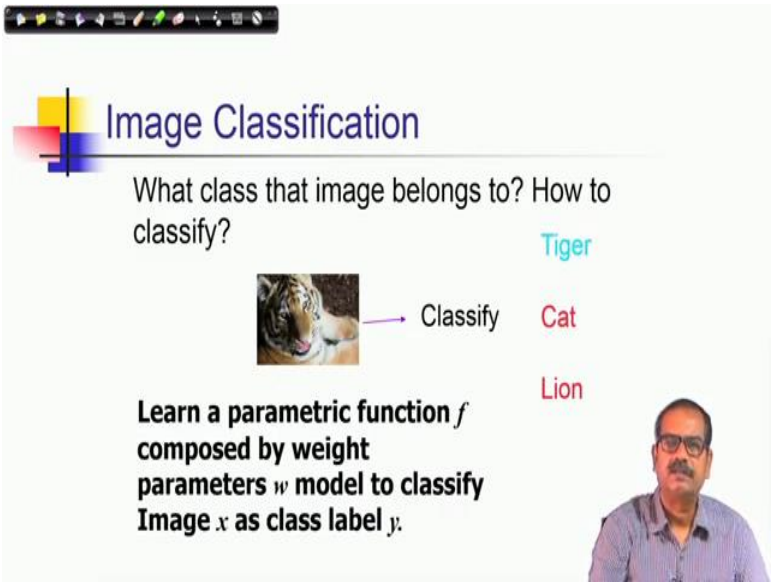


Mostly we will be considering supervised learning and in the deep neural architecture, particularly classification problem as we have discussed that you have the training

samples, you are training the network and then you are classifying. So, this is then ideal supervised classification setup and we call it also supervised learning.

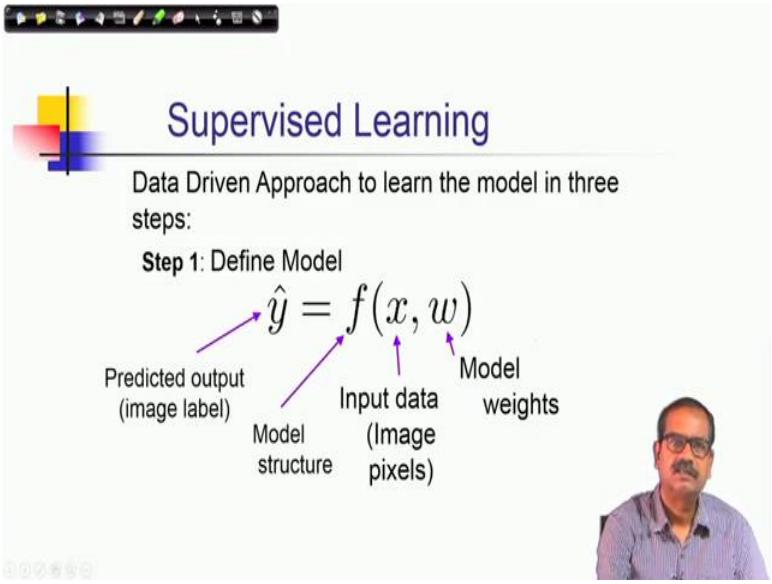
So, the problem here is that you have x data and y label, so I am just revisiting this we have discussed also in the context of classification previously. So, x is the data, y is the label and our goal is to learn a function f which will map the data to the label. And different kinds of problems like classification, and regression, object detection, semantic, segmentation, image captioning all could be mapped to this kind of learning framework.

(Refer Slide Time: 16:39)



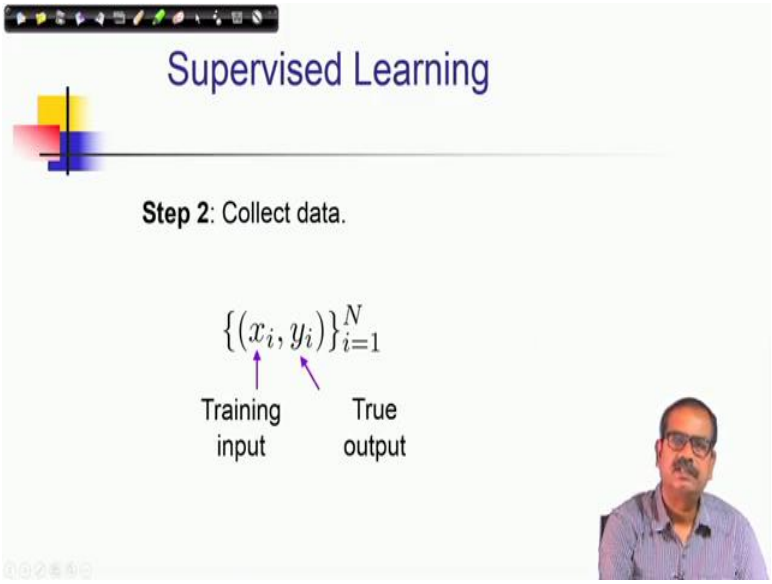
So, let us consider image classification and in the image classification the problem is at what class that image belongs to and how to classify, we have already understood this problem statement. So, in this case we would learn we would like to learn a parameter function f which composed by weight parameters of the model to classify image x as class label y.

(Refer Slide Time: 17:03)



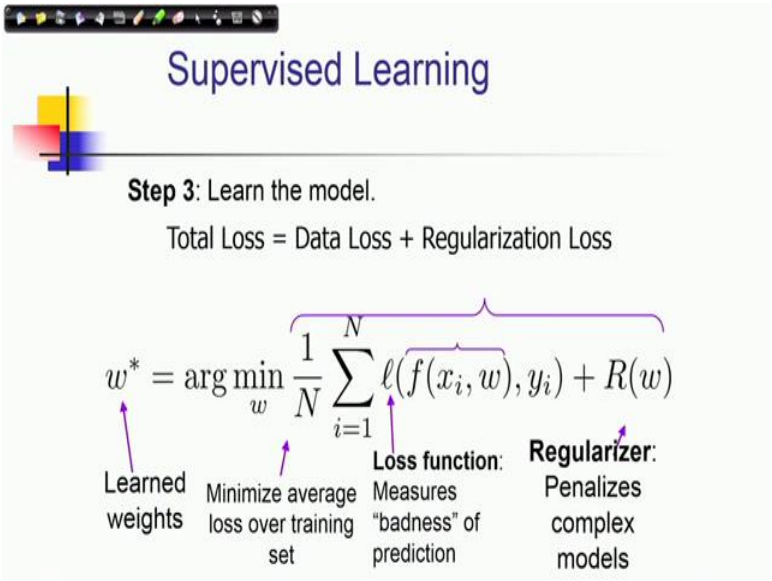
So, it is a data driven approach to learn the model in three steps; first you have to define the model like we have defined the prediction of class y given the input and the model w. So, artificial neural network we discussed that is one example of model and as I mentioned that deep neural architecture is nothing but the same artificial neural network model only you have more number of hidden layers. So, you have input data as image pixels here and you have the model weights and this is the model structure, and you have to this is a predicted output or image label.

(Refer Slide Time: 17:43)



So, first step is defined model next, step is you have to collect data. So, this is for this supervised learning it is important that you know you should have annotated data, xi is instance, and yi is the label of the data. So, this is the training input and this is the true output yi and then you learn the model.

(Refer Slide Time: 18:03)



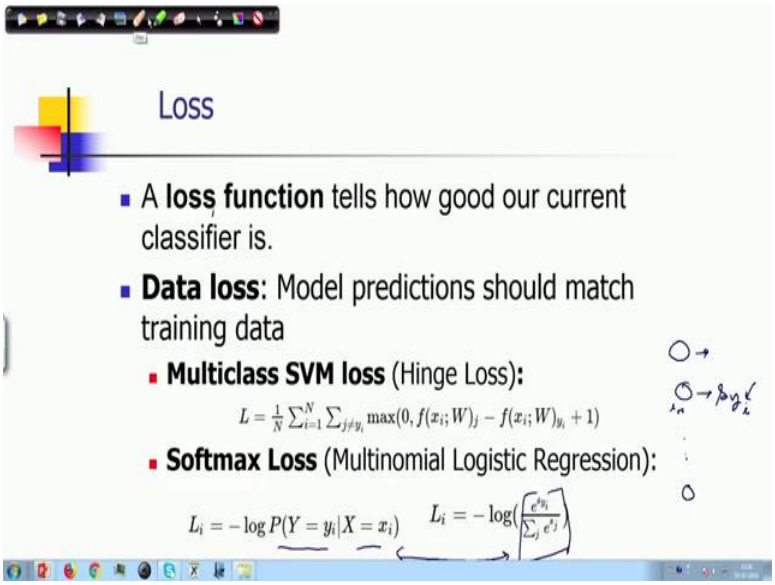
So, here we consider that that there is some mapping function that you have learn and how good is that mapping function and what is the gap then we try to update that gap, so in terms of weights. So, whatever weights are there what is the performance? If the loss is more then we need to update, so that loss should be minimized or loss should be reduced.

So, we use a loss function here to just the to assess the performance of a model, so which is represented here by these notation. So, this is the loss function what is the predicted output, and what is the ground truth depending upon this loss function is defined. And for a set of training samples we accumulate this loss functions and this is the regularization term, because your model should not be too complex we will discuss this part also.

So, let us see, so this is predicted output and this is a loss function and we would like to minimize the average loss over training set and this is called regularizer or panel, which penalizes the complex models and this is learned weights. So, total loss can be considered as the data loss and the regularization loss. So, data loss is given by the functions l here that is the data loss and regularization loss is given by R(w).

$$w^* = \arg\min \frac{1}{N} \sum_{i=1}^{N} l(f(x_i, w), y_i) + R(w)$$

(Refer Slide Time: 19:39)



So, a loss function tells how good is the current classifier and data loss is the model prediction should match training data, so what is the deviation. So, there are different kinds of examples of data loss functions like in multiclass support vector machine we can use hinge loss. Here, now we can see that each wrongly classified sample produces a loss. So it is the difference of the functional values of the predicted sample predicted function functional value predicted sample and the models sample.

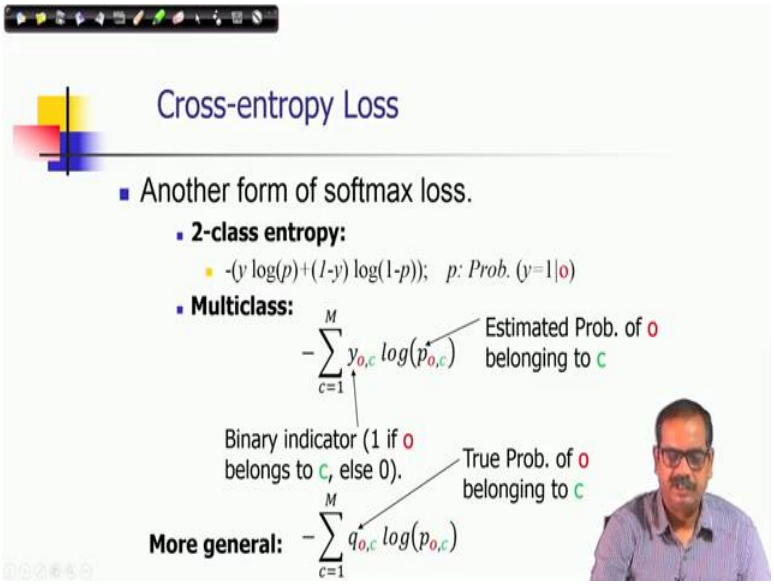
So, you can see that functional form here in this case and the softmax loss or multinomial logistic regression laws that is given in this form. So, in this function we are estimating the probability of a level true level given the input data. So, if this probability is very high then this loss will be less, so using this negation, negative sample we make it a positive function.

Sometimes this probability is computed with respect to the responses of all other classes. So, in the output layer see if you have in the output layer say N classes and the response is provided by say syi of the ith class. Then we consider that see yi is a true class, so we consider the syi of the true response of the true class.

Now, the probability of that class is given by this expression this is the softmax expression you raise it to the exponentiation and somehow normalize it by the sum of all these exponentiations. So, that is that would give you the probability value and it is actually modelling this particular term itself it is giving this probability of y given data this is (Refer Time: 21:55).

$$L_i = -\log P(Y = y_i|X = x_i) \rightarrow L_i = -\log(\frac{e^{y_i}}{\sum e^{y_i}})$$

(Refer Slide Time: 21:57)



So, cross entropy loss is also another form of softmax loss when you have 2-class entropy then you have either say class 1 or class 2 that will object class and background class for example.

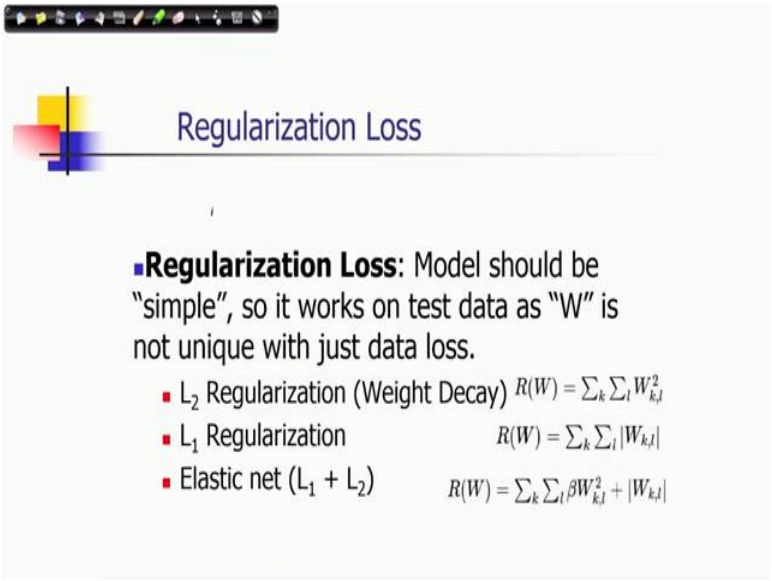
$$-(ylog(p) + (1 - y)\log(1 - p)); p: Prob.(y = 1|0)$$

So, when your object class y is 1, label is 1 then you are only considering if it is predicted as you know y then you are considering the, when the object is true class of the object is; true class of the sample is the object then you are using log(p) otherwise you are using log (1-p).

In the multi class you are extending this particular computations with this expression where you can see that y o, c is a binary indicator it is one if the object belongs to class c. And we are considering only probability of that class for the kind of sample. Here o is the

input sample, so estimating probability of o belonging to class c. In a more general framework you may have a distribution of probability distribution of the true class also, so it is not binary always. So, it is true probability of o belonging to c use this, you can use it for the cross entropy loss.

(Refer Slide Time: 23:39)



So, in the regulation laws we would like to have the model simple and there we would like to have the W, the size of the w as small as possible. And it is not an unique with the data loss, see if there are different kinds of regularization function which tries to put a constraint on the values of the W.

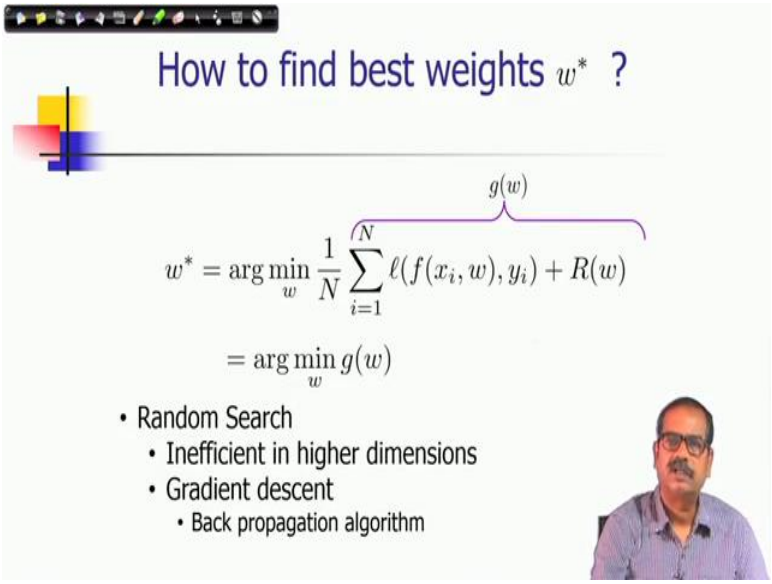
For example, you would like to minimize the

$$L2\ regularization: R(W) = \sum_k \sum_j {W_{k,j}}^2$$

$$L1\ regularization: R(W) = \sum_k \sum_j |W_{k,j}|$$

$$Elastic\ net\ (L_1 + L_2): R(W) = \sum_k \sum_j \beta {W_{k,j}}^2 + |W_{k,j}|$$

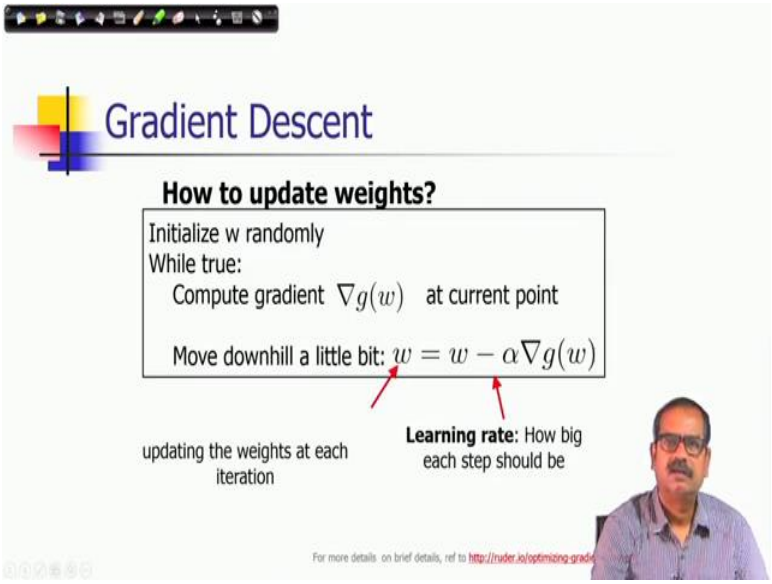
(Refer Slide Time: 24:27)



So, finally, for the training we would like to solve this optimization problem. So, given this loss function which has the given this objective function which has data loss term and regularization loss term and then we would like to minimize with respect to a chosen weight. So, what is the best weight which can minimize it, so this is g(w) is your the sum of these losses and we would like to minimize g(w).

So, what we could have done that we can use random search; that means, once again we need to choose some initial guess of w and or pick here and there and try to see what kind of loss you are getting and try to get the minimum out of them, but it is very inefficient. So, standard algorithm would be gradient descent again the same back propagation algorithm what we can use it here.

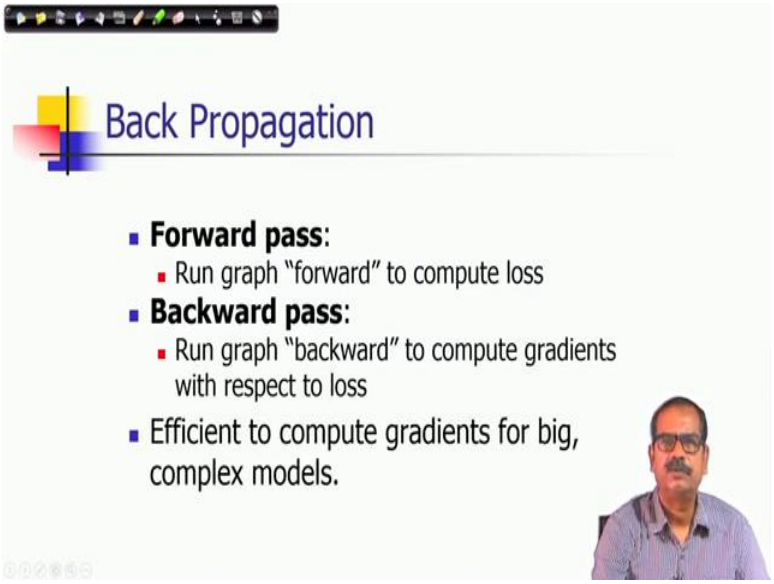
(Refer Slide Time: 25:29)



So, gradient descent algorithm we have discussed earlier let us revisit once again in the context of this particular topic. So, we can initialize w randomly and then we can go ahead computing the gradient of that w at the current point. So, which is expressed as this grad of g(w) and then move down in a little bit because that is updation process.

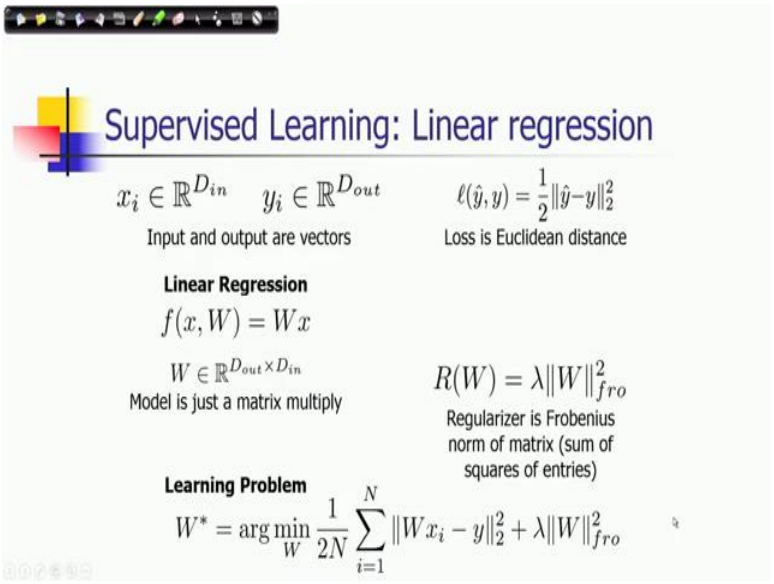
So, move along in the opposite to the steepest increase of the gradient, so which means steepest descent of the gradient value, so $-\alpha \nabla g(w)$ is a learning factorial. So, this is the updating the weights at each iteration and this is the learning rate alpha is the learning rate we get step should be that needs to decided.

(Refer Slide Time: 26:17)



And back propagation algorithm also we discussed that it has two passes, one is forward pass, another is backward pass. In the forward pass we run the graph forward to compute loss and the backward pass we run the graph backward to compute gradients with respect to loss. And we know that it is efficient to compute gradients for big complex models.

(Refer Slide Time: 26:41)



So, supervised learning could be also considered as a linear regression problem where your input is a vector and output is also a vector. So, in a linear regression one of the linear regression problem is that now the model is linear.

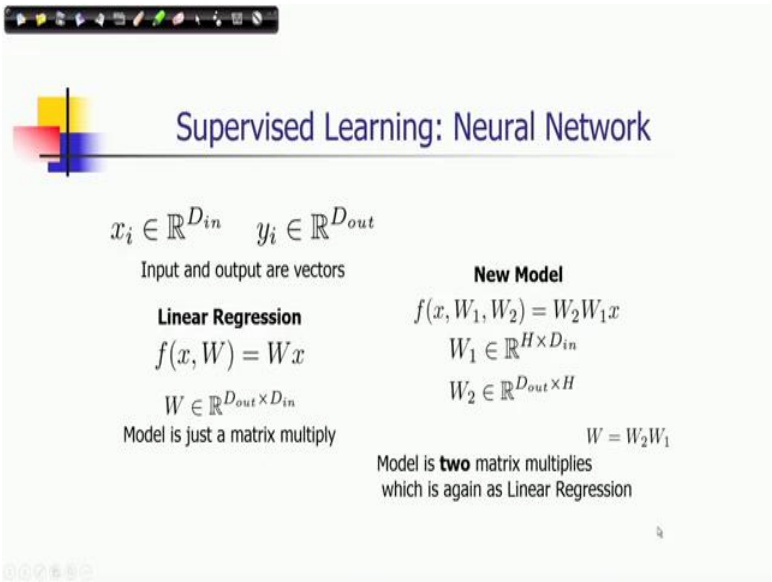
$$Linear\ regression: f(x, W) = Wx$$

$$Learning\ problem: W^* = \arg\min \frac{1}{2N} \sum_{i=1}^{N} ||Wx_i - y||_2^2 + \lambda ||W||_{fro}^2$$

So, the if your input is x, then use the weight linear combination of input that is Wx it is non-linear combination you multiply with any matrix w that would give you another vector and there is a linear model here.

So, dimension of W should be $D_{out}XD_{in}$, because your output vector is of size D_{out}. So, it is just a matrix multiplication that is the model here and you can use the euclidean distance as a loss norm of the deviations. So, you are learning problem here is a linear regression learning problem and coupled with regularization the corresponding objective function as been shown here. And regularization is using frobenius norm of matrix or sum of squares of this entries.

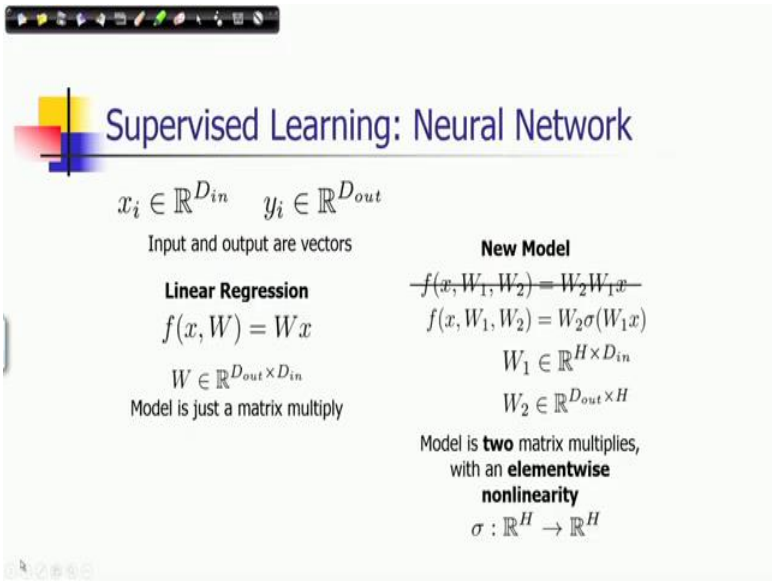
(Refer Slide Time: 27:51)



And supervised learning you can use the neural network where you can consider it is a sequence of linear layers. So, it is a new model once again as you can see that you are learning problem it is almost remaining same, because what you are doing if you are using multi layer neural network. So, first layer W1 since this is linear, next layer also you can multiply W2, but this $W=W_2W_1$, so it remains the same learning problem. So, even if you

increase the layer finally, it is not increasing the capacity of problem solving it remains the same power of your problem solving remains the same.

(Refer Slide Time: 28:37)



So, by introducing non-linearity here, you are giving another complexity to your model and increases the power of solving this problem. So, let me take a break here and we will continue this topic in the next lecture.

Thank you very much for listening this lecture.

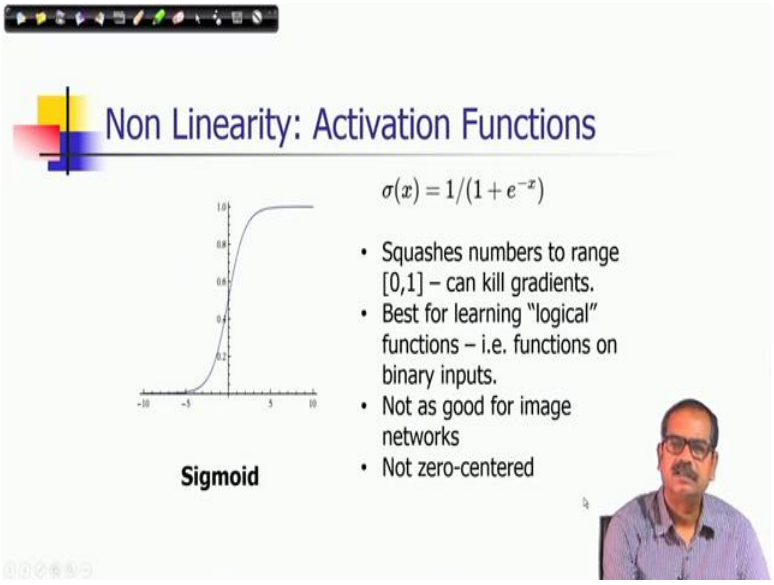
Keywords: Deep architecture, loss function, regularization loss, supervised learning.

Computer Vision
Prof. Jayanta Mukhopadhyay
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

Lecture - 56
Deep Neural Architecture and Applications Part – II

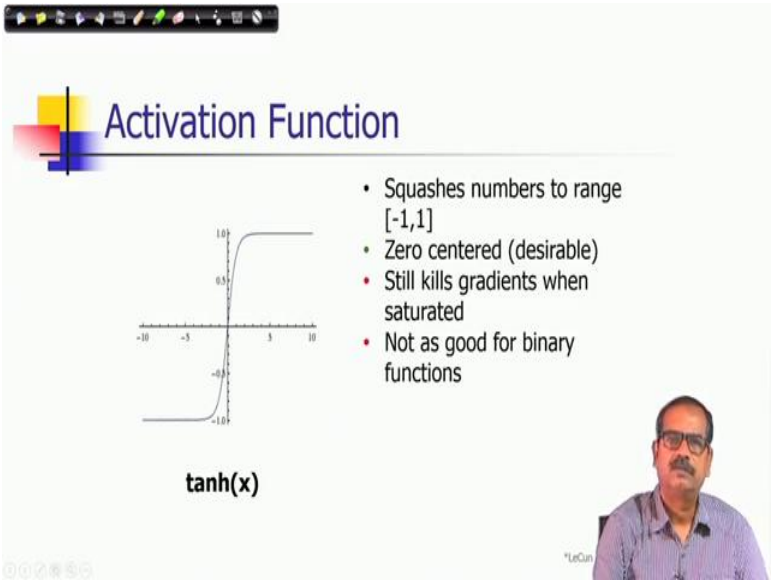
We are discussing about Deep Neural Architecture. And in the last lecture we discussed how the deep computation is different from classical computations. And, then we have considered different types of supervised learning problems or different facets of supervised learning problem in the context of deep learning architecture.

(Refer Slide Time: 00:42)



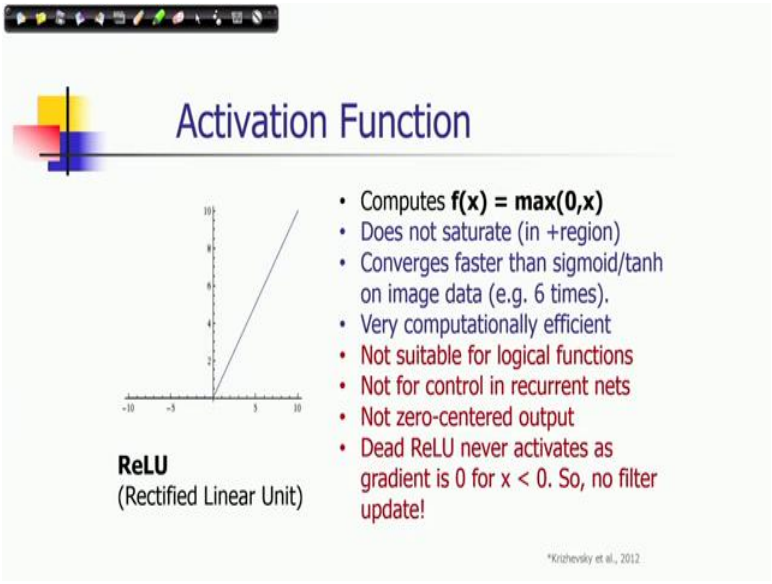
Now, in the neural network there is a nonlinearity function, that we know that is used and those are also called activation functions. And, we have seen particularly the sigmoid function that is used in the artificial neural network very often. But so, the property of the sigmoid function is that it squashes numbers to range 0 to 1 And so, it can kill gradients as you can see it saturates when the value increases or value decreases and they are the gradient becomes almost 0. And, then it is best for learning logical functions that is functions on binary inputs, but it is not good for image networks and it is not 0 centered also

(Refer Slide Time: 01:23)



So, there is another activation function which is called tan hyperbolic x. So, it is also ranging from -1 to 1 and it is also 0 centered, it is 0 centered which is desirable. And, still it has the problem of that it can kill gradients when it is saturated and it is not as good for binary functions.

(Refer Slide Time: 01:49)



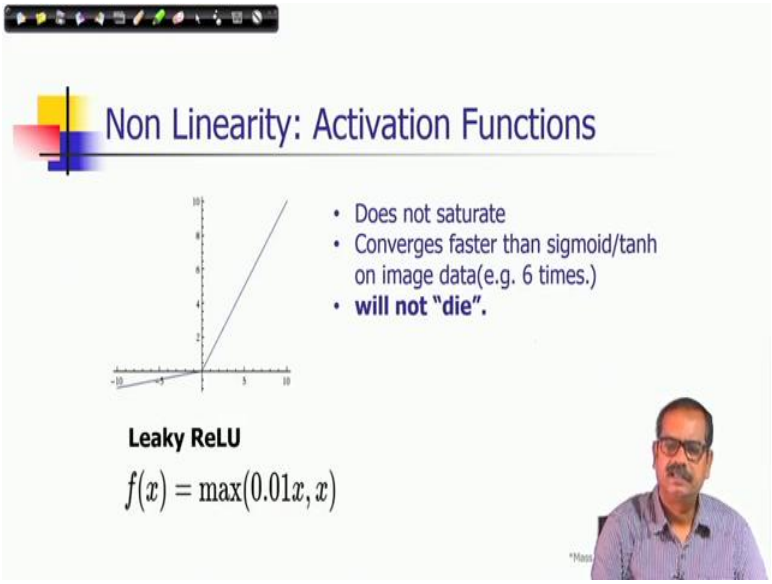
This particular function rectified linear unit, it is very popular in convolution neural network in different neural architecture. So, the functional form has been shown here that is you can see only positive half of the input it is linear and then for the negative half it is

the value is 0. So, that there is a non-linearity and they discontinuity at the value 0 that is first order discontinuities there.

So, but the advantage is that it does not saturate in positive region. And, it converges faster than sigmoid and tan hyperbolic data for example, 6 times. And it is very computationally efficient, but it is not suitable for logical functions for representing modelling logical functions not for control in recurrent nets not 0 centered output.

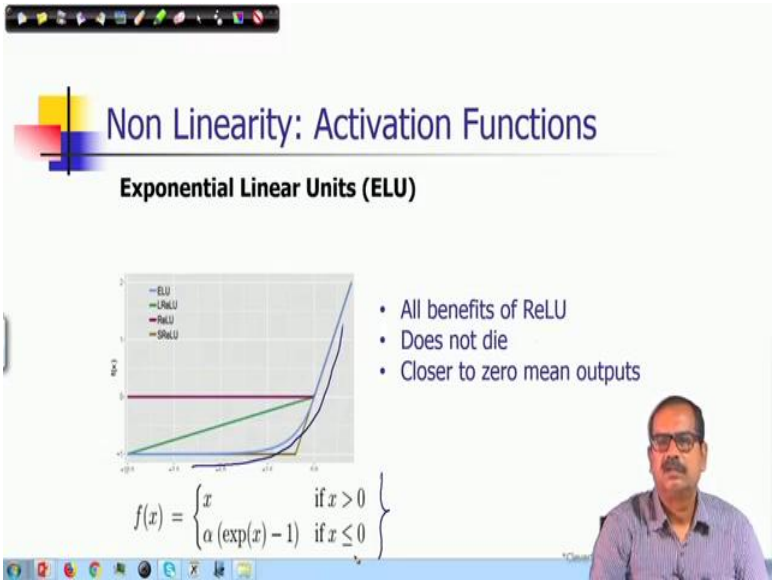
And, if it is going into the red region which is the negative part then it is not going to activate the filters or activate the gradient. So, it stops there so, there is a dead zone in this case.

(Refer Slide Time: 03:00)



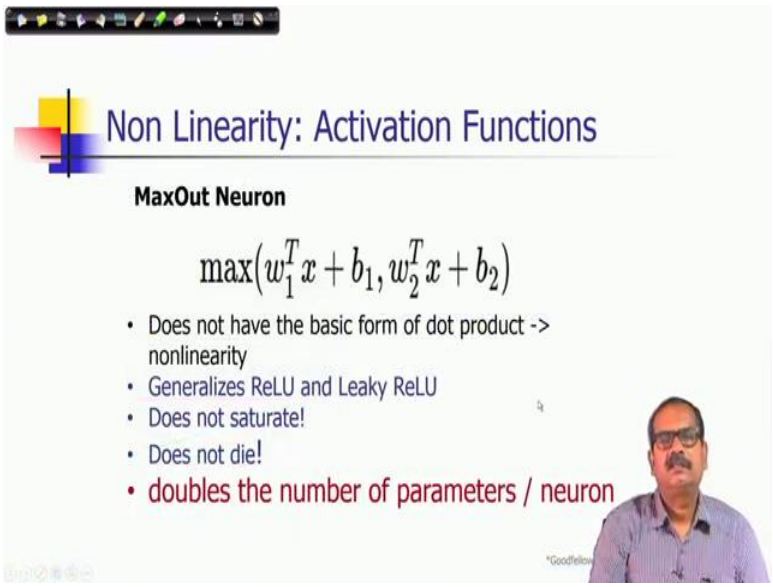
So, to avoid that we can have leaky ReLU so, that you have very small increase in the negative part also the small gradient. So, which it has those advantage that it does not saturate, convergence faster and the gradients will not die.

(Refer Slide Time: 03:20)



Other kind of activation function like ReLU we have also exponential linear units. In this particular graph this blue graph is the exponential linear units which corresponds to this function. So, the benefits of all ReLU benefits are there and it is closer to 0 main outputs.

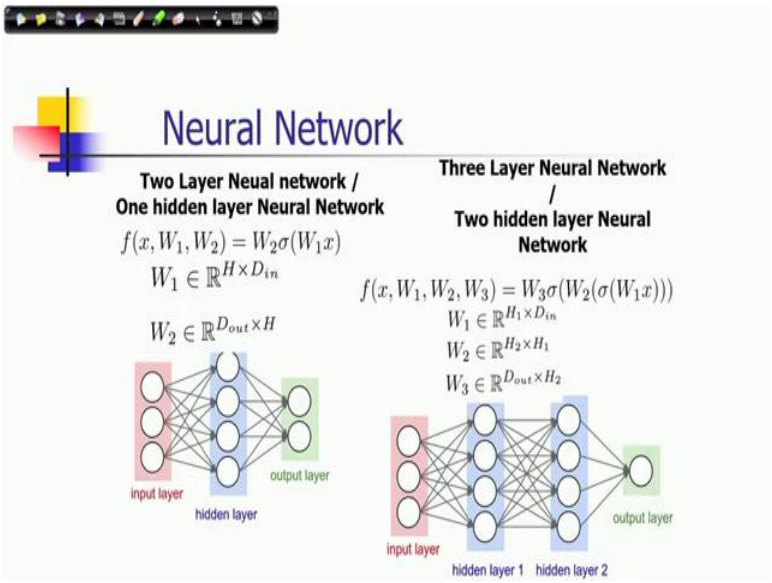
(Refer Slide Time: 03:47)



Another non-linear activation function which is called max out neuron; here it is a maximum of two linear part, maximum of two linear combinations of inputs $w_1^T x + b_1$ and $w_2^T x + b_2$. So, it does not have basic form of dot product of course, it is non-linear

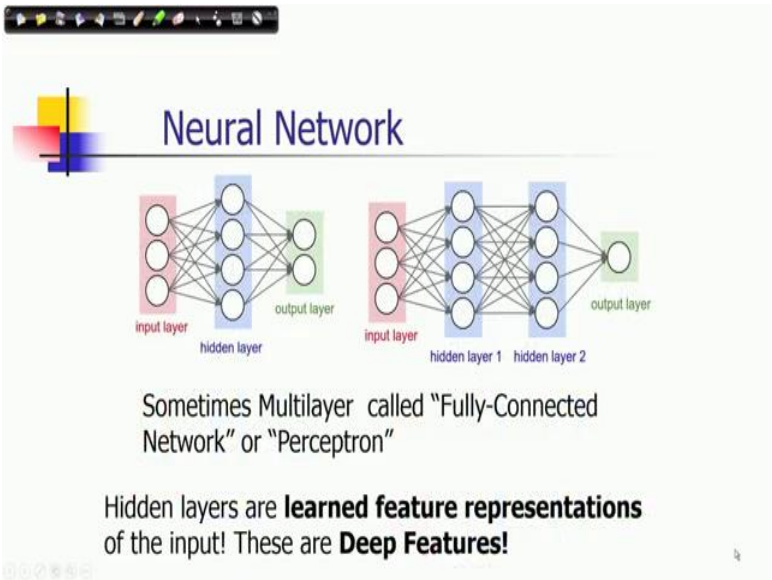
and generalizes ReLU and leaky ReLU does not saturate does not die, problem is that it doubles the number of parameters per neural.

(Refer Slide Time: 04:22)



And, we have seen this is the standard architecture of two one hidden layer neural network or two layer neural network. And, another three layer neural network architecture.

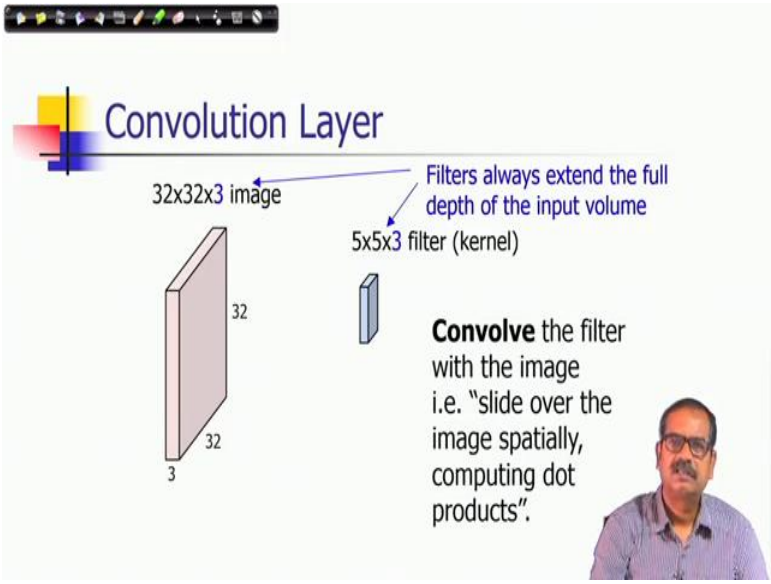
(Refer Slide Time: 04:35)



And sometimes this is called multi layer or fully connected network or perceptron and hidden layers are learned feature representations of the input and these are deep features.

So, with this actually I have talked about general features of neural networks and also deep neural architectures. But, let me discuss the specific a specific deep neural architecture, which is very popular in various applications and which one is convolution neural network.

(Refer Slide Time: 05:05)

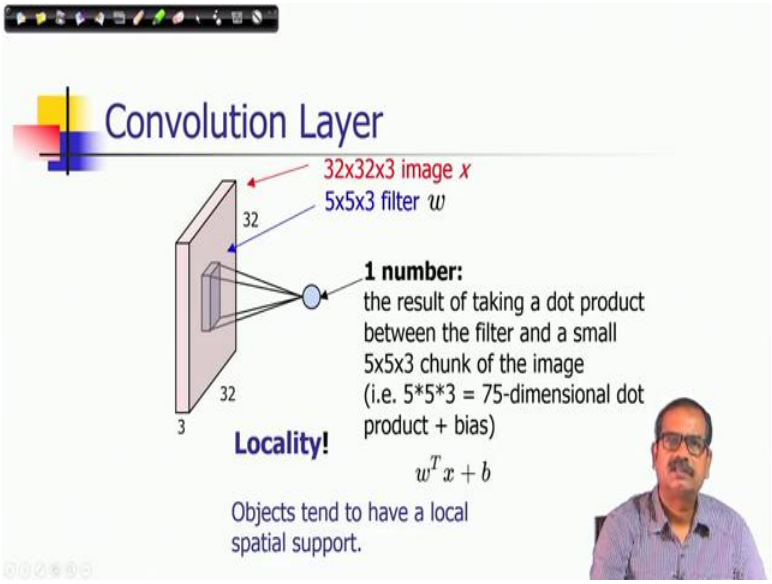


In conventional neural network, the conventional layer that we need to understand that is a hidden layer. And, in the convolution layer what we have that input is an image as a $32 \times 32 \times 3$ image. That means, for example, it could be RGB 3 channel RGB image or it could be any other three components.

And there is a filter so, this filter performs the convolution operations over this image. And size of the filter sometimes it is called kernel also for example, in this size if here it is given a typical value say $5 \times 5 \times 3$. So, first thing the filter extend the full depth of the input volume. So, 3 is the depth in this example.

So, the filter has to span over the depth and then what it does it takes a dot product. So, I will explain that so, it convolves with respect to the image which means this filter slides over every point and computes convolution. That means, a dot products with the input with the filter weights and then produce that output at that at the central location of that central pixel of that mask.

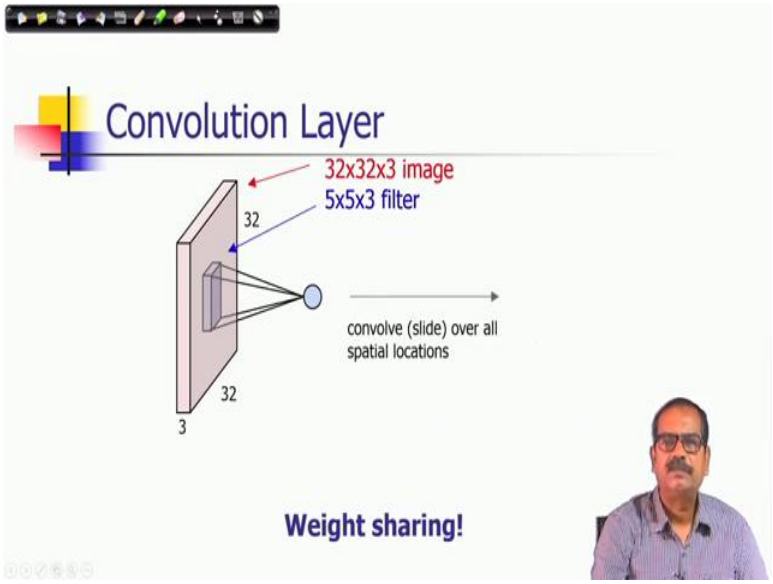
(Refer Slide Time: 06:25)



So, this is the example that you have $5 \times 5 \times 3$ filter w and it is placed within a pixel of image. And then it takes a dot product and then produces a value and this value is the output at that at that point. So, for example, in this case you have $5 \times 5 \times 3$ chunk of the image; that means, 75 dimensional dot product you are doing and there.

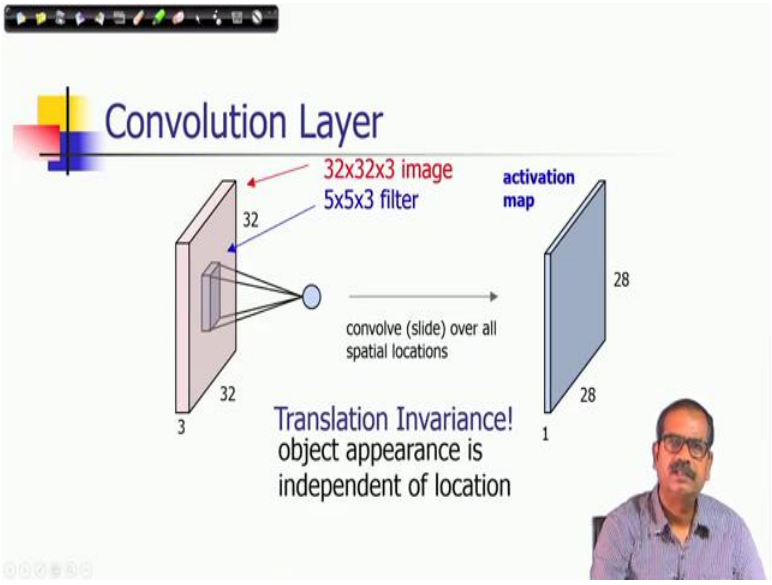
In the neuron as you and as you know that it is the dot product and in the neuron there is a bias term. So, then you add the bias term and that could be also the output of the convolution layer. So, one of the feature of this convolution is that it is the locality which means that objects tend to have a local spatial support. So, this could be exploited using convolution.

(Refer Slide Time: 07:22)



And the other feature is that weight sharing.

(Refer Slide Time: 07:27)

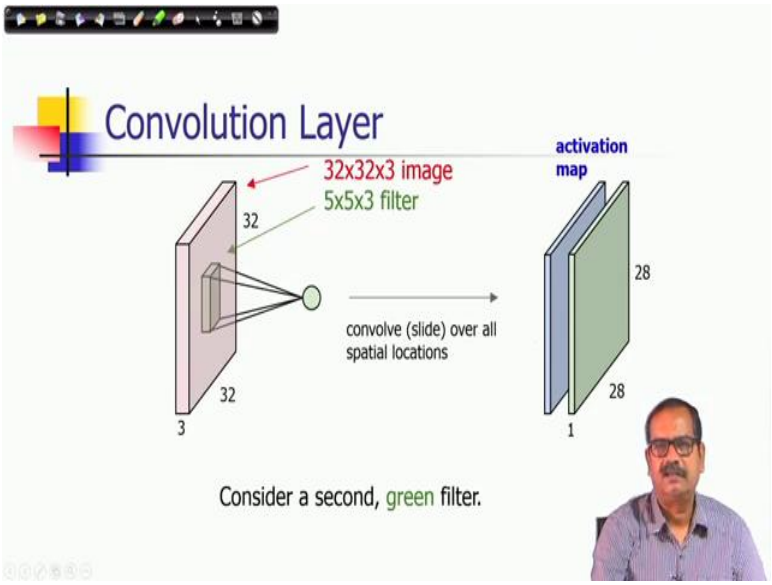


Since, it will be your convolving over all special locations so, the same weights are used. And finally, you are generating the output. So, if we consider that boundary conditions that only pixels, which are fully embedded the convolution mask which is which is fully embedded within the image at those pixels sites. You are only considering output from those pixels sites then the size of the activation map get reduced.

As you can see the filter is of size 5×5 ; that means, along height and along width you have to leave out two pixels each and at each edges. Because, those are not those pixels are not fully embedded. So, finally, size would be 28×28 and along depth wise also other planes are not coming into picture because they are not fully embedded. Only the central pixels convolutions for the central plane those are only considered here. Because those for those pixels only this pool embedding is available.

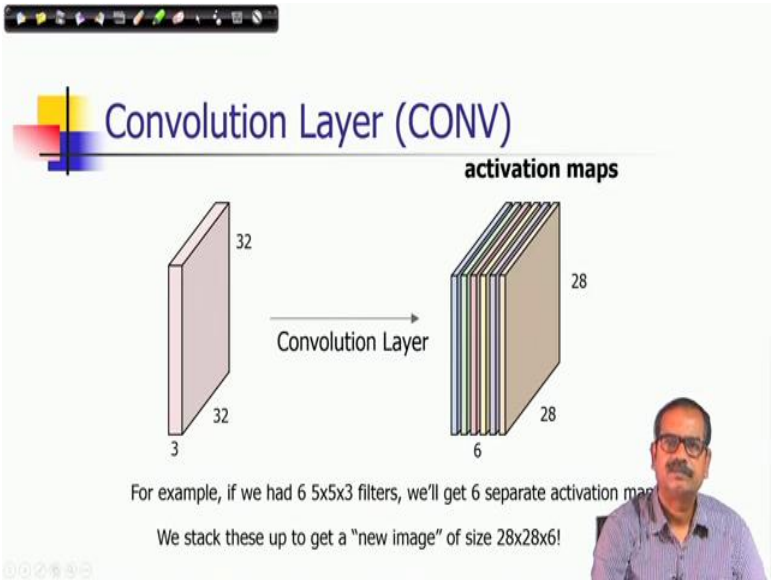
Say another feature of this computation that it is translation invariance which means the object appearance is independent of location.

(Refer Slide Time: 08:43)



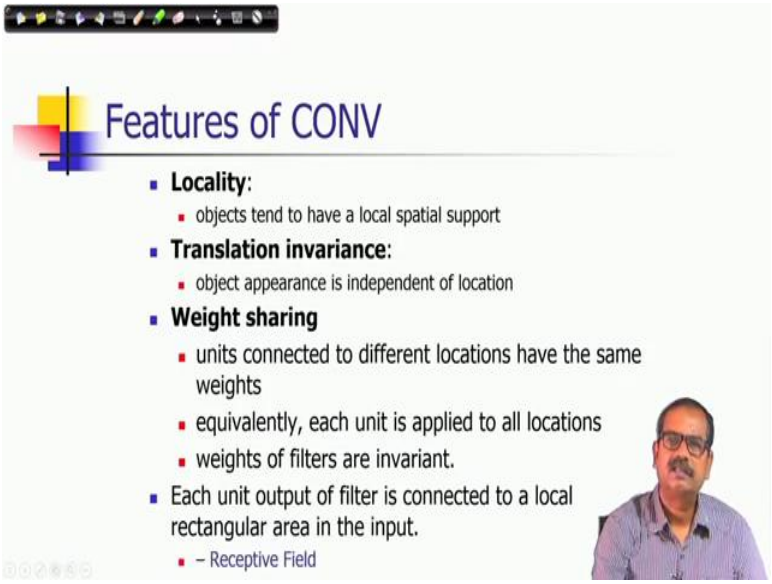
Now, you consider that there are other filters. So, there is another second say green filter of the same size, but its weights could be different. And, it also produced another we call it channel output. So, it also produce another output.

(Refer Slide Time: 08:59)



And, if there are 6 such filters, you have 6 such activation maps six separate activation maps. And you can stack this to get a new image of size $28 \times 28 \times 6$. So, 6 number of convolutional filters.

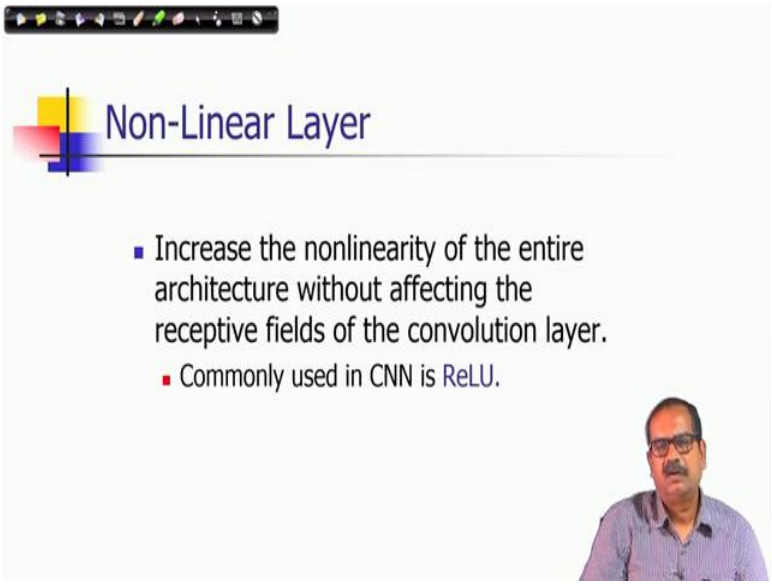
(Refer Slide Time: 09:20)



So, the features of CONV layer or convolution layer we can consider that is locality, which is the feature of having local spatial support for objects. Then translation invariance where object appearance could be independent of location; and weight sharing which means the units could be connected to different locations having the same weight.

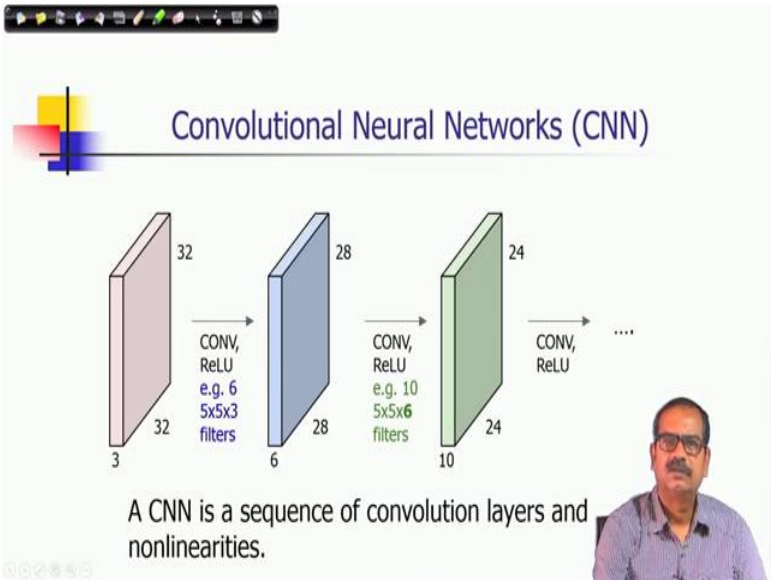
And equivalent each unit is applied to all locations and weights of filters are invariant. This keeps the number of parameters small or convolution neural network. Of course, since there are many such layers finally, total number of parameters will be quite large. Then each unit output or filter is connected to a local rectangular area in the input and that is considered as the receptive field.

(Refer Slide Time: 10:13)



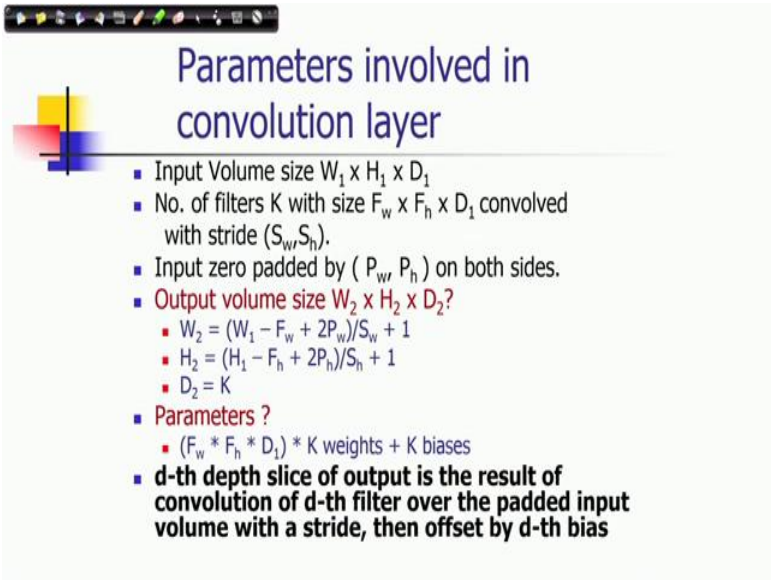
So, as there is a non-linear function in each neuron. So, after convolution layer we consider the that the non-linearity at every point itself is considered as a layered response from the system. Whose input is the output of the convolution layer then it goes through the non-linear stages at every pixel. So, it is the point wise non-linearity and it increases the non-linearity of the inter architecture without affecting the respective fields of convolutional layer, which generalizes the model further.

(Refer Slide Time: 10:50)



And, ReLU is commonly used that I have mentioned. So, what is a CNN it is a sequence of convolution layers and nonlinearity. So, you have a this is your input you have the convolutional layer and also the ReLU this is one layer. Then another layer of convolutional layer and ReLU second layer and it goes on. So, CNN is sequence of such convolution layers and nonlinearities.

(Refer Slide Time: 11:23)



So, let us also see what are the parameters involved in convolutional layer. Consider your input is of size $W_1 \times H_1 \times D_1$. So, in my previous example we have taken $32 \times 32 \times 3$

that was in size of the input. So, value of W_1 was 32, H_1 was 32 and D_1 is 3. And considered there are K, number of filters in the previous example this value of K was 6; we considered 6 filters. And size of the filter is $F_w \times F_h \times D_1$.

So, in the previous example we have taken $5 \times 5 \times 3$ filters, which means F_w was 5; F_h was 5 and D_1 is kept 3. So, you note that the depth wise the filter has to have the same number of channels what you have in the input or same number of depth slices, but is there in the input. Because the idea of convolution layer is that the filter should be totally embedded within the input image.

And that to at the central slice and that is why the depth has to be the same. And they need to provide only single channel output. And then there is another parameter called stride. So, it is considering that the point where you are performing convolutions. So, how the point could be separated in the grid? So, if they are adjacent then stride is 1; that was the previous case.

Suppose you leave 1 out of them then the stride would be 2. And also input can could be 0 padded on both sides to keep the size same. So, if you would like to include also the edge pixels or boundary pixels of the image, then pad the input to 0 and then perform this computation. And it would produce the output volume size of $W_2 \times H_2 \times 2$ So, what should be this output volume size? Now these values are all related with the values of input sizes.

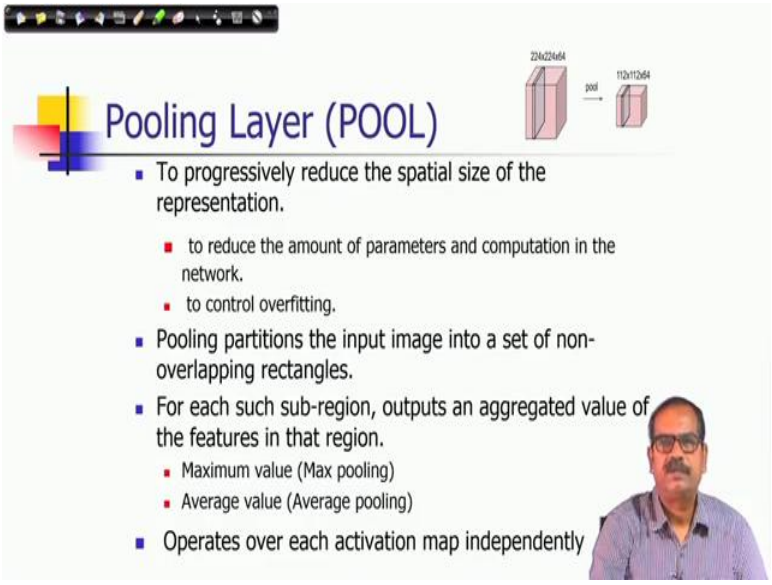
Like values of in $W_1, H_1, D_1;;$ F_w, F_h, D_1, D 1 is not very important here, but W_1 and H_1 F_w, F_h, S_w, S_h and P_w, P_h. For all those values they will determine what should be the size of your output. For example consider the value of W_2 and as this is the expressions. So, it has taken care of striding it has taken care of 0 padding, it has taken care of the size of the filter and complete embedding of the filters within the input.

So, you observe that the depth number of depth slices should be the number of filters K here. And what could be the number of parameters? Since the parameters involves say the weights of the filters and since there are the size of the filter is $F_w \times F_h \times D_1$. So, that many number of weights should be there.

So, it is $F_w \times F_h \times D_1$. And if there are K filters you have to multiply with K. So, we are assuming here that in the convolution layer all the filters of same size and all the filters of same size. So, the parameters as you can see $F_w * F_h * D_1 * K$ that many weights and each neuron could have a bias so; you can have K biases.

So, your summary of this particular operations is that d th depth slice of output is the result of convolution of d th filter of the padded input volume with a stride. And then offset by d th bias.

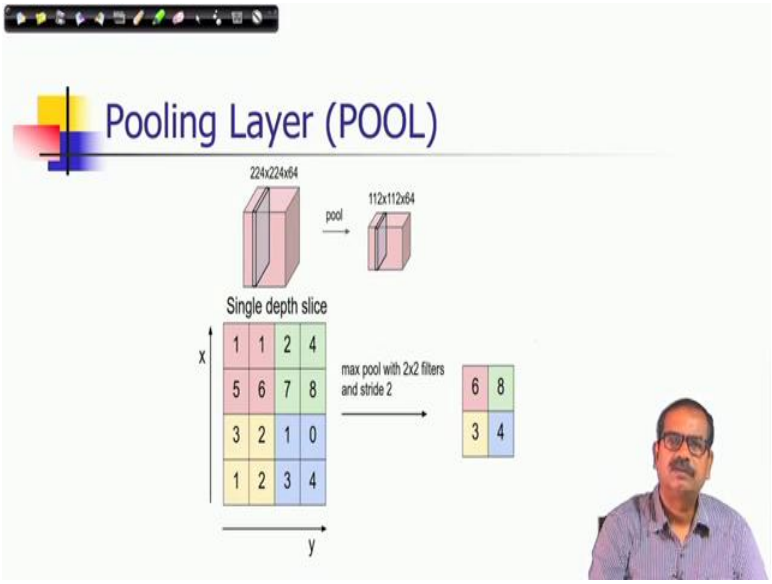
(Refer Slide Time: 15:28)



There is another kind of layer in CNN, which is called pooling layer. So, in the pooling layer it tries to progressively reduce the spatial size of the representation that is it tasks. So, that it can reduce the amount of parameters and also computation in the network it controls overfitting. And pooling partitions input image into a set of non overlapping rectangles.

So, for each sub region outputs an aggregated value of the features in that region. There are two types of aggregation one is max pooling, which considers maximum value. The other one is average pooling which takes average value. So, it operates over each activation map independently.

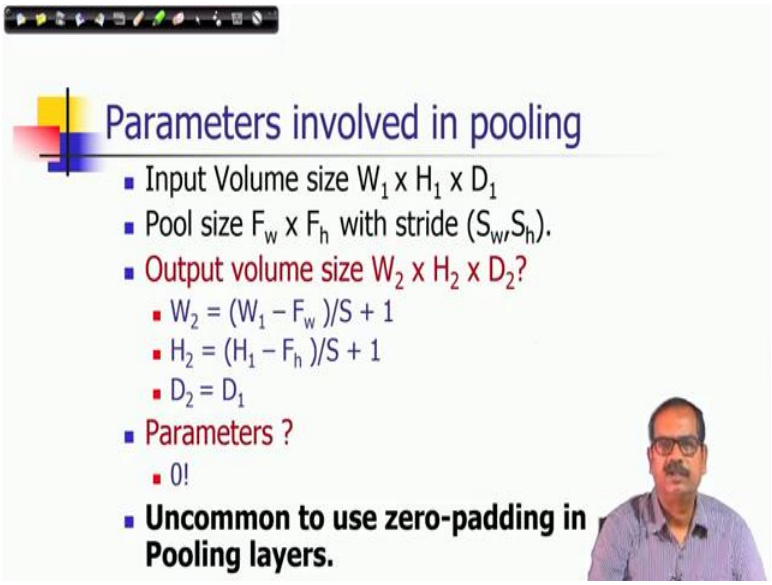
(Refer Slide Time: 16:13)



So, this is an example of pooling layer, you can see that the input was $224 \times 224 \times 64$ doing max pooling with 2×2 filters and stride 2 your living one sample in between. That is a stride 2 and that is how the input size output size half of the input size. So, you have 224×224 output would be 100×112, but the number of depth remains the same.

And the example here it is showing max pooling first we have partition the input into 2×2 . Rectangles into rectangular sizes blocks and from there you are choosing the maximum value. And you are replacing you are forming your one pixel for each block.

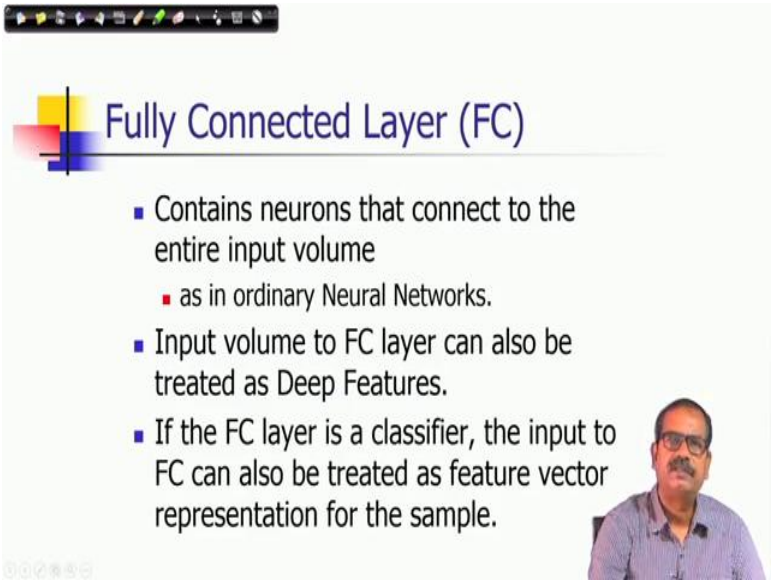
(Refer Slide Time: 17:01)



So, parameters which are involved in pooling let us observe that. So, here input volume size is once again $W_1 \times H_1 \times D_1$. And pool size could be $F_w \times F_h$ with stride (S_w, S_h). So, output volume size is $W_2 \times H_2 \times D_2$ and that is related with input parameters. So, W_2 would be $\frac{W_1 - F_w}{S_w} + 1$. So, that is why that is what you are doing. And H_2 is $\frac{H_1 - F_h}{S_h} + 1$. So, that is that is the H_2 and your number of depth remains the same as the input.

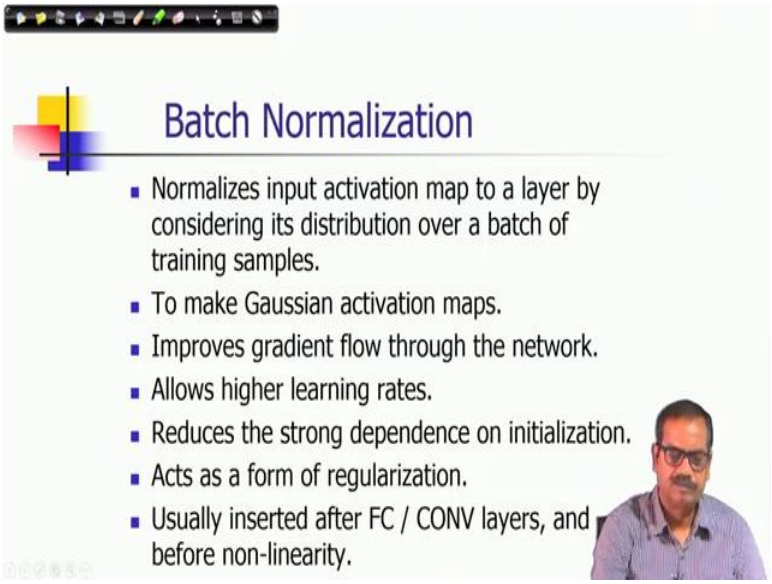
What about number of parameters as you can see? There is no weight no bias nothing you are it is without when without specifying any things you can perform this computation. So, actually there is no parameter involved in this operation. It is just the computations that is providing you the output of the here. It does not depend upon any parameter. And, it is very uncommon to use zero-padding in a pooling layer.

(Refer Slide Time: 18:12)



Then we will discuss about the fully connected layer which is also coupled with the convolutional layers or CNNs. So, it contains neurons that connect to the entire input volume that is why it is fully connected as it is there in ordinary neural networks. And input volume to FC layer can also be treated as deep features. So, it is either it could be deep features or it could be the feature representation itself for a classifier. So, these are the two options for FC layer.

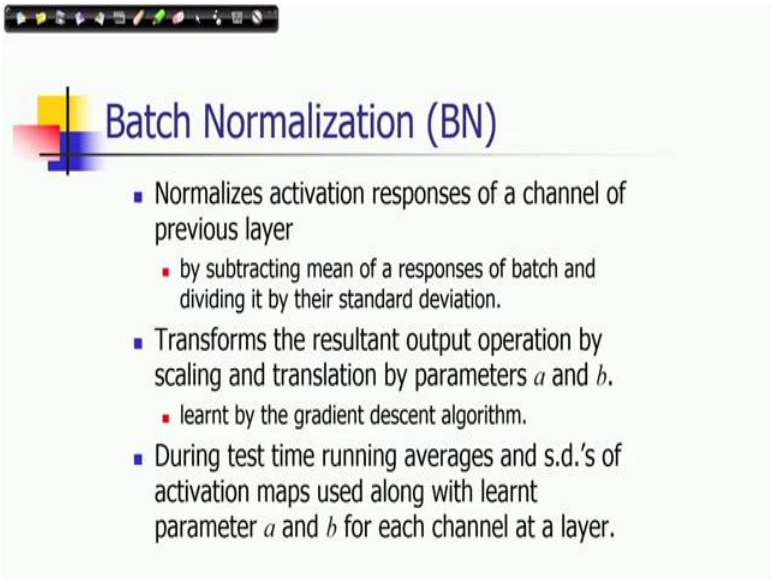
(Refer Slide Time: 18:42)



There are various operations, which are required for efficient training and testing of CNN or for efficient computations of CNN effective computation of CNN. One of the operation is known as batch normalization. So, it tries to condition the input and also the intermediate responses. So, what it does? It normalizes input activation map to a layer by considering its distribution over a batch of training samples.

Consider for example, you can apply your Gaussian activation maps Gaussian model there. Using batch normalization you can improve the gradient flow through the network and it allows also higher learning rates. So, your convergence becomes faster. And it is, it reduces the strong dependence on initialization this also acts as a form of regularization of the network. And, batch normalization is usually inserted after fully connected or convolution layers and before non-linearity.

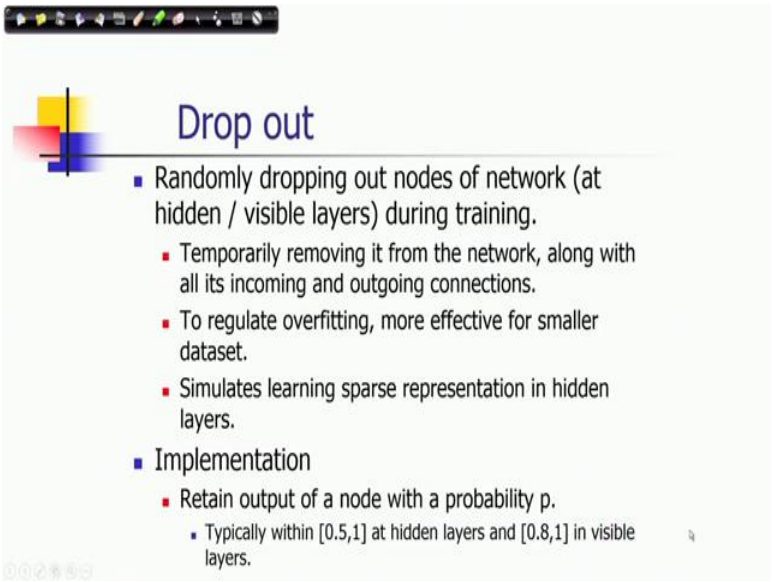
(Refer Slide Time: 19:46)



So, let me elaborate a little bit about this computation. As I mentioned it normalizes activation responses of a channel a previous layer. And how does this normalization work? It subtracts mean of responses of a batch and divides it by their standard deviation. And transform the resultant output operation by scaling and translation by parameters a and b. In fact, this is also learned by the gradient descent algorithm.

So, during test time running averages and standard deviations of activation maps are used with the learned parameter for each channel at a layer.

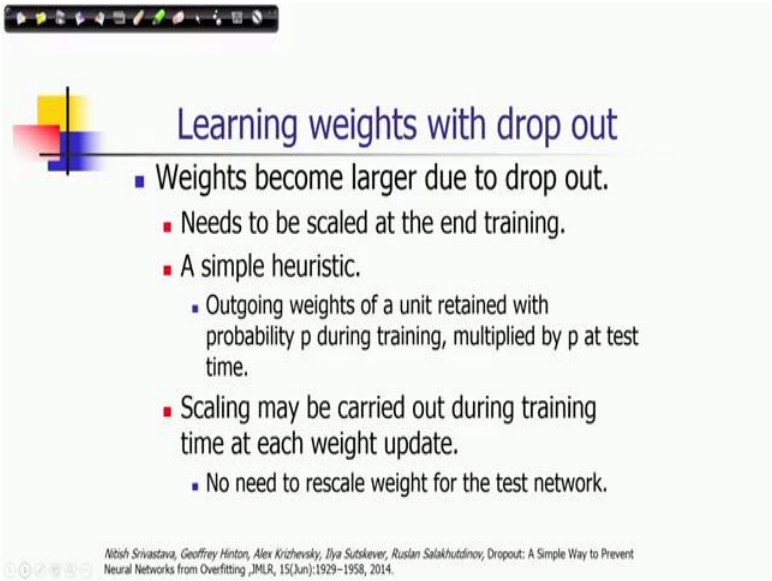
(Refer Slide Time: 20:25)



So, that is how the batch normalization is carried out in CNNs. There is another operations which is also quite common and which improves the generalization of the model and that is called dropout. What it does? It randomly dropout nodes of network at hidden or visible layers, it could be hidden and visible layer during training.

So, dropout means it temporarily removes that node from the network along with all its incoming and outgoing connections. And, it regulates overfitting and which is more effective for smaller dataset it simulates learning sparse representation in hidden layers. So, implementation of dropout to could be in this way that, you can consider that you can retain an output of a node with a probability p. And, typically the value lies between 0.5 to 1 at hidden layers and 0.8 to 1 in the visible layers which means input or output layers.

(Refer Slide Time: 21:32)



So, weight becomes larger due to drop out that is one of the effect of dropping out that whatever weight you compute, you learn now the values could be actual value would be larger effect would be larger and you get large weight. So, you need to scale down that weight at the end of the training. And, there is a simple heuristic that if you are outgoing weights of unit is retained with probability p during training.

Then you should multiply by p at the test time or you can consider these operations during training time itself at each weight update. And, then you do not required to do it on during testing. So, you can use the same weight as you learn. So, with this let me take a break. And, we will continue this discussion in the next lecture, where we will be

discussing different types of convolutional neural networks, different kinds of architectures in my next lecture.

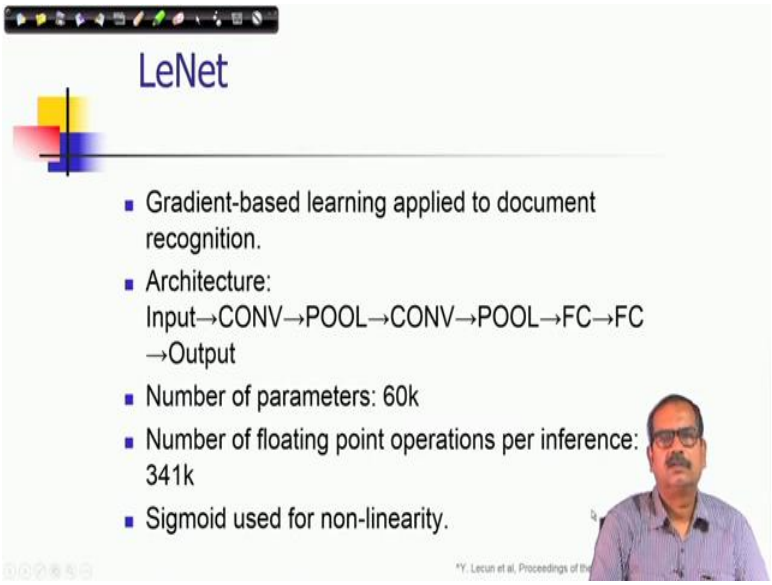
Thank you very much.

Computer Vision
Prof. Jayanta Mukhopadhyay
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

Lecture - 57
Deep Neural Architecture and Applications Part – III

So, we are discussing about convolutional neural network and in this lecture I will provide some examples of CNN architectures or Convolutional Neural Networks which are used for different particularly for image and object classification.

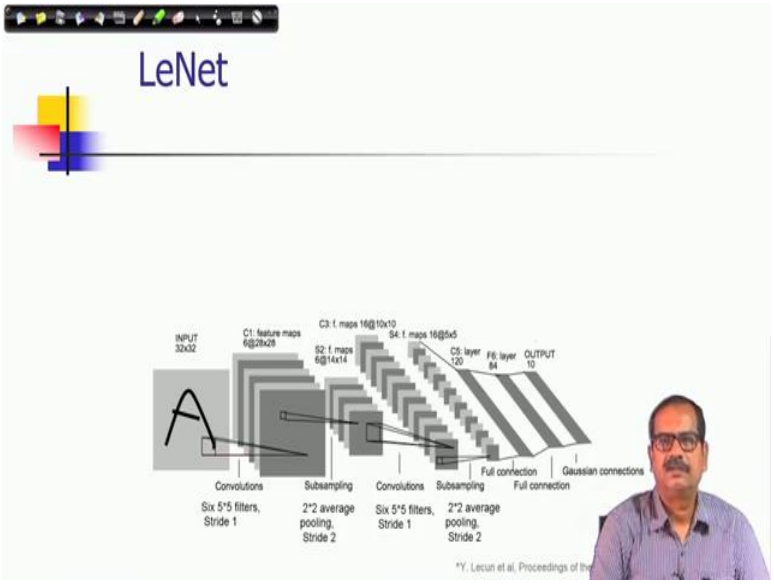
(Refer Slide Time: 00:32)



So, the first; the very first network which was introduced for as in the applications of image classifications in fact, in document image processing, document image processing or character recognition this is called LeNet. And, it is a gradient based learning applied to document recognition as you can see and this architecture uses two convolutional layers, there are two pooling layers, two fully connected layers.

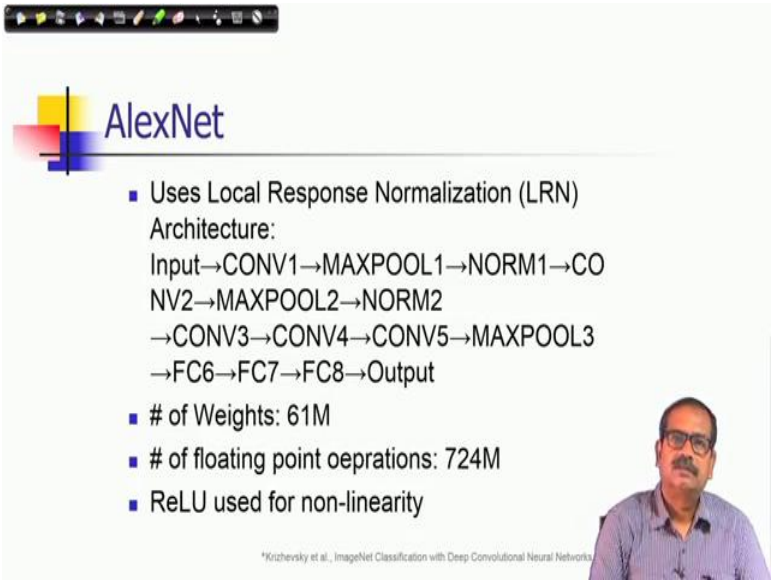
And, it was reported in 1998, it was quite long back and it is a quite a pioneering work in that sense. And, its number of parameters it was around 60,000 and number of floating point operations per inference was 341 k; that means, about 341000 and it uses sigmoid as a nonlinearity function.

(Refer Slide Time: 01:38)



So, this is a diagram which shows schematically how this architecture looks like. So, you can see that there are convolutional layers and then you have the sub sampling which means it has the pooling layers and as you discuss there are two convolutional layers, two pooling layers and there are two FC layers and finally, you get the output, out of this process.

(Refer Slide Time: 02:08)

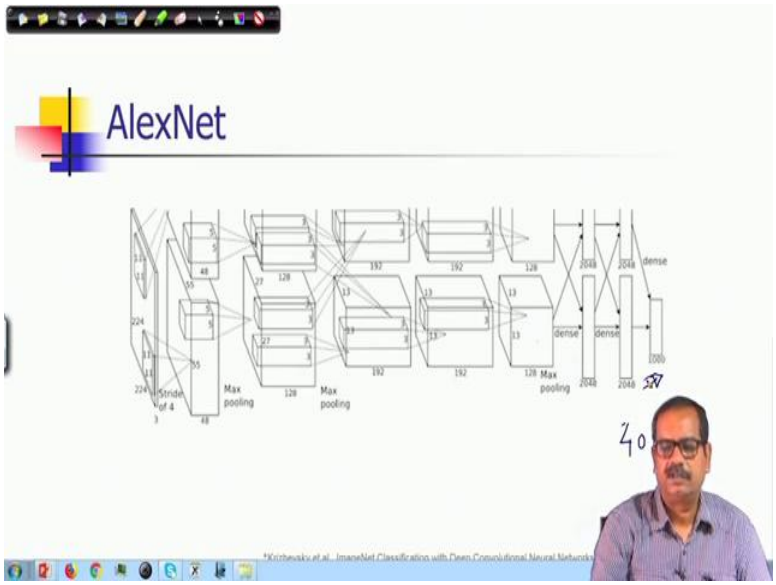


Then in 2012 in fact, the another this network is used and this has provided very good result for the image net classification and this is called AlexNet. So, in this network it

uses a normalization of the responses that local response normalization and its architecture it consists of 5 convolutional layers and you can have it has also you know 3 pooling layers. And, you can see that the first pooling layers are adjacent to their convolutional layers, but the last pooling layers it has it operates after processing through after getting the process data through third, fourth and fifth convolutional layers; that is what it is designed, that is how it is designed.

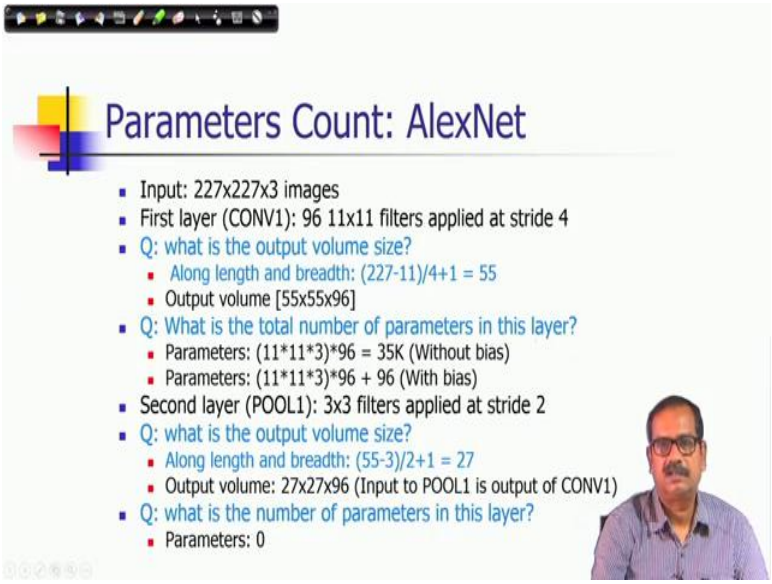
And, there are normalization operations after first and second convolution pooling layers and there are three fully connected layers from which the output is and then finally, there is an output layer. So, the number of weights in this case was 61 million and number of floating point operations is 724 million. So, it is a very large network compared to very primitive network of LeNet earlier what has been reported and here they have used ReLu as the non-linearity, that is one of the major change of non-linear functions in neural network.

(Refer Slide Time: 03:42)



So, a schematic diagram of AlexNet has been shown here, it also shows some of the information related to the size of the input say; it processes 224×224 inputs. It takes filter sizes of say 11×11 and then it uses max pooling of 5×5. Finally, you can see that the feature representation what you were waiting 4096 is the dimension of the feature representation and that is finally, classified for the 1000 notes. So, this is how this classifier is used.

(Refer Slide Time: 04:28)



So, if you would like to count the parameters of AlexNet let us consider how the parameters could be counted. See consider the input image size of $227 \times 227 \times 3$ images that is the input size, it was not 224. It is $227 \times 227 \times 3$ and the first layer convolution layer there are 96 filters each of size 11×11 and it is applied with a stride 4. So, the gap between two samples in the input pixels so, that stride should be 4. it should leave 3 samples and then the fourth sample it should operate with the convolution operation.

So, what should be the output volume size? So, as we know that it is related to with their dimension of input width and height and also the filter size. So, along length and breadth you can see that it has to be the $227 - 11$, 11 is the filter size and divided by 4. So, that is how the dimension and then you add with 1; so, because of the central you can add with 1. , this is the expression we have discussed earlier also. So, the 55 would be the size of the length and size of the breadth and your depth is the number of filters that is 96. So, the size should be $55 \times 55 \times 96$ so, that is that should be the output volume; it is $55 \times 55 \times 96$.

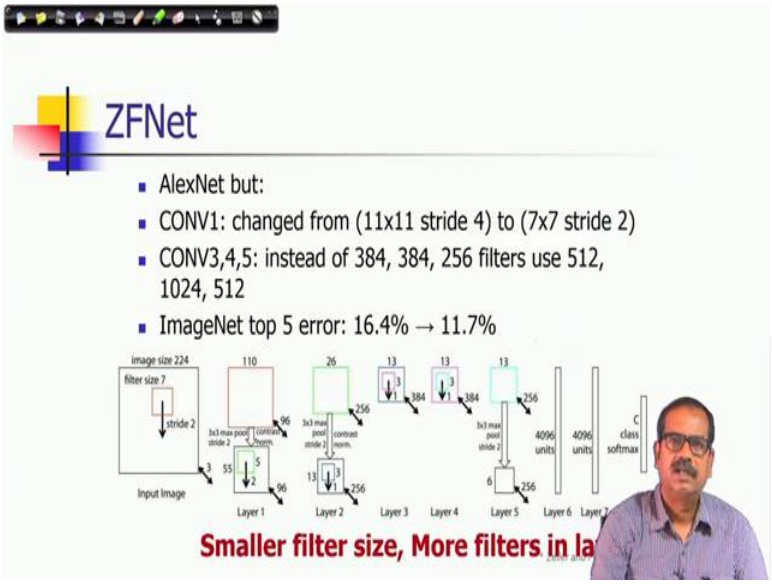
Similarly, what should be the number of parameters in this layer? So, the number of parameters involves the filter weights and also biases. So, since there are 11×11 filters that is a size $11 \times 11 \times 3$ there is a filter size. So, number of parameters will be $11 \times 11 \times 3$ for 1 filter and since there are 96 filters, it should be $96 \times 11 \times 11 \times 3$. There is a

total number of parameters that is without bias and if I consider each should have a bias; so, for each filter there would be one bias. So, you have to add another 96 biases.

So, it should be 90 this number which is around 35 K around 35 K or 35000, there is approximate number and that should be again plus 96. Then consider a second layer that is POOL 1, it has 3×3 filters and which is applied at stride 2. So, what should be the output volume size in this case? So, once again 55 minus filter size 3 divided by the stride 2 and plus 1 I.e., $\frac{55-3}{2}+1$. So, output the length and breadth should be 27. So, output volume would be $27\times27\times96$ because the input depth of the input was 96, it remains the same.

And, that is what is your output volume and, what is the number of parameters in this layer? And, as we mentioned that in pooling layer there is no parameter involved, you do not require any weight or bias. So, number of parameters would be 0. So, in this way you can count parameters by knowing the size of the filters and you can count the you can also determine the size of the output by knowing the size of the filter and size of the input and number of filters that is also important.

(Refer Slide Time: 08:35)

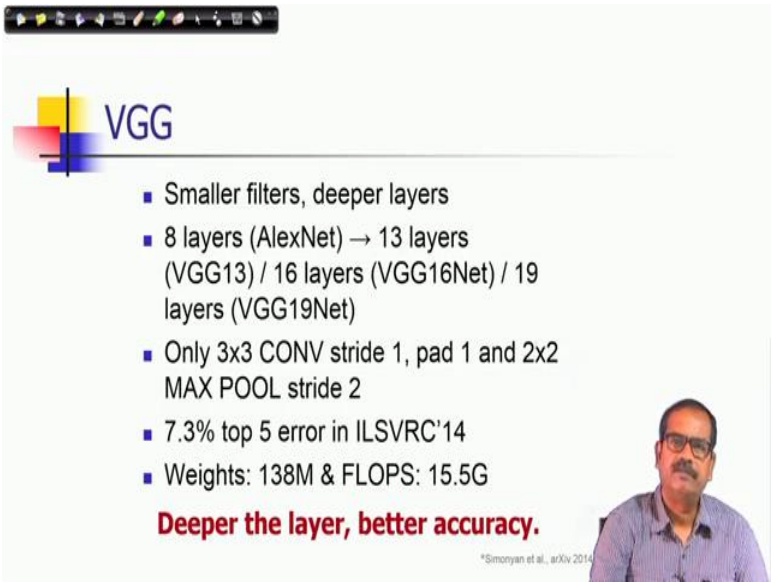


Other network that is also quite distinct that is ZF network, it is I am basically providing you some kind of historical evolution of CNNs. So, ZF network came from Alex network, the changes this modifications. So, what it does? It uses filters of smaller size.

So, earlier it was 11×11 and stride 4, in ZF network they use 7×7 stride 2. And, but what they do also they have used more number of filters say for example, in CONV 3, 4, 5 in AlexNet you have 384, 384 and 256 filters.

In ZF net you have 512, 1024 and 512 and it has improved the accuracy of the image classification for the ImageNet data set. So, this is one that is smaller the filter size it is smaller; in number filter size is smaller, but more filters in layer is used. A schematic diagram is shown here, it shows the size of the different processed output and also the size of filters and other things.

(Refer Slide Time: 09:57)



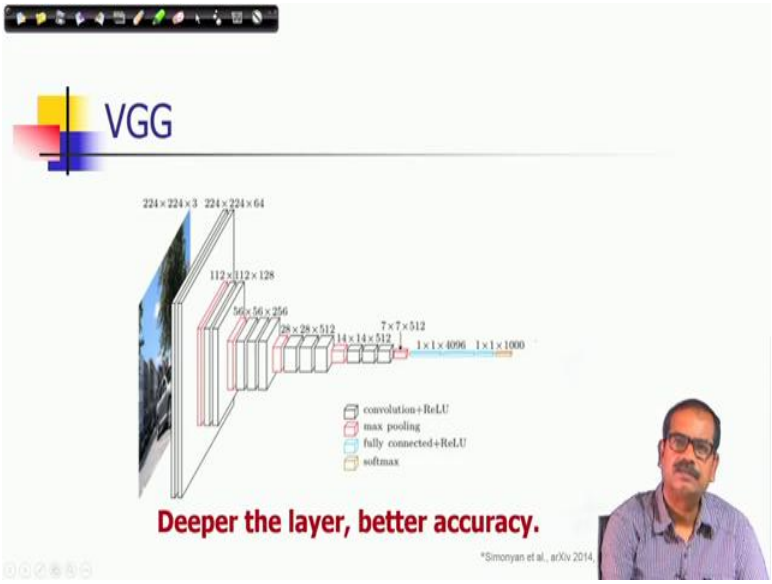
Next you know evolution of this CNN architecture, next stage was VGG and the here in this architecture you have a very, you have a good number of deep layers. So, the idea is that you should have smaller filters, smaller even then the ZF net and you should increase the number of layers. So, in ZF net you have smaller filters more number of filters, it is not only more number of filters also increase the more number of layers. And, by doing that they could increase the, performance of the image classification using the same data set whatever is referring.

So, in this case VGG net it uses different kinds of number of layers in architecture. There are 13 layers which is called VGG 13, there are 16 layers which is 16 or 19 layers which is called VGG 19. And, only 3×3 convolution filters are used with stride 1 pad 1 and

2×2 max pool with stride 2. So, the filters specification is very simple for VGG network. So, here the observation is the deeper the layer better was the accuracy.

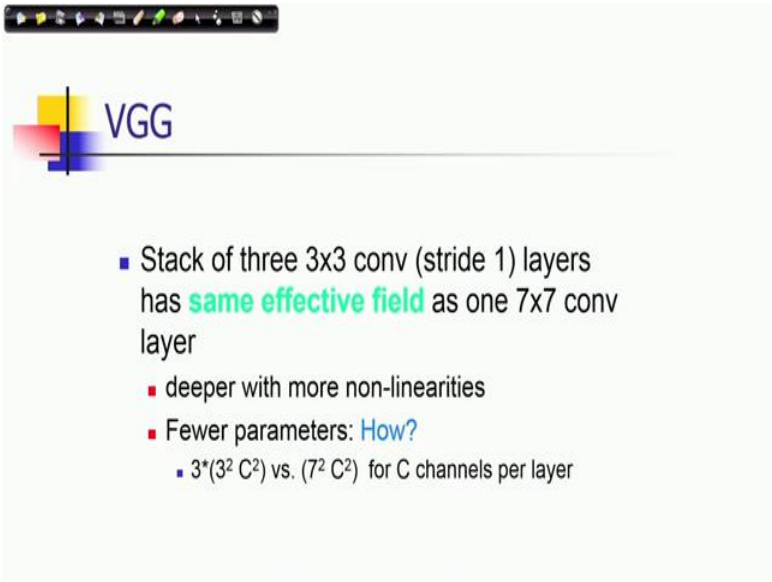
In this particular VGG network, suppose it is I am not sure which number it is in a typical VGG network; it is 138 million number of weights and number of floating point operations for each you know influencing it is about 15.2 billion floating point operations.

(Refer Slide Time: 11:48)



So, one example of VGG network, it has been shown here you can see that typical sizes of input which starts from $224 \times 224 \times 3$ and then it down samples still 7×7. And finally, the feature representation goes towards the $1 \times 1 \times 4096$ and then you use the soft max probabilities, you use the output layer 1000 class classification.

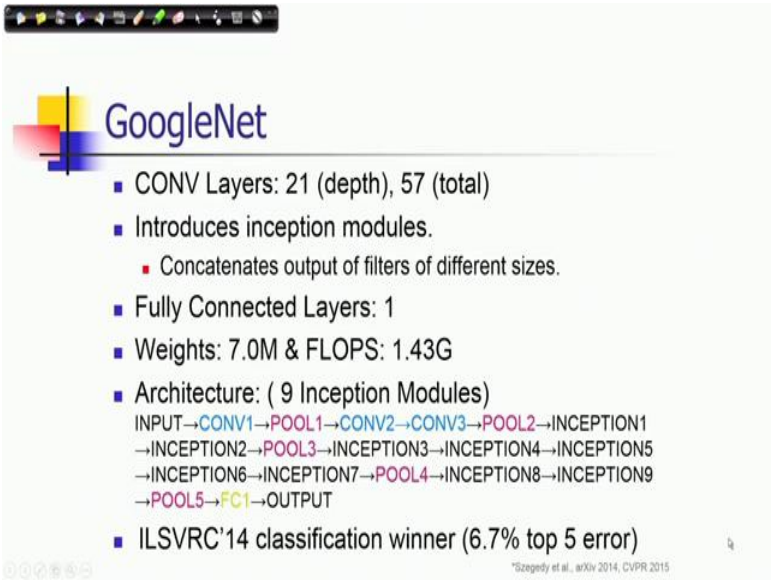
(Refer Slide Time: 12:21)



It has been observed in or it has been found in the VGG network that stack of 3×3 convolutional layers has the same effective field as one 7×7 convolutional layer. So, successively if you use 3×3 convolutional layer let me explain that. So, you have an image. So, apply 3×3 convolution with this image, then you produce another image, then you apply against 3×3 convolution. In this way successively you do it, the effect would be that same as if you have 7×7 convolution at a single stage and so this is one advantage.

So, that is why you have so many number of layers, but you are having the same effect and you have more nonlinearities introduced. And, that would give you the fewer parameters also, because you know if you are using this stack of three then you get fewer parameters. Like for this case you have say $3*(3^2C^2)$ that many number of parameters should be there; if you assume C channels per layer C is a constant whereas, for 7×7 filter it is (7^2C^2). So, that number is large compared to $3*3^2$ which is 27.

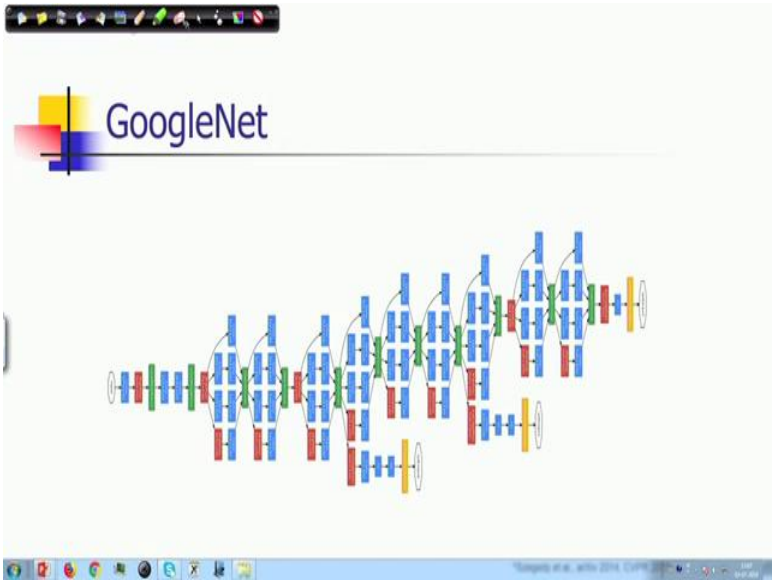
(Refer Slide Time: 13:57)



Then we came across GoogleNet and in GoogleNet you have more number of layers 21 depth and total 57, it introduces inception modules, that is the novelty in GoogleNet modifications. So, what it does instead of using a filter of same size in a particular layer, it concatenates output of filters of different sizes and that increases the power of the model. So, it has only one fully connected layer and number of weight is 7 million.

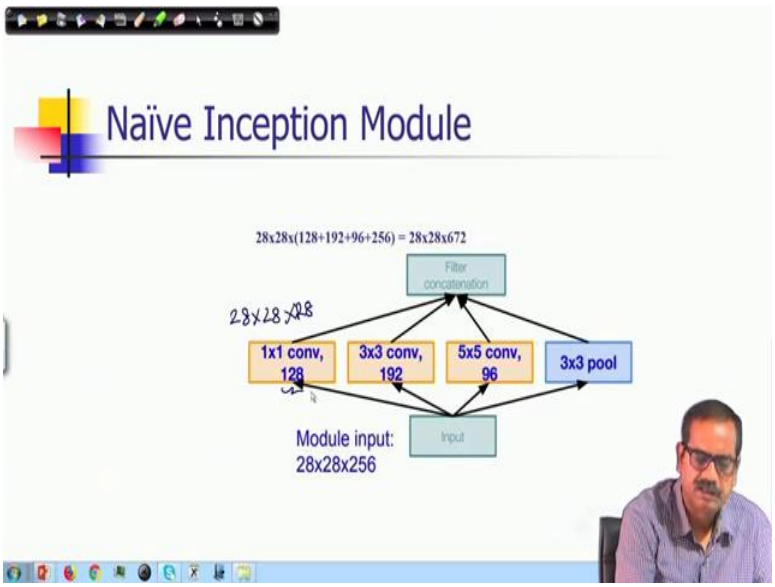
And, number of floating point operation is 1.43 billion and the architecture example architecture there are 9 inception modules as I mentioned. So, there are convolution, 3 convolution layers and there are 2 pooling layers followed by this convolutional layers, but after that you can see it is inception layers again there are pooling of inception layers. So, if I explain, if I give an example; so, GoogleNet also provides a very good performance 6.7 percent top 5 error.

(Refer Slide Time: 15:12)



So, this is a one short diagram for picture of GoogleNet how does it look and you see that these layers are inception layers. So, these layers are inception layers; so, it is using different sizes of filters and then it concatenate this filter.

(Refer Slide Time: 15:39)



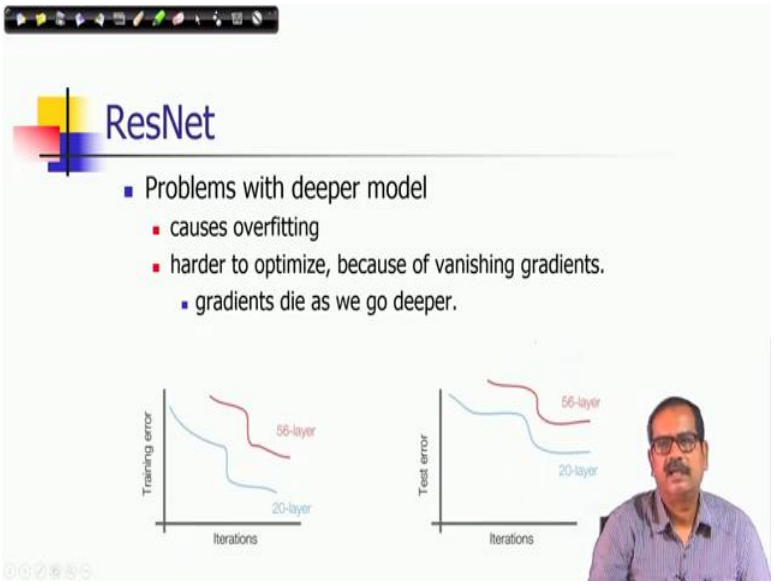
There is a enlarged diagram here. So, here it is shown that from the input there are filters of 1×1 . So, 1×1 means it is filtering along the depth only. So, your filter weights are along depth only, then there are 3×3 convolution filters, 5×5 filters and 3×3 pooling. And, number of filters with different sizes also they vary say 128 1×1 filters 192

3×3 filters 96 5×5 filters. And then of course, 3×3 pooling which does not require any number of filters which should give you the same depth value of the input.

So, finally, you can say that if I use 128 1×1 filters and your output would be $28\times28\times128$ that would be your output. Similarly for 3×3 convolution and here by performing 0 padding, you are keeping the input size same output size same as the input size; so, that is also done in the inception model. So, you have $28\times28\times192$ output from this model. So, this is giving $28\times28\times128$, because there are 128 filters this will give $28\times28\times192$, this will be giving $28\times28\times96$ because there is a 0 padding.

So, you are keeping input and output same size and this will give 3×3 pool, but there is no stride; so, this will also give $28\times28\times256$. So, total input size output size would be say 28 cross 28 cross and there are so many channels. So, this is how the inception model works, it increases the number of channels and sometimes it reduces the number of channels like here instead of 256, the number of channel becomes 128 here.

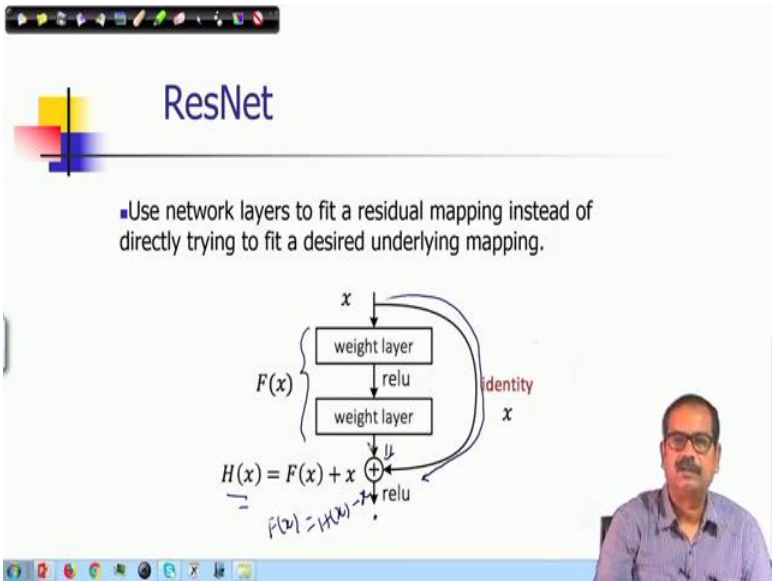
(Refer Slide Time: 17:54)



So, that was the you know modifications done in GoogleNet and it has found to be given good performance. Then the next kind of deeper model is the residual network or ResNet because the problem, if we increase the more number of layers; it has been observed that it causes overfitting and it is harder to optimize because the gradients they get managed. So, the since the managed for the vanishing gradients you cannot run the optimization, it drive soon as we go deeper.

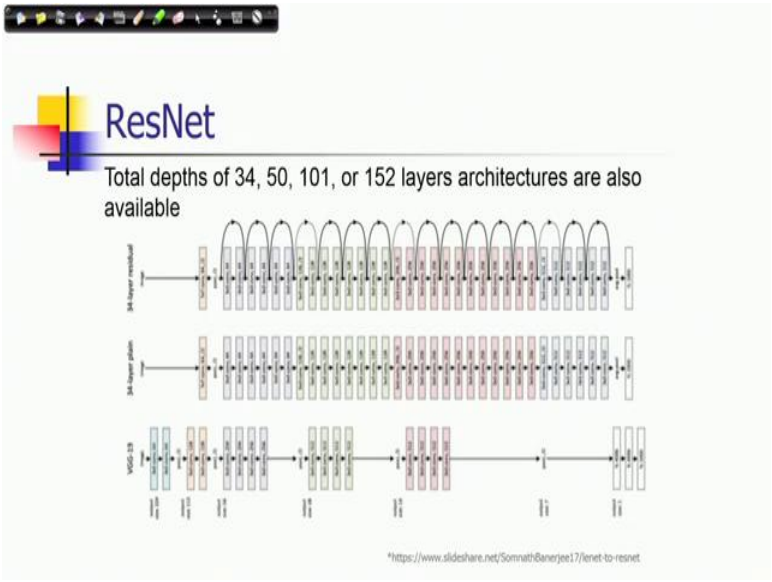
So, this ResNet we will discuss the architecture. So, this is the problem. So, you can have a good training error, but your tester would be quite high, if you increase the number of iterations, if you go by more number of layers. So, this is a normal CNN, this is CNN with less number of layers and this is the CNN with more number of layers. So, you will have this problem that your error will be higher.

(Refer Slide Time: 18:59)



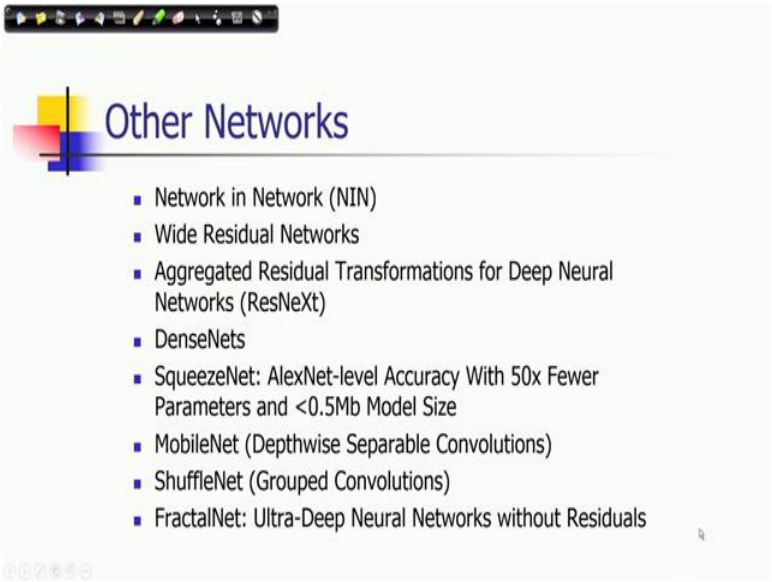
So, in the residual network or ResNet what we learned? We try to learn the residual errors. So, it fits the residual mapping instead of directly fitting the in fitting the data itself. So, as you can see in this unit that the learn; what you are learning; you are learning the residual part effects. Because, there is a skip connection here, it is called skip and it is added with this residual part and then only you are getting the output H x. So, what you are learning H(x). F(x) is H(x)-x which is the residual part that is what you are learning.

(Refer Slide Time: 19:48)



So, this is one schematic diagram where ResNet are shown, as you can see that the skip connections are there regularly and your number of layers could be very large. It could be more than 100 now and it has increased the performance of the classification to a great extent.

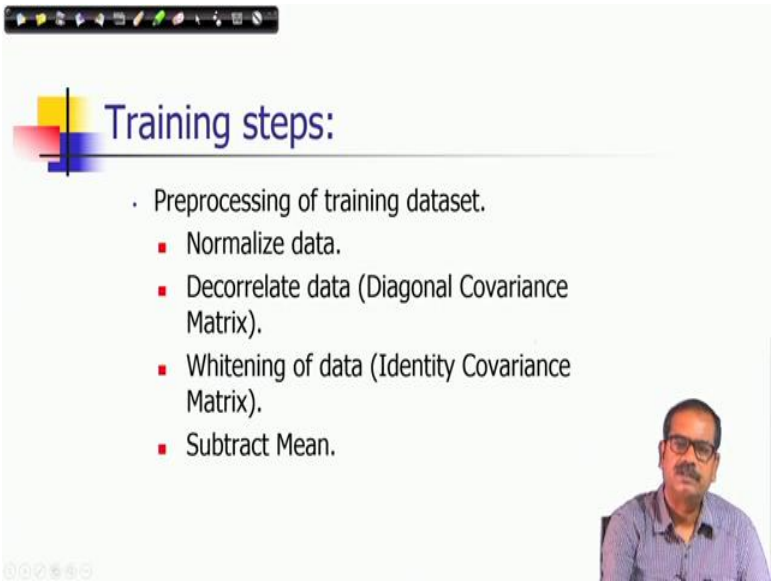
(Refer Slide Time: 20:08)



There are other examples of networks also like some list is provided here, I am not going to read all of them. So, you may go through these references. In particular let me point

out one network which is called MobileNet, it is used for low end applications and here all the filters are of size of 1×1, they are all separable convolutions.

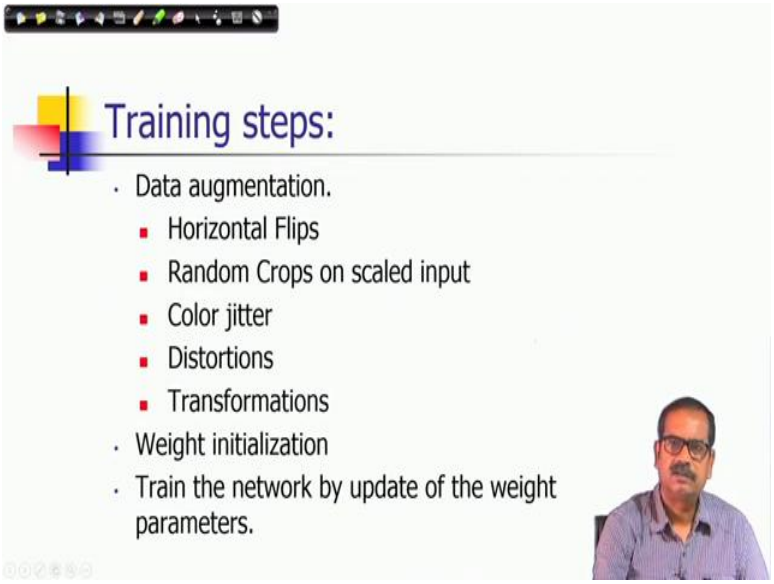
(Refer Slide Time: 20:36)



So, to there are few things we would like to discuss about training of this network. First thing is that we would like to pre-process the training data set, different kinds of pre-processing is required. We need to normalize the data with respect to the variances of the data that we have to normalize. Then decorrelation is also required which means diagonalization of the covariance matrix We know the decorrelation of data is principal component analysis, that we have learned and whitening of data is identity covariance matrix.

So, which means that during normalizations you have to divide it by the variances, you have to normalize the ranges and or you can subtract the mean. So, different kinds of pre-processing operations that may involve, while using these data for the purpose of computation.

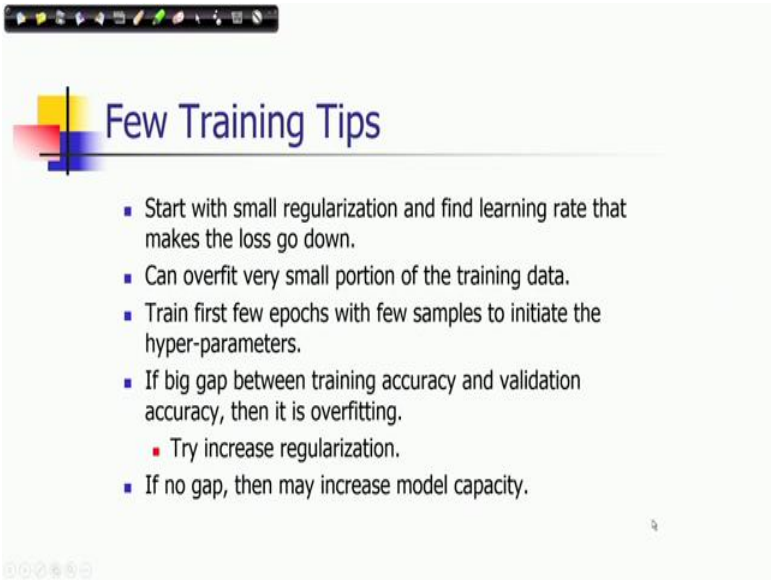
(Refer Slide Time: 21:34)



Then you may require to generalize your data set because, the data set what you have with level data set that may not capture the all variances of in the class. So, that is called data augmentation because by doing some simulation you can also generate some more data. So, like you can perform different kinds of geometric operations on data, images say horizontal flips, random crops on scaled input, color jitters.

You can add noise, you can add variances, add different kinds of color variations, distortions, then transformations. So, there are different kinds of simulations you can do on the given input data set and generate more input data set and use it for training. Then you need to initialize weight, there are different policies for initialization of weights. I will discuss that and train the network by updating the weight parameters.

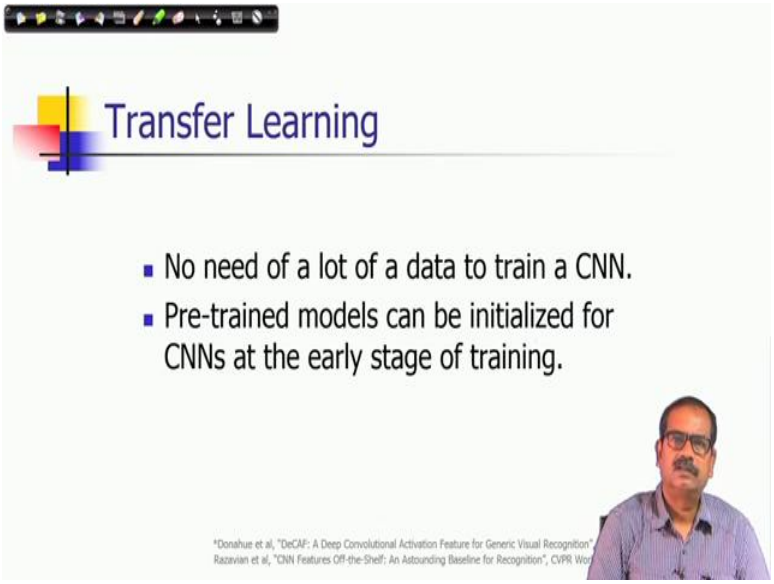
(Refer Slide Time: 22:44)



So, there are few training tips that is required that I would like to provide here. So, we should start with small regularization and find learning rate that makes the loss go down. So, regularization parameter; so, there is a loss function you have the regularization term and also the loss function due to the cross entropy term for example; so, it is a linear combination. So, start with the regularization component keeping it a small and also try to find out learning rate that makes loss go down. So, it can over; you can overfit very small portion of the training data and then train first few epochs with few samples to initiate the hyper parameters.

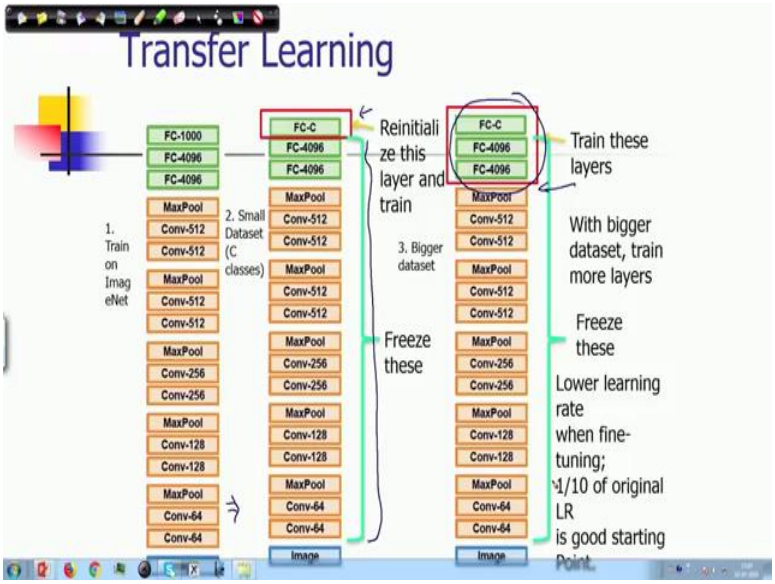
So, first you have to introspect how it is behaving and try to select those parameters which are giving you which is providing you the directions of convergence, good directions of convergence. So, those parameters you need to consider and because it is very time consuming; so, once you fix the parameters then again empirically you have to observe how it is behaving. So, these are certain tips by using it you can reduce your time of experimentations. So, if you find there is a big gap between training accuracy and validation accuracy then it is a overfitting case. So, what you should do? You should increase the regularization term in that case or if there is no gap then you may increase the model capacity.

(Refer Slide Time: 24:18)



So, one of the policy for initialization of the weights is called transfer learning and in this case even if you have a small data set, you can work with deep learning paradigm. You do not require a lot of data set to train CNN; usually deep learning requires lot of data set. Because, what you can do you can use a pre-trained network which has been trained using a very large data set for some applications. But deep, but use that feature representation in your own application. So, pre-trained models can be initialized for CNNs at the early stage of training.

(Refer Slide Time: 24:59)



So, the example here I will providing say for example, you train on ImageNet data set, a particular network. Here we have shown that there are few convolution networks and then fully connected networks given the image and it also shows number of channels etc. And, now in your small data set you fix this portion so, this is your feature representation. So, the whole feature representation is fixed so, you can freeze this and you can transfer the weight which has been learnt in this network, you can transfer it directly to this because your architecture is kept same.

So, that is a constraint you have to keep the same architecture; so, you transfer those weights and use these weights to learn the features. And, then only you train the output layer; that means, the final classifier you only train that part and you can reinitialize and train for the classification in your particular to your problems, what you are working on. If you have a large data set bigger data set, then you can have some other you can have other kind of policies. For example, in this case you can consider the all the fully connected layers to be trained and by keeping your future representations only at this point.

So, you can consider that and your lower; you have a, you can fine tune finally so, what you can do also that you can fine tune once you get a convergence some weights then you can again fine tune. So, you can train with your input data samples and using a lower learning rate and which is like one-tenth of the original learning rate; I mean that is a good starting point and then you know you can vary it.

So, these are you know different pragmatic policies and you know some of the intuitions, heuristics those are used while training a good deep neural network which takes lot of time and also lot of resources. And finally, a good network when it gives you a result that requires a lot of experimentations and it involves lot of experimentations you should understand that.

So, with this let me stop here and we will continue this discussion on deep neural architecture in my next lectures.

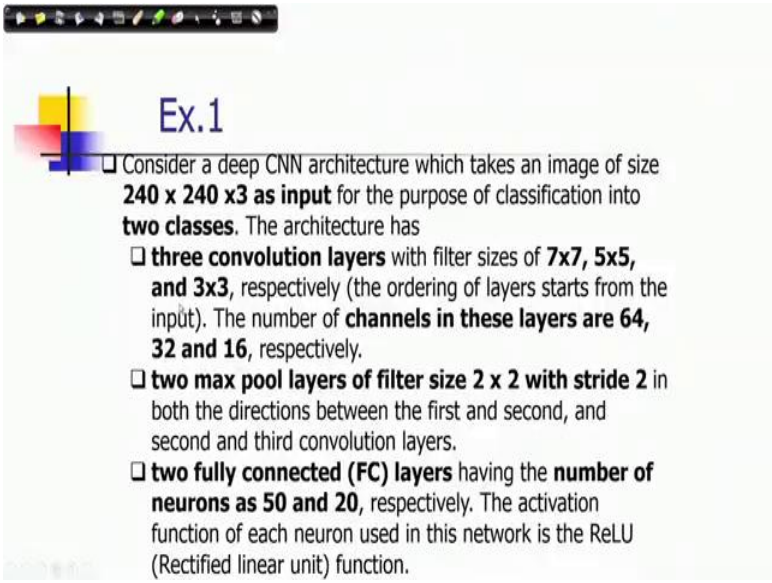
Thank you very much for your listening.

Computer Vision
Prof. Jayanta Mukhopadhyay
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

Lecture - 58
Deep Neural Architecture and Applications Part – IV

We continue our discussion on Deep Neural Architecture and Applications. In the last lecture we have discussed about convolutional neural networks and we have also considered different architectures of convolutional neural network.

(Refer Slide Time: 00:35)

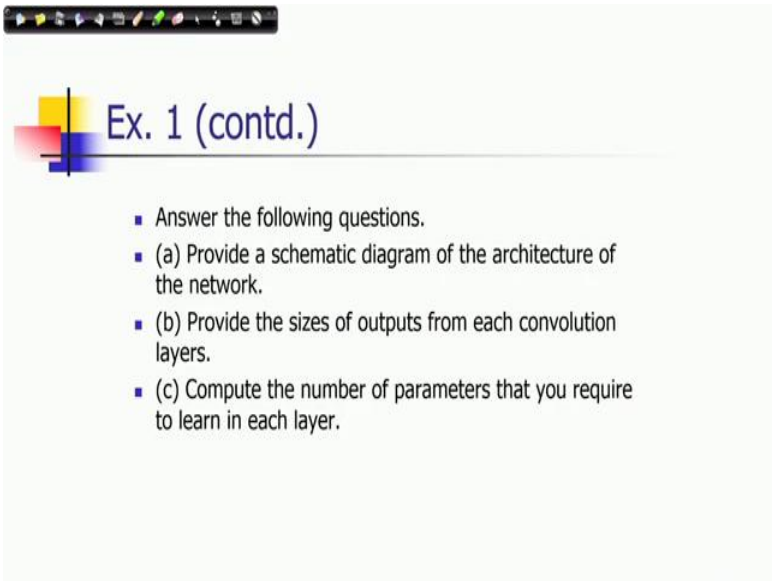


Let us consider a problem at this stage to find out the number of parameters and also to determine the sizes of the output or input to a particular layer given a specification of convolutional neural network. So, consider this exercise, a deep CNN architecture which takes an image of size $240 \times 240 \times 3$ as input.

So, in the input there are 3 channel input and for the purpose of classification into two classes and the architecture has three convolution layers, where in each layer the filter sizes are of 7×7, 5×5 and 3×3 and the number of channels in this layers are 64, 32 and 16. So, these layers they are specified from input to the output and then there are two max pool layers in between every convolutional layers.

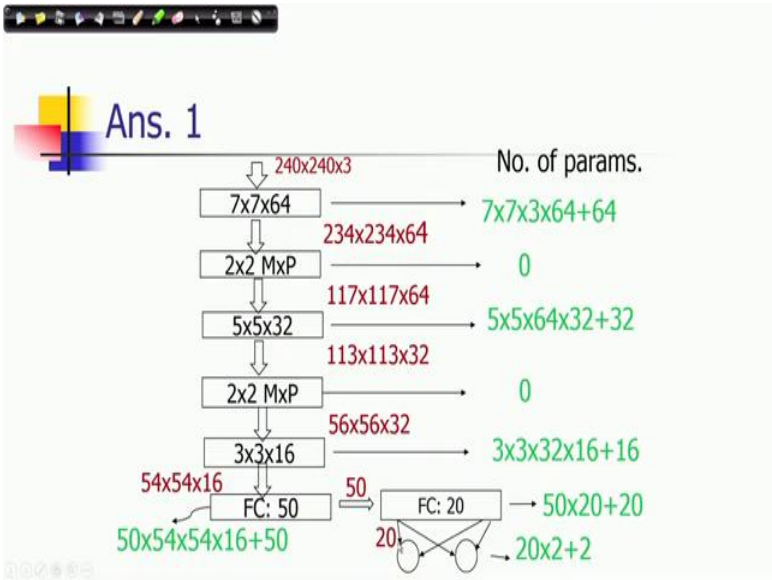
So, which means two max pool layers of filter size 2×2 with stride 2 in both the directions between the first and second and second and third convolutional layers. And after the third convolutional layers we have two fully connected layers having the number of neurons as 50 and 20 respectively. And the activation function of each neuron used in this network is the ReLU or rectified linear unit function.

(Refer Slide Time: 02:09)



So, the problem here is that we have to provide a schematic diagram of the architecture of the network, provide the sizes of outputs from each convolutional layers and compute the number of parameters that you require to learn in each layer. So, let us see how we can find or we can compute which have been asked here.

(Refer Slide Time: 02:33)



So, let us consider the architecture, a schematic diagram of the architecture itself. So, as you can see that there is an input layer and which is going to the first convolutional layer of filter size 7×7 . And since there are 64 channels so, it should be a filter size of $7 \times 7 \times 64$. Then it is followed by a max pool layer of 2×2 and stride is 2 there.

And, then again the output from the max pool layer is fed to the fed to a another convolution layers whose filter size is $5 \times 5 \times 32$, because there are 32 channels output from that convolution layer. And, then again it is followed by a max pool layer of 2×2 size, this is the third convolution layer or final convolution layer of our architecture.

So, output from this max pool layer it is fed to that convolution layer where filter size is of 3×3 and also the there are 16 channel output. So, it is $3 \times 3 \times 16$ that is what will be considering, that is not exactly filter size it is 6; the last 16 is showing it is the 16 channel outputs. Filter size will be 3×3 will see that thing. And, then this is being fed to the fully convolutional layers where there are 50 neurons.

So, the output from the 16 channel output from the third convolution layer, they are all flattened and they are considered as a say linear vector and or in the 1 dimensional feature vector. And, then that is fed to the fully connected layers of 50 neurons, this is also this output from this layer also is fed to another fully connected layer of 20 neurons and then those are again connected to output neurons. So, this is the output layer where there are 2 neurons and it is used for the classification because there are 2 class

classification problems; so, there are 2 neurons at the output. So, this is a schematic diagram of the architecture.

Let us now determine what should be the sizes of outputs from each layer. So, the input here as I has been specified that we have it will be handling and a size of $240 \times 240 \times 3$ this is the input size. So, the output if I consider 7×7 filter and as you know that it is only the output should be from those pixels of the input where this filter is fully embedded. So, filter size is $7 \times 7 \times 3$; that means, it is the number of channels of the input that would determine the depth of the filter and 7×7 is its width and height.

So, filter size is 7 cross 7 not 64, if I have mentioned wrongly earlier 64 is the number of channel here in this architecture output channel. So, that $7 \times 7 \times 3$ filter is embedded and then fully embedded on those pixels only from there the output has been provided. So, you have to leave around those boundary pixels where, it could not be fully embedded.

So, you have to basically subtract 6 pixels from width and height and only 1 plane will be coming out of this process. So, you have 234×234 and the since the output has 64 channels; so, there are 64 such filters. So, the output is $234 \times 234 \times 64$. So, this is our output size from the first convolution layer, then this goes to the max pool layer of size 2×2 and stride 2 which means they fill the input size should be half of the output of max pool layer should be half of the input size and number of channel will remain same which is $117 \times 117 \times 64$.

Then it goes to the next convolution layer where the filter is a 5×5 and since there are 64 channel input; so, filter size would be $5 \times 5 \times 64$ that is the depth of the filter. Once again you get only know 1 plane output 1 channel output from each filter there are 32 such filters.

So, you have to leave aside those boundary pixels where they are not fully embedded. So, 117 you have to subtract 4 from there so, it would be 113×113 and since there are 32 channels; so, it is $113 \times 113 \times 32$. This will again go to the next max pool layer of 2×2 and it has to be divided by 2 and since it requires full embedding of max pool layer; so, it would be only the lower ceiling or of the $\frac{113}{2}$.

So, it is $56 \times 56 \times 32$ channel remains same. Once again $3 \times 3 \times 16$ filter there is a third convolution layer, actually size is $3 \times 3 \times 32$ and there are 16 just filters. So, you have to subtract 2 from 56 and the size will be $54 \times 54 \times 16$ because there are 16 channels. Now, the all these 2 dimensional array output, it will be flatted into a 1 dimensional vector of size $54 \times 54 \times 16$ and that would be input to the fully connected layer of 50 neurons.

So, each neuron will have that many number of weights. So, the total number of weight will be coming to the total number of weight whatever. So, output of 50 neurons from the fully connected layer would be 50 only. And then output of 20 neurons from the fully connected layer it will be 20 only and then you have 2 class outputs. So, there will be from there will be 2 class output.

So, which is not written here, but there are even 2 outputs from this layer. So, this is the sizes of different outputs from different layers and next we are going to count the number of parameters. So, I have indicated by arrows by showing that we will be counting the parameters of those layers which is associated with those layers. So, consider the first layer so, we will be considering number of parameters. Consider the first convolution layer as the filter size is $7 \times 7 \times 3$ because input is 3 channel. So, there 1 filter size would be $7 \times 7 \times 3$, but there are 64 such filters.

So, it would be $64 \times 7 \times 7 \times 3$ that many number of weights and each 64 filters will have also bias; so, plus 64 bias. So, in the green color I am showing this count of parameters. So, this is answer that the number of parameters in the first convolution layers will be this.

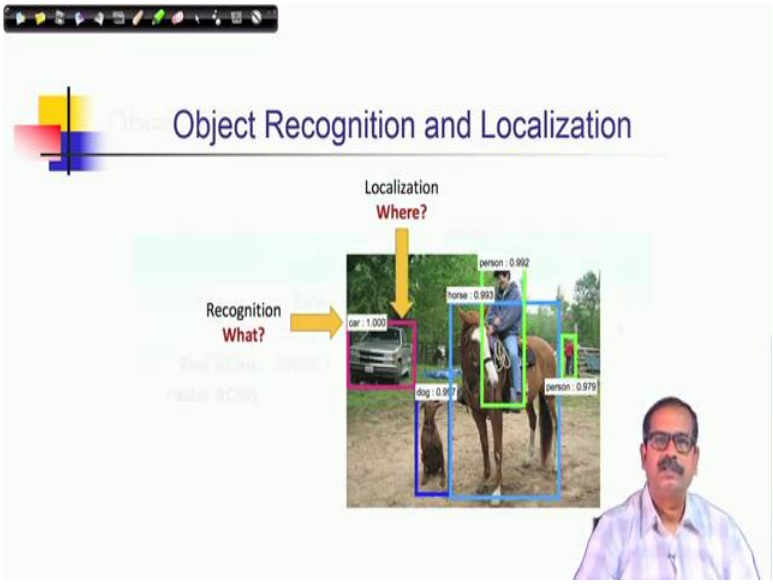
Similarly, for the max pool layer since we know that max pool layer it does not require any weight, it is already specified by its sizes itself. So, the number of parameters would be 0 there. So, that is what is max pool layer, next layer 5×5 then the filter size is 64 5×5 is 64 depth is 64. And how many filters are there? There are 32. So, it should be $5 \times 5 \times 64 \times 32$, that many weights plus there are 32 filters; so, there are 32 bias.

So, this should be the number of parameters from this layer. Again there is a max pool layer so, we will have 0 here. In the next it will be fed to the 3×3 filters and depth would be 32 because, the input to that layer it has 32 channels. So, it is $3 \times 3 \times 32$ and there are 16 filters. So, $3 \times 3 \times 32 \times 16$ plus for each 16 filters there are 16 biases. So, we

will have $(3\times3\times32\times16)+16$ Next it goes to the know fully connected layer and as I mentioned that the size of the input here is $54\times54\times16$ each input will have a weight to the fully connected there and there will be 50 such neurons; so, there will be 50 such bias. So, it should be $(50\times54\times54\times16)+50$ so, that is what here and here the size is 50.

So, it will be 50 weights should be connected to every neuron of 20 neurons and there are 20 biases. So, it is $(50\times20)+20$ and again since there are know still you require more weights because this is the output layer. So, output here its size is 20; so, each neuron will have 20 weights and 1 bias so, it should be $(20\times2)+2$ So, this is the total number of parameters those are involved in this architecture, what I have specified in the problem statement. Understand that this is how that parameters and sizes could be determined given the specifications of a particular CNN.

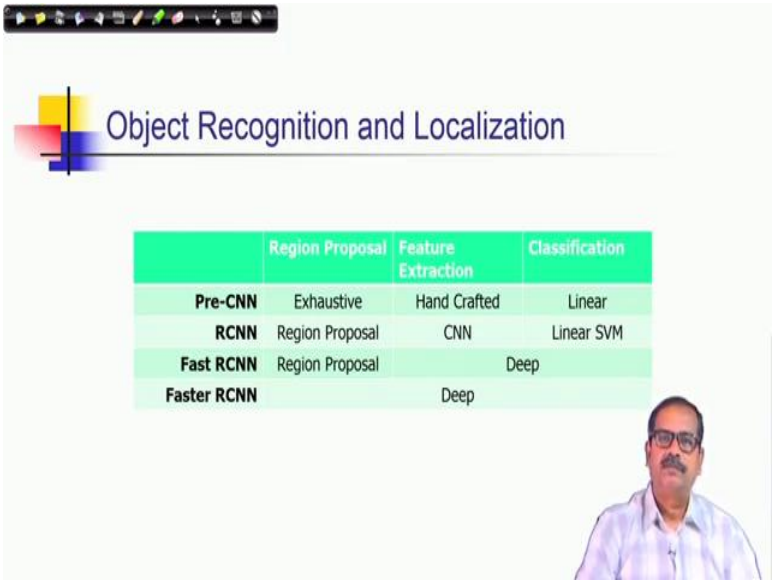
(Refer Slide Time: 12:22)



So, let us now consider another kind of problem which can be solved once again using convolutional neural network. This is this problem is called object recognition and localization. So, in this problem the task here is that given an image not only you have to identify the level of the object, but also you have to identify which portion of the image where this object lies, in the form of a rectangular block.

So, any rectangular block can be defined by the two corners of the block in pixel coordinates. So, that is how the specification of rectangular block will be there.

(Refer Slide Time: 13:05)



So, there are different techniques, this is a very classical problem and even before the deep neural architecture there are methods by which people have tried to solve it. So, this is a pre-CNN and the fact is that the object while it is which the region of the object in the image that needs to be identified. And, we call that task is as a region proposal and which means you would like to find out a probable region which contains an object and then you try to find out that what is the object in that region.

So, this is a kind of approach; there are stages like first you propose the region and then you describe that region by features. So, you have to extract features and then use those features to classify and if you could find some objects in your target in your classes. Then how you decide about that, otherwise you may say that region does not contain any object. So, there are various methods which have been reported.

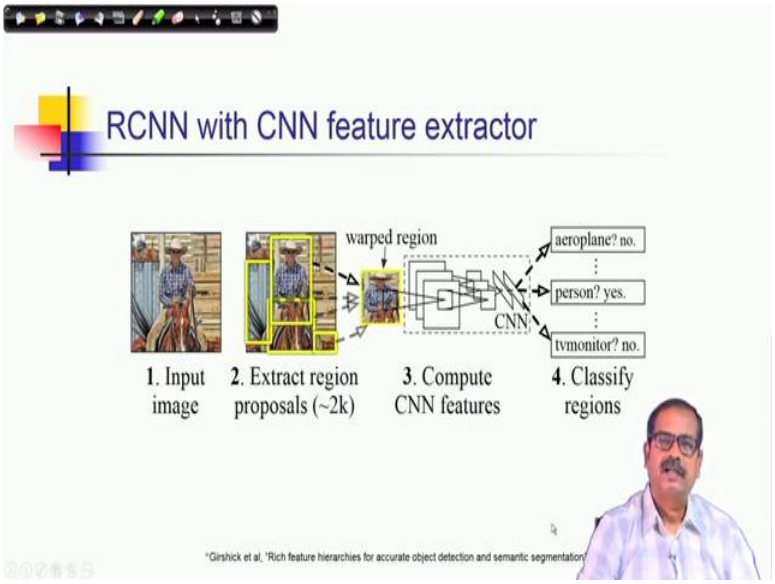
So, one method was very exhaustive means you are searching every rectangular type of rectangular block which is a combinatorially a hard problem, you need to use some heuristics there. So, that is what you should do it. And there are methods also to propose you certain heuristics to propose those regions. And then for feature extraction you can use some handcrafted features. Handcrafted features I mentioned what is used considered as a handcrafted features by considering the classes of objects, you predetermine that what kind of features will be computing.

For example, you can use shift operator, you can use the descriptor of shape or other kind of key point detectors and key point descriptors those. And, then combine them aggregate them to define a feature. And, then there are different classification algorithms you can use like linear classification algorithms. And, for deep neural architecture based technique there are three such variations. So, very first a technique called RCNN has been proposed.

So, RCNN what it does? It considers the task of region proposal should come from the conventional methods and then the feature extraction part is only should be done using CNN. So, that you do not have to use any handcrafted design. So, as per your problem features could be extracted learnt and extracted and then use a classifier to find out the object.

For example, linear SVM that is a classical classifier a rather another improvement or variations over on top of RCNN, where the both feature extraction and classification is done by a deep neural network. Whereas, the region proposal is still carried out using the conventional methods and the third variation that whole thing can be performed can be done is in a deep neural architecture.

(Refer Slide Time: 16:29)

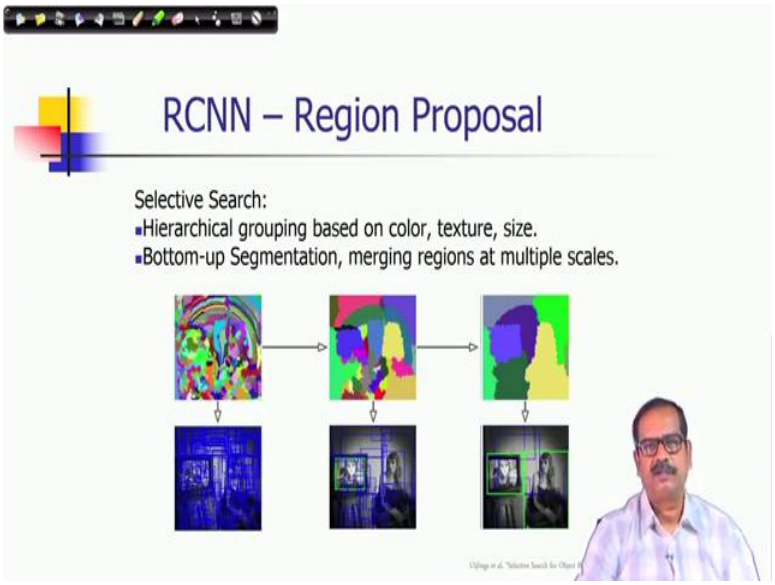


So, this diagram it is explaining this flow of this computation. So, you have an input image and then the regions are proposed. So, these yellow boxes are proposed regions by conventional methods, then you extract them one by one and then fed it to the CNN, as

CNN takes a fixed input size. So, you need to work those regions and then you know compute the features.

So, last layer of the CNN will give you a future descriptor and which is used in a classifier. Once again this classifier is the conventional classifier and then you decide what kind of class it is based on those target classes of this classifier.

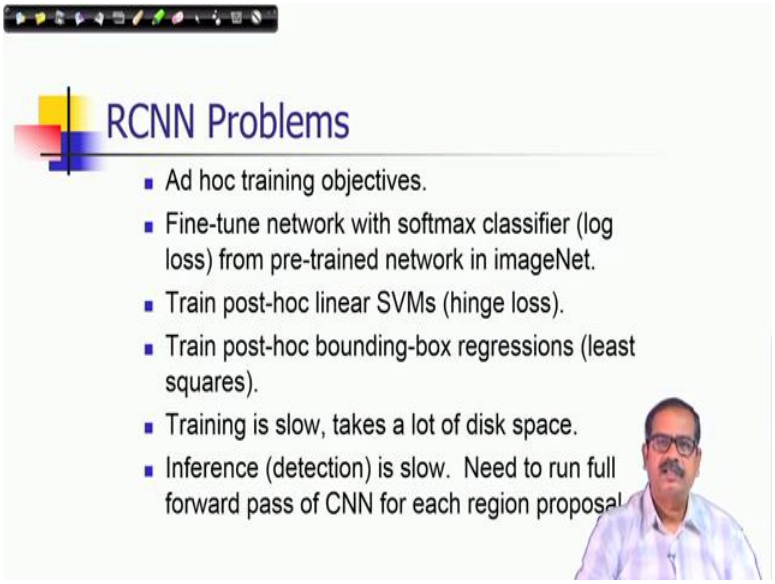
(Refer Slide Time: 17:13)



For this particular method as I mentioned that it uses conventional method for proposing regions. So, there is a method based on selective search which uses hierarchical grouping based on color texture size and it is a bottom up method. So, it performs bottom up segmentation then it merge regions at multiple skills and then based on the regions property it decides whether it contains any object or not. So, this figure tries to summarize this process.

So, it considers very small segments at the bottom layer, then it merges it and the bigger segments and it tries to identify out of them some of them are considered that they contain the object. And then you use the rectangular box enclosing those particular segments.

(Refer Slide Time: 18:12)



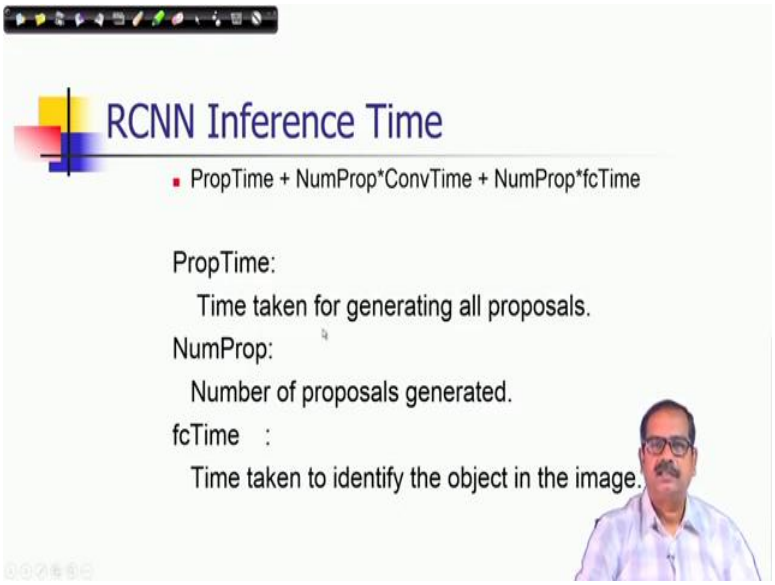
Their problem in this method is that the training objectives are not very clearly mentioned here. As you can see that the CNN is a feature extractor stage, but know what is what should be the objective function of that CNN. So, far we have discussed about training of CNN based on some classification task. So, even the features are learned based on the end objective of classification, but in this case since the classifier is a conventional classifiers; so, classifier is independent, independently designed.

So, this is a problem; so, you have to consider some intermediate objectives to learn the CNN. So, kind of adhoc is remains there. So, you can use some pre trained network, but now again you need to do fine tuning based on your domain, based on your problem domain. So, that is required or and also we require to once again train the SVMs, Support Vector Machines based on those features and usually feature descriptors are quite large; so, this will take lot of time. So, and also another training you have to do because you have to find out the regions.

So, though the regions are proposed, but you need to do some kind of fine tuning. So, some kind of regressions based on those proposed regions you need to do; so, that also requires some training. So, there are so, many different kinds of trainings are involved that is why it is very slow and also it takes a lot of disk space. So, inference is also slow, it need to run full forward process of CNN for each region proposal.

So, that is what because for each region proposal you are extracting features independently and you will need it so, many times you need to iterate this process.

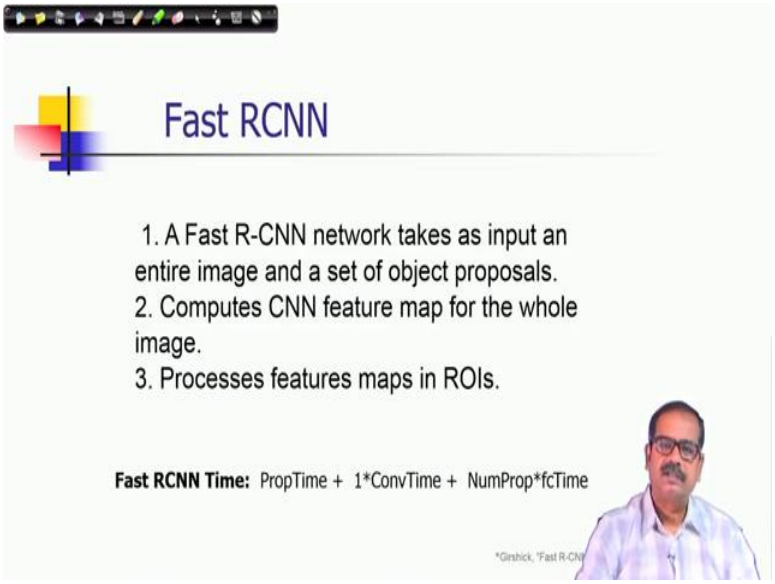
(Refer Slide Time: 20:05)



So, that is why its inference time is very large, it takes quite a bit of time. So, what are the times components here? You can see there is a time called prop time or proposal time. So, it is time taken for generating all proposals, then number of depending upon number of proposals that is generated at this stage. So, each proposal or each region has to be fed to the CNN.

So, it extracts the feature. So, it is number of proposals into the time taken for computing features using CNN and finally, each one once again has to be classified. So, that there is a classification time and time taken to identify the object in the image. So, this is a total inference time, if I consider for RCNN.

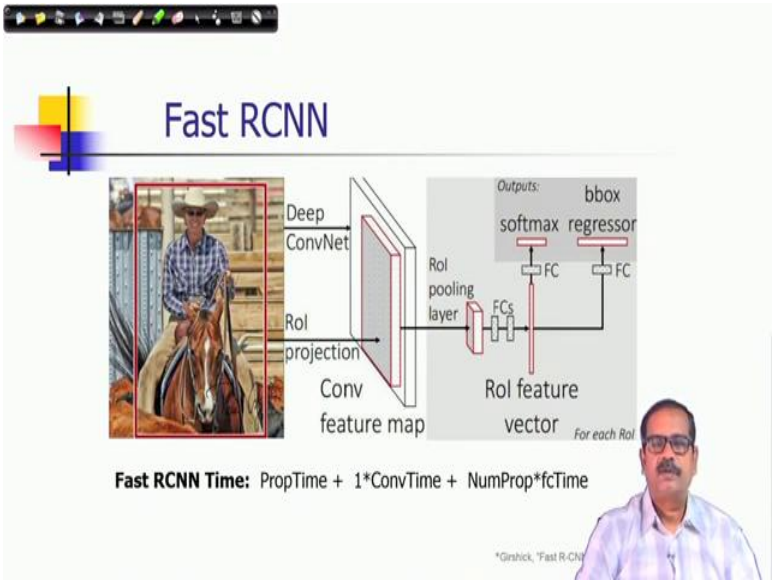
(Refer Slide Time: 21:06)



So, to an improvement over this particular approach is that you can use a first RCNN network. And here the idea is that instead of using an SVM, we can use also the whole thing classification and feature extraction can be carried out at the same go. That means, for every region it will be passing through both feature extraction and classification in this case.

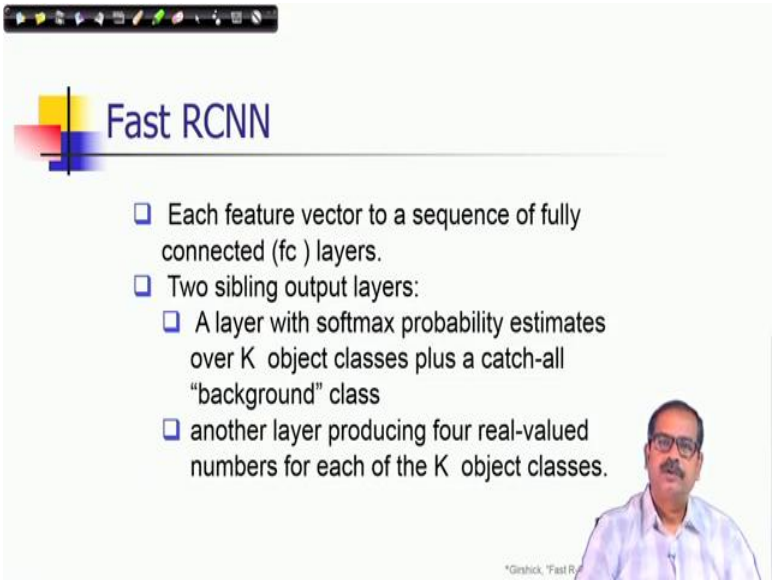
So, it computes CNN feature a map of the for the whole image. So, and also it is not required to compute the feature map of the whole image. You can just process the feature maps in ROIs and you know here only the thing is that it pools the ROIs and then again it passes to the classification stage. So, it is not that deep neural network a deep architecture is not there at the classification stage it is a conventional classification.

(Refer Slide Time: 22:12)



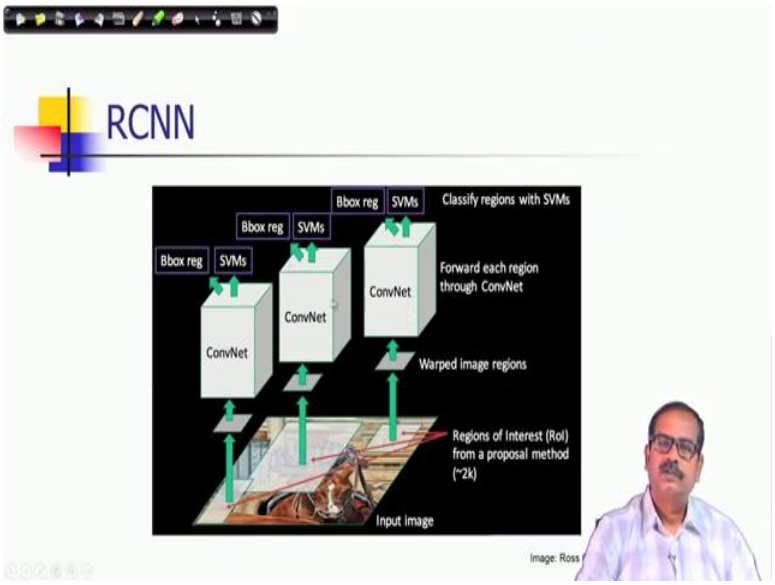
So, this is what it is doing it, it only uses the same conventional neural network. So, given the image it computes the whole feature set, then using the region proposal it pools those features and use a classifier to compute to classify it and also the objects. So, the time here is the proposal time it remains proposal time and then since it is only computing 1 time all the features, it is only 1 time convolution network is used. So, it is 1 × that conv time, but every region every proposal is separately classified. So, it is number of proposals × fc time.

(Refer Slide Time: 22:59)



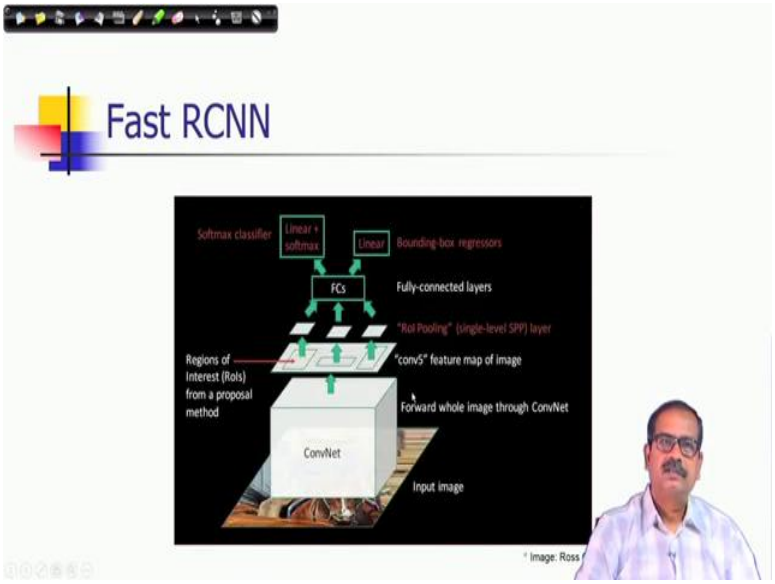
So, first RCNN the features are that each which are vector to sequence a fully connected layers and two sibling output layers. So, a layer with the softmax probability estimates of our K object classes plus a catch all background class. So, you are considering that and another layer producing four real valued number for each of the K object classes, this is actually regression of the regions.

(Refer Slide Time: 23:25)



So, it just shows the architectures of RCNN, you can see that every we proposed region is passed through separately through the convolution layers. And, then those are used for classifying and regression of the regions using regression module and also the SVMs for classifications; every region every regions are processed in this way.

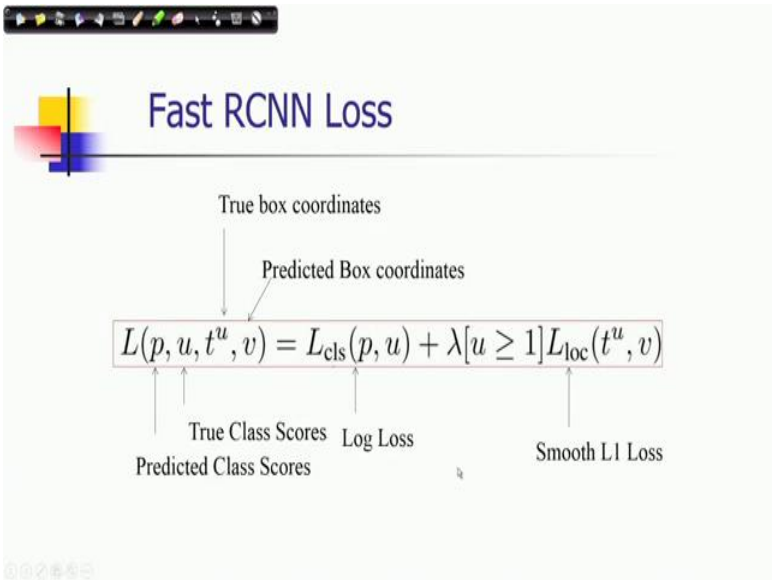
(Refer Slide Time: 23:55)



Whereas, in fast RCNN you have the only one time the convolutional neural network is used to generate all the features and also it has propose the regions. So, use those regions in the feature map those are used for classification; so, those are fed again to the fully connected layers and then.

So, it is a classifier which could be also neural network classifier like ANN classifier and those are used for classifying the object and also the finding out the regions; that means, fine tuning of estimates of the regions using a regression box.

(Refer Slide Time: 24:41)

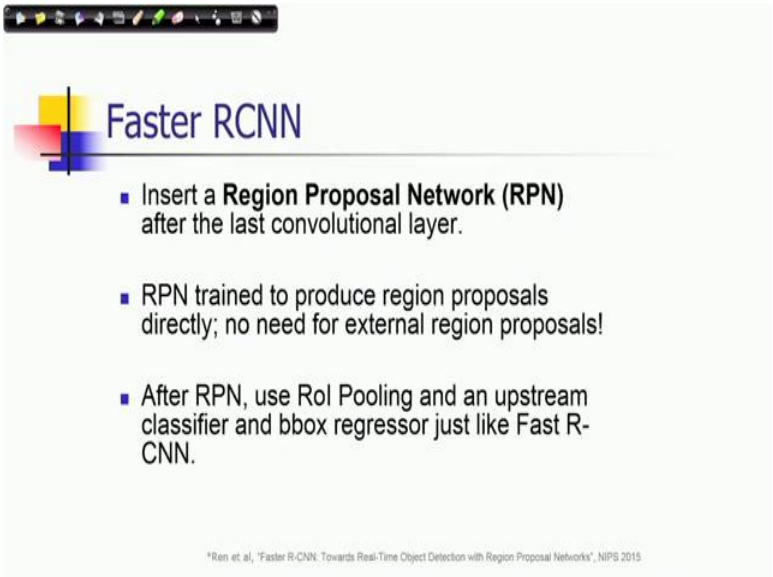


The loss what is used in the stages classification stage as I have shown where there is a fully connected layer which is simultaneously classifying the object and also detecting the box or a rectangular area by the object is contained. So, there are two components of this loss; so, one component is due to classification and the other component is due to the regression.

So, and we are only considering those samples those classes which are object classes. So, u is the ground truth, p is the predicted class. So, if only the it is a object class then only we are considering the loss due to the regression and this is a linear regression λ is any parameter. So, these this is the thing this is the log loss which is cross entropy loss.

This is predicted class course, p is the predicted class course and u is the true class scores. And, this is the smooth L1 loss, L1 is the function which is used for computing the regression the box, the error deviation of the localizations or deviation of the rectangles from the true rectangle to the predicted rectangle. So, t^u is the true box coordinates and v is the predicted box coordinates. So, this is how the loss function is defined.

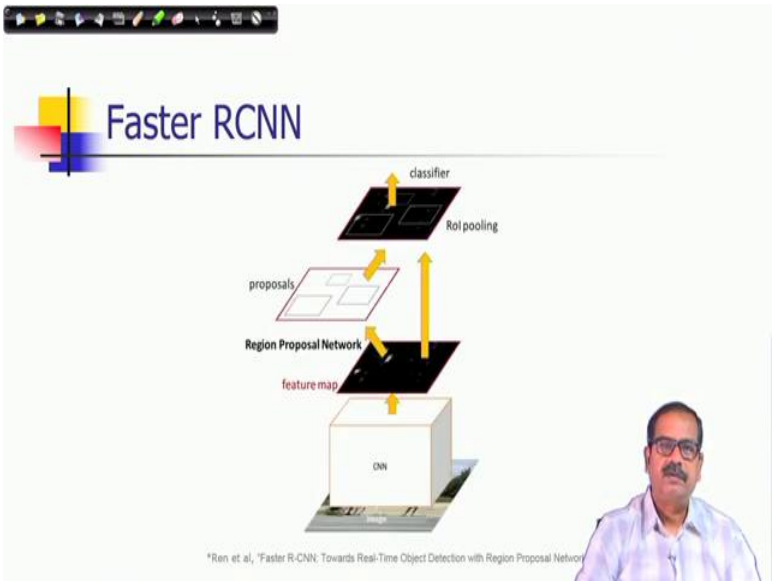
(Refer Slide Time: 26:10)



So, then improvement over even faster RCNN is faster RCNN, where the whole task has been considered you doing it in a convolutional network; even the region proposal also is done using a convolutional network. So, we call it as a region proposal network. So, what we can do that using the features what is generated in by the convolutional layers, the same features are used for generating region proposals.

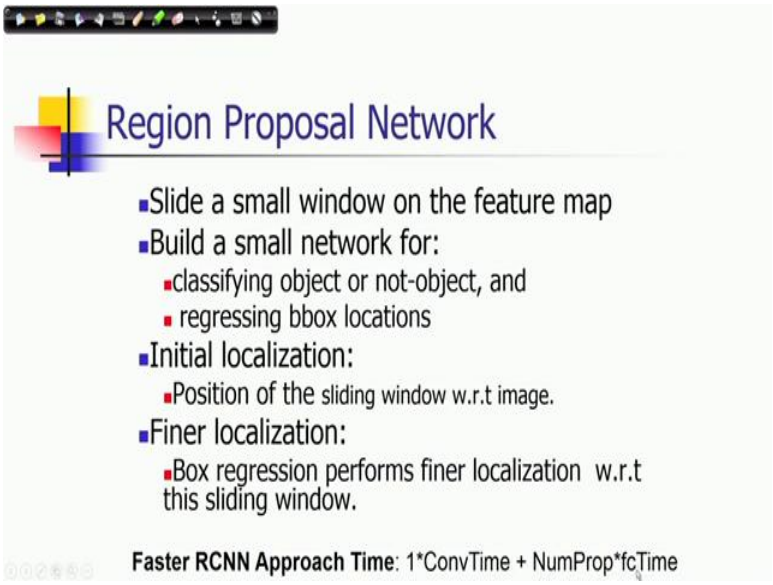
And, from the region proposals you pool those features which are there in that re-proposed region and used it for classification.

(Refer Slide Time: 26:53)



So, that is why it becomes so fast and you can see that the first stage is that you generate a feature map. So, use this feature map to propose regions and also use this feature map to classify and also regress the rectangular boxes.

(Refer Slide Time: 27:07)

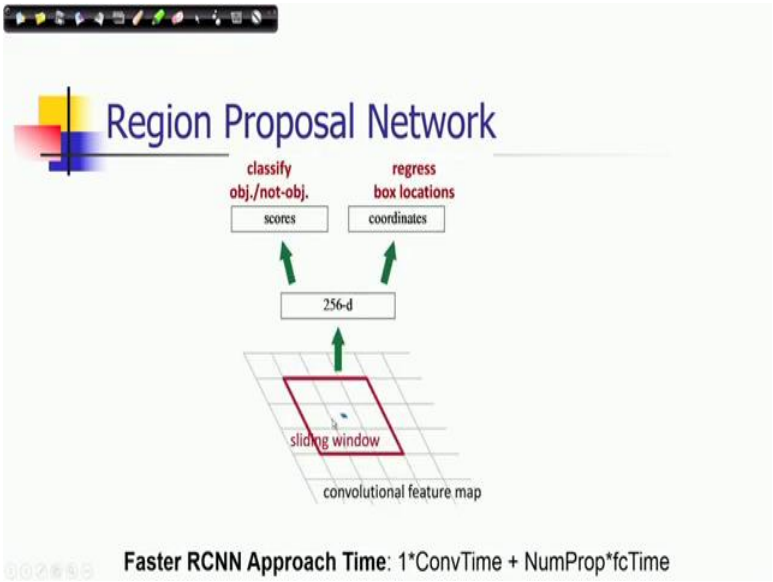


So, this is what is faster RCNN and in the region proposal network in the task is that we can slide a small window on the feature map and then we can build a small network for

classifying object or non-object. And so, here the task is that you have to classify the object or non-object and also regress the bounding box locations and this initial local this is the initial localization.

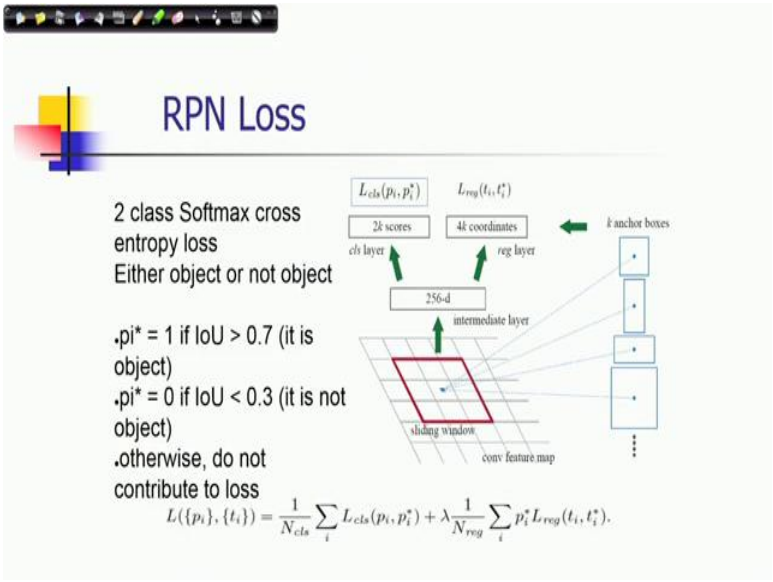
So, these positions are used and final localization is considered at the final stage of classification and regression, where box regression performs final localization. So, and the faster RCNN as you can see that it requires only there is no proposal time separately, it is 1 time convolutional time and then number of proposals into the time required for classification.

(Refer Slide Time: 27:57)



So, this is describing the region proposal network. So, you take a sliding window in the feature map and it is a 256 dimensional feature descriptor. Then you are classifying as object or non-object or regressing the box location. So, this will give you the initial box locations of the object, again in the final in the last stage when you are classifying the objects you can make a fine tune of that locations.

(Refer Slide Time: 28:25)



So, for region proposal network loss what it considers that in this sliding window there are different shapes of windows are also considered, which may contain a box or object. So, there are k anchor boxes; so, each one is tested whether it contains a object and also this box itself becomes the coordinates those are output of this region. So, it contains there are two components: one is once again classification loss, once again it is a regression loss. So, these two components are used in the loss function.

In the 2 class loss function standard cross entropy loss is used and because now you are deciding whether it contains object or not. So, if your predicted object is intersecting about 0.7 of the true object 70 percent of the true object, then we call this is 1 otherwise it is 0. So, it is intersection of union.

So, it is the ratio of intersection of predicted object and predicted region and true region divided by union of predicted region and true region. So, if this ratio is more than 0.7 then we call it is 1, if it is less than it is less than 0.3 it is 0 and otherwise it do not contribute to loss. So, with this let me take a break here and we will continues this discussion in the next lecture.

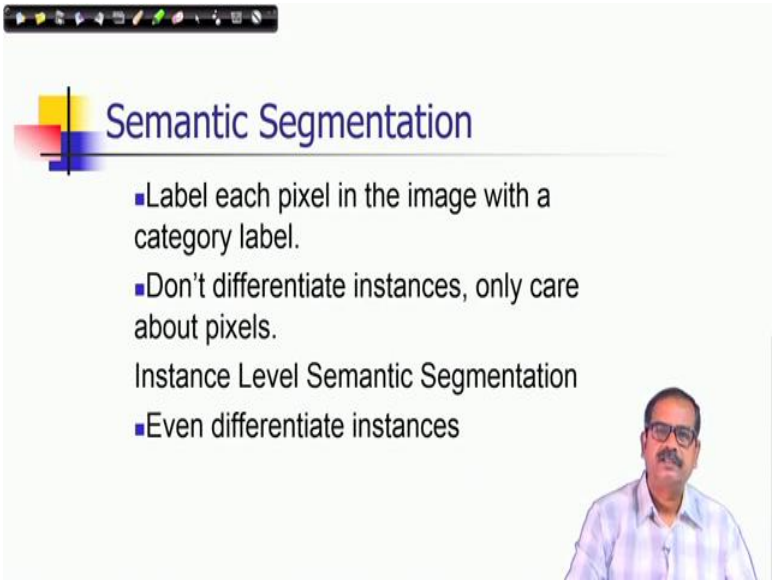
Thank you very much for listening.

Computer Vision
Prof. Jayanta Mukhopadhyay
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

Lecture – 59
Deep Neural Architecture and Applications Part – V

We are discussing about different kind of operations or different type of tasks that we can perform using deep neural architecture. In the last two lectures we discussed about task of classification of objects and also task of classification as well as localization of objects. In this lecture we will discuss about the segmentation of images.

(Refer Slide Time: 00:41)

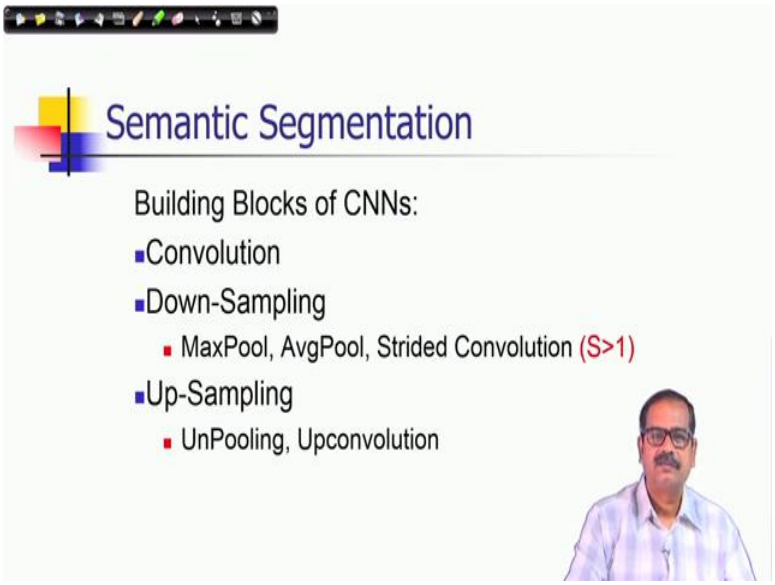


So this kind of segmentation is called semantic segmentation, with respect to this particular architecture; because the task here is that we would like to level each pixel in the image with a category label. So, you can see it is also akin to classification, but instead of classifying the objects or a group of pixels, we are trying to classify each pixel. And with that also you are assuming the segmentation of images that we understand what is segmentation, that grouping a pixels of same classes or same categories.

And it does not different differentiate instances only care about pixels; that means, if there are say 3 human profiles and they are all connected profiles. So, it is sufficient if you denote the segment as all 3 connected as human profiles. But there could be instance

level semantic segmentation also. So, even the task could be that you have to differentiate, even individual humans in that connected segment of human profiles.

(Refer Slide Time: 01:47)

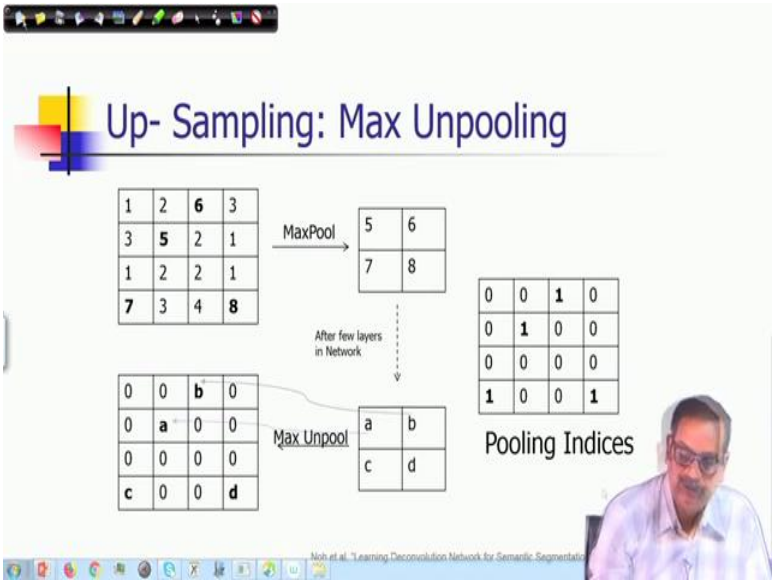


So, in a semantic segmentation, the building blocks uses convolutional neural network. And these are the certain things some of the computations what is being carried out standard convolution. Then it is a task of down sampling because it tries to learn the features with aggregations at different resolutions.

So, in down sampling, as we know that we can perform down sampling by this different kinds of processes; like you can do pooling, max pooling, average pooling even you can use strided convolutions, which means you leave some of the pixels and do not do convolution there. And in the output only you consider those pixels where you have convolved the image and that itself will down sample the image. And then there is a stage of up sampling because finally, in the segmentation the input and output both the sizes would be same; because every pixel of the input has to have some semantic level. So, this learned feature are again used and it is projected in that semantic space.

So, those are leveled those are learnt this projection is also learnt and that is up sampling task. So, there are operations like unpooling and upconvolutions I will explain those operations we understand the maxpool or pooling operations and the down sampling operations In this lecture we will discuss what is unpooling and up convolution.

(Refer Slide Time: 03:25)



So, let me explain this part first what is first the unpooling, max unpooling operation. Consider this particular image the example is shown here and its a 4×4 image and we would like to perform the maxpooling by 2×2 filters or its not exactly maxpooling what we are trying to do it is a kind of decimation. So, let me call no yes it is maxpooling and what we do while doing maxpooling, you also generate the index of the particular position.

See in this 2×2 block, the maximum value is here 5. So, output will have 5 from these 2×2 block and also its position in this 2×2 block has to be also stored and that information also you should be kept so that during unspooling, we will be using that information.

So, as you can see in these block 6 is the max in this block 7 is the max which these are shown in the bold font and 8 is the max. So, the maxpool operation will give you 5, 6, 7, 8 and also as it is shown here. And also the position can be stored in a binary mask; where the positions where the value from the value position from the position from here. The value is taken those position is indicated by 1 and all other positions are 0.

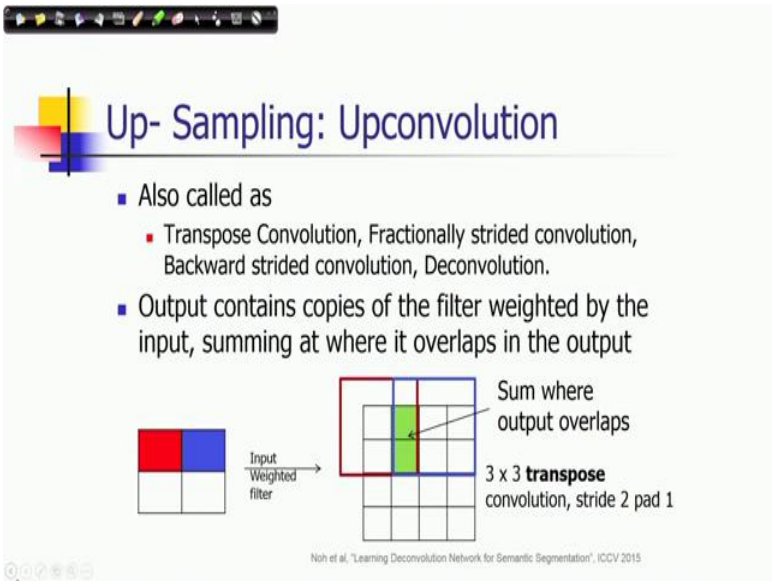
So, its a kind of indication indicator functions, we call this or a as pooling indices. So, this two things goes hand in hand. So, you are not only computing the maxpool operation and also you are computing the pooling indices. So, this pooling indices are used during the unpooling operations. So, how it is used? Let us see now the input to

your unpooling operation is the values of maxpool operation; that means. Now this is your input and also it would be added by pooling.

So, if I consider max unpooling, then this is your input and also this is also your input because you will be using this indices to finally, generate the output. So, after a few layers of network, which means in the processing pipeline the unpooling operations can come later. So, you should tag the pooling indices to that layer. This is a output or this is the input to your maxpool through your un-pooling; because after a few network a few layers in network these values will change it is not the same value.

And suppose these values are shown here by these variables a, b, c, d. Then the unpooling operations max unpool will be like this. So, you can see that using pooling indices, we are putting the values to the corresponding locations we are positioning. So, a is positioned here from here and this is also guiding the position of a similarly position of b. So, this is the position of b; with respect to this block and you see that with respect to this block b is put here. So, in this way we are doing a max unpooling. So, this is how they are showing.

(Refer Slide Time: 07:15)



Now let us understand how we can perform up sampling and up convolution. You consider input to a block, input to the up sampling up block as I say 2×2 input here the colors are shown of the input. And there is a weighted filter from there. So, what we do

that for each input we are multiplying this filter. So, so the idea is that output contains copies of this filter weighted by the input. And its summing at where it overlaps in output.

So, the let me explain it suppose your input here is given the value say $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ and for each input we have a 3×3 weight filter; say 3×3 weight filter we will have say $\begin{bmatrix} x & y & z \\ u & v & w \\ p & q & r \end{bmatrix}$ So, what we will do and say v is a central place. So, far 1 will be generating the output as x, y, z, u, v, w, p, q, r then you consider 2; so the place of 2 if I consider the place of 2. So, so this would be the place of 2 and from there we will be using.

So, this should be I should say 2 x plus, 2 y plus, 2 z then v would be this is 2 u plus 2 v plus 2 w. So, this plus is this is not plus because this is the new these is the new position and still there was 0 earlier. And this is q plus 2 p r plus 2 q and 2 r so; this is how the 2 is used. Similarly for 3 we have to perform the same job; that means, if I consider the position of 3.

(position 2 become the following)

$$\begin{matrix} x & y+2x & z+2y & 2z \\ u & v+2x & w+2v & 2w \\ p & q+2r & r+2q & 2r \end{matrix}$$

Central position of 3 is here that is what you need to take a when you are considering position of 3; because you have to place the array and place this thing as the central positions and accordingly in each pixel locations you are doing it. Similarly when you place the central positions 3, then you will find that u will be 3 this will be 3 x. This will be added with 3 y this will be added with 3 z, this will be added with 3 u this will be added with 3 v, 3 w.

$$\begin{matrix} x & y+2x & z+2y & 2z \\ u+3x & v+2x+3y & w+2v+3y & 2w \\ p+3u & q+2r+3v & r+2q+3w & 2r \end{matrix}$$

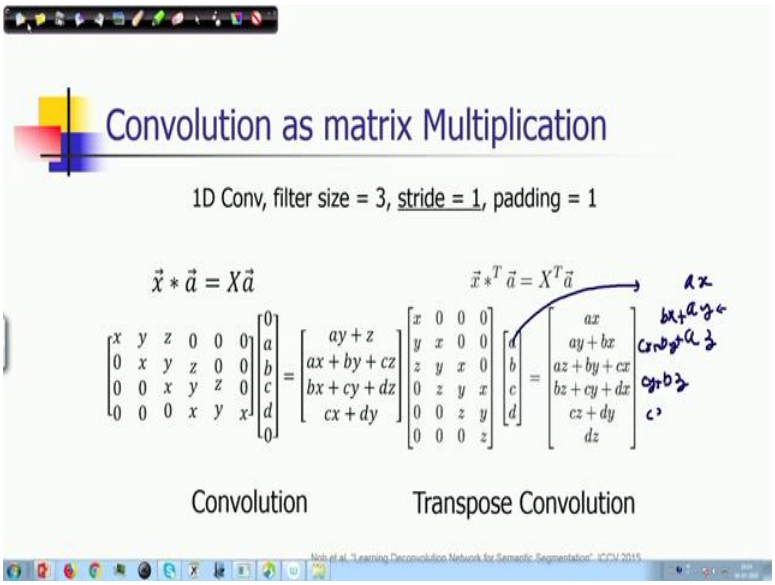
And so, this is about 3 and also the lower row will come which I am not computing showing here. Similarly for 4 it should be this location. So, this would be 4 v once again

this would be 4 x plus, 4 y plus, 4 z and this value will be 2 r it is 4 w. And this value would be here it is 4 u and also here the low bottom row I am not showing. So, in this way what you are doing you are you are coping the filter weighted by the input.

$$\begin{matrix} x & y+2x & z+2y & 2z \\ u+3x & v+2x+3y+4x & w+2v+3y+4y & 2w+4z \\ p+3u & q+2r+3v+4u & r+2q+3w+4v & 2r+4w \end{matrix}$$

So, you are placing all those things in those locations and in the overlapping locations you are summing it up that is what is your output. I will do one exercise later on and you can find out how it is carried out there if you will understand fully this part. So, this kind of operation is also called us transpose convolution, fractionally strided convolution, backwards strided convolution and deconvolution these are there are different other names.

(Refer Slide Time: 11:58)



The expression what I have given to you it can be also considered in a different form and that is very precise definitions from the mathematical form. So, if I consider 1 dimensional convolutions, then it would be more clear that what exactly you are doing we can show that why it is called transpose convolution. So, convolution operation in 1 dimension you can express very easily using matrix multiplication. And when you are doing 2 dimensionsthat you have to use if you have separable properties, then you can also perform it using matrix multiplications.

But let me first consider the 1 dimensional convolutions for the sake of convenience of our discussion. Consider a filter of size 3 and it is strided 1; that means that every pixel you are performing these computations and padding is 1. So, it is a the input is padded with 0s only at 1 pixel gap. So, and it is not pixel it is since these are samples these are 1 dimensional let me call it as sample. And then let us consider this convolution operation.

So, you can see that here the input is considered as a sequence of values $\begin{bmatrix} 0 \\ a \\ b \\ c \\ d \\ 0 \end{bmatrix}$ So, this is the input and you are performing a convolution whose response is given by x, y, z. So, if I consider a input say a, b, c, d and we pad it with 0 and you know how convolution is perform. it is a marks x, y, z which needs to be multiplied with this and then the central value will be replaced.

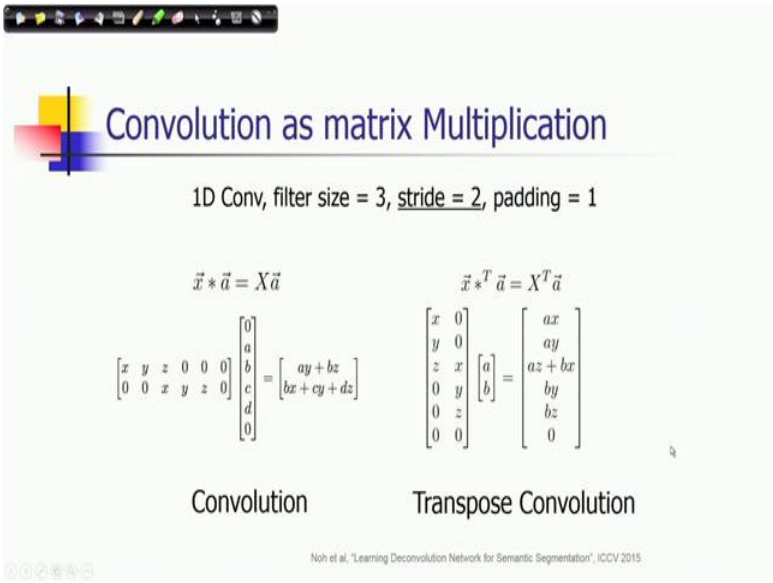
So, if I consider these value it should be $ay + bz$. Similarly this marks will be shifted here x, y, z and the central value will be replaced by this b will be replaced by after convolution $ax + by + cz$. Exactly that is what is being shown here in this computation so, what you can see, we can construct a matrix where each row consider shifted response of the corresponding impulse response x, y, z it. And that matrix is multiplied with the input and you get the output of the convolutions.

So, convolution can we consider it as matrix multiplication. And we can show also this up convolution, which is nothing but what I explained that what you are doing. You are basically multiplying the filters using the input in the corresponding up sampled size up sampled array. And then add those values in the overlapping locations. So, that can be shown equivalently. So, this is what is convolution.

So, you just transpose this matrix, convolution matrix and multiply with the input a, b, c, d and you will get this up sampled value. Once again we can see that this pixel a should be replaced by as we know that it should be multiplied with ax then ay this is a central location then az. Similarly for b it would be bx, by, bz and you are adding them. For c it will be cx, cy, cz and you are going on doing these things. So, that is how you get all

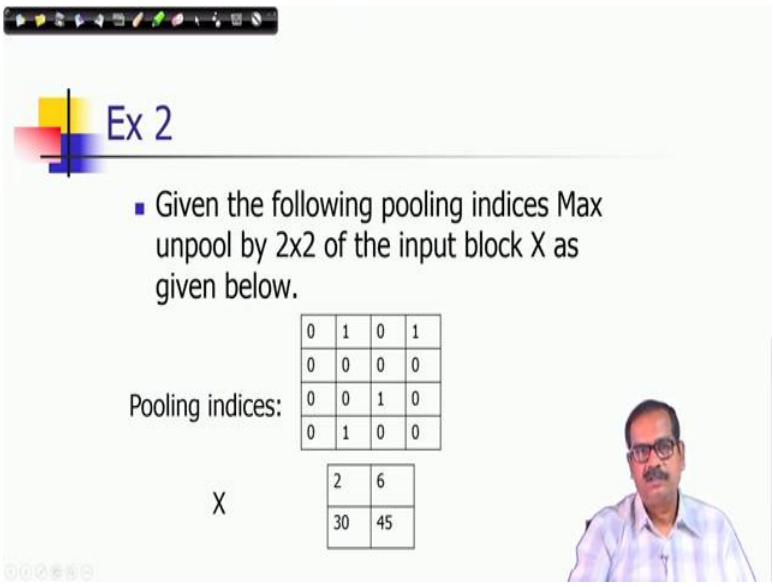
these values. And in fact, that operation is nothing but your transpose of convolution matrix and multiply that transpose of convolution matrix with the input a so, that is what you get it here.

(Refer Slide Time: 16:19)



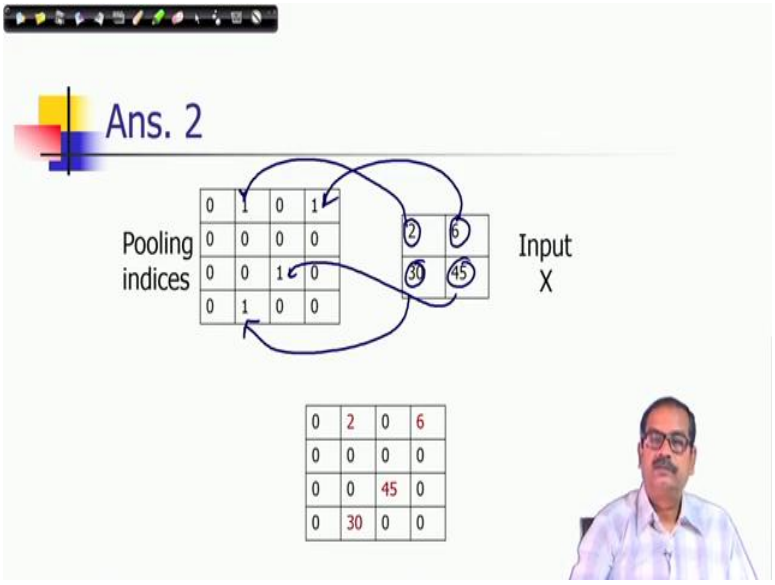
Even for stride two also we can compute. Here in this case if it is stride to your computing at a then c and you can get the samples. So, you can get up sampling and upsampling with some kind of you know these upper stride two operations.

(Refer Slide Time: 16:37)



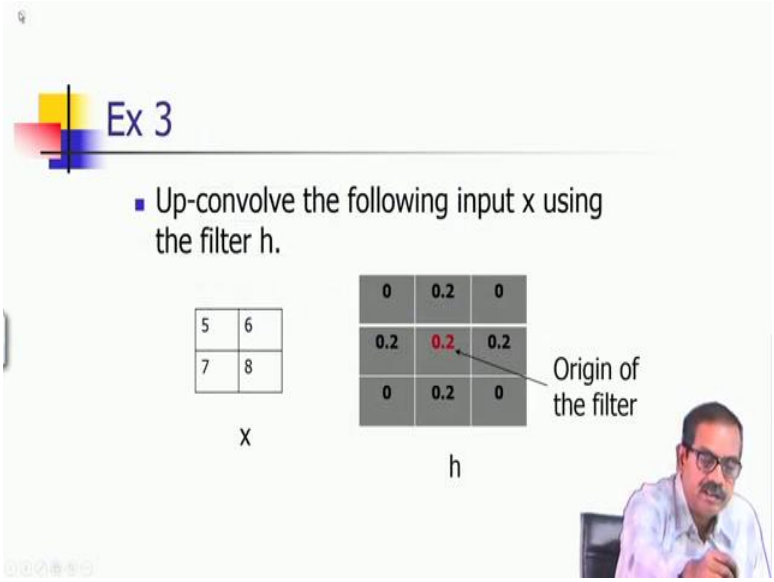
So, let me to explain these particular computations, let me solve one exercise. Suppose you have the following pooling indices and max un pool by 2×2 of the input block X the pooling index indices is shown here and also the input blocks is shown here. So this is your input and what you need to do? You need to unpool this particular indices. So, which is not written here, but you need to un-pool max you have to perform the max unpool or unpooling of these values X.

(Refer Slide Time: 17:19)



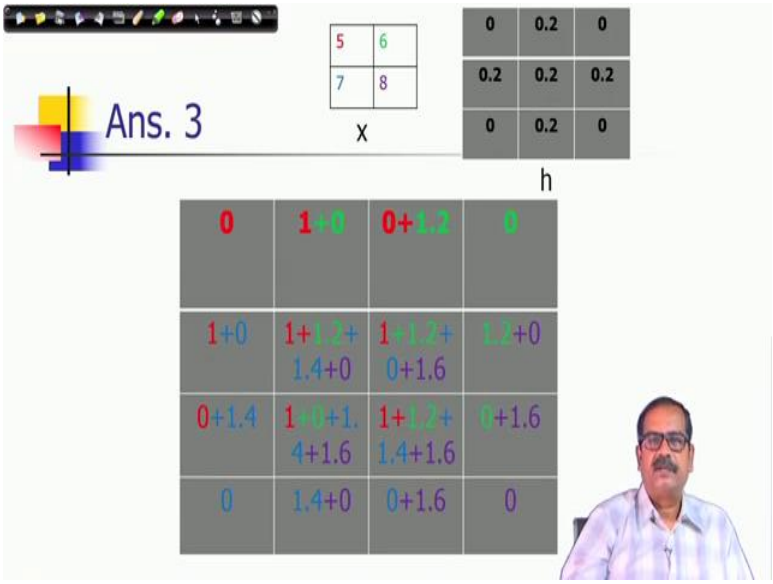
So we have already discussed how unpooling can be done. So, you have the pooling indices like this and you have the input X. And so, what you need to do you need to place the value 2 here, value 6 here, value 30 here and value 45 here, rest of the values should be 0. So, from 2×2 you are getting 4×4 image and these are the values you can get. So, this is how you get the unpooling of the input.

(Refer Slide Time: 17:57)



Let us consider another exercise that we need to up convolve the input x using a filter h. And as you can see origin of the filter is also given here. So, filter is a 3×3 filter and you have those input as 2×2 input values are shown for a toy example $\begin{bmatrix}5 & 6\\ 7 & 8\end{bmatrix}$. So, you need to solve this problem. As I have mentioned that each input has to be multiplied with this filter in the up sampled block.

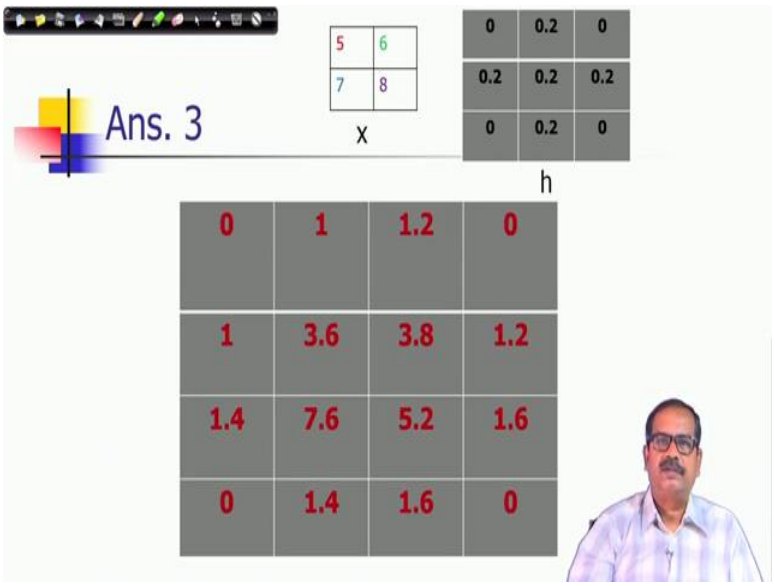
(Refer Slide Time: 18:34)



So, we need to solve this problem. So, what I need to I should do here? As I consider that I have shown here with colors the pixel values because they are contributions to the up sampled values can be shown by color. So, for example, 5 we will be contributing a 3×3 block centering at that pixel with say 5 in (2 0) locations in the diagonal neighbor and like this. So, let me show you by the add itself. So, you can see all these contributions are shown here.

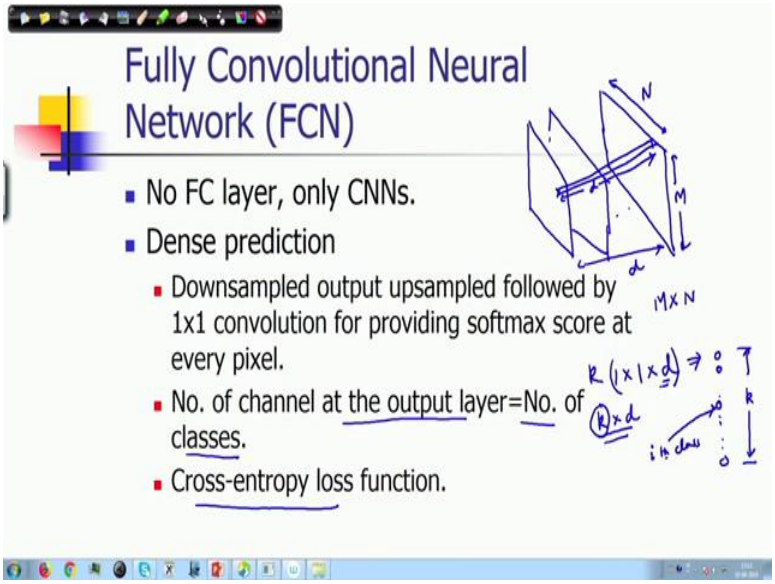
So, all the red values are contributed by a 5 all the green values green values or colors here they are contributed by 6 similarly blues from 7 and violet from 8. And at respective cells respective locations you are adding all of them and this is how you are doing up sampling and if you add them, then you get this value.

(Refer Slide Time: 19:40)



So, this is a output this is a up sampling value from 5, 6, 7, 8 you get this particular values.

(Refer Slide Time: 19:53)



Performing semantic segmentation, we use a bit different form of neural architecture and which is called fully convolutional neural network. So, in this case we do not have a fully connected layers as we use in traditional CNNs which is used for particularly classification of images. Here we use all the layers as if fully convolution as a convolutional neural network layers convolutional layers.

So, so finally, we have an output is also an image of certain number of channels. And usually the size of the output the number of pixels associated with that output is the same as the number of pixels of the input. And every pixel at every pixel we perform a prediction of the class to which the pixel belongs and that is why this kind of prediction is called dense prediction. And a belongingness to the class that would determine the semantics associated with that pixel and through which you can perform the segmentation.

Now, let us understand how this kind of classifications or this kind of prediction is carried out at pixel level. As we understand that in the process of computation input image is processed at several layers of convolution. And also there are downsampling operations either by max pooling or by strided convolutions or by pooling operations. So, at certain stage you will get the downsampled processed results processed arrays processed tensors. And those are to be again up sampled.

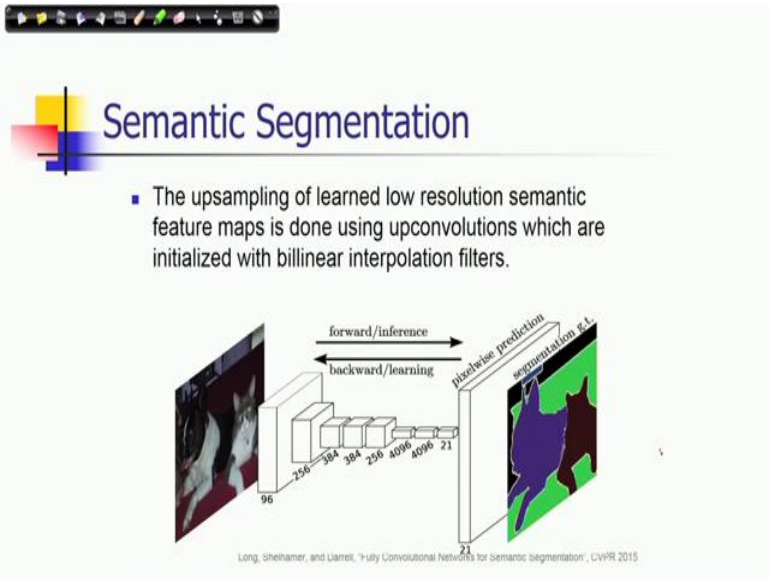
So, that it could be brought to the size of the input of the pool network. So, when we have the same size, but the number of channels at that layer at the layer just previous to the output previous to the prediction layer that number of channels may differ and let us consider an example. Suppose before dense prediction we have d number of channels let us consider there are d channels. And each of size say $M \times N$; that means, there are there are M rows and N columns; so, each of size $M \times N$ And here what we would like to do we would like to predict the level of a pixel. So, we perform a convolution with a filter if it is an 1 dimension filter of length d; effectively in the convolutional neural networks terminology we specify this filter as 1×1 filter and its length is d. So, this is one filter and that would give some response some value. And if I consider k of these filters for each pixel you will get k such response. So, there will be k such response. And each filter is associated with certain class. So, the say i th filter say associated with say i th class.

So, by using the softmax principle we can compute the probability of that pixel. Probability of a class that would be assigned to that pixel and it is the same principle that we can follow here. And these are the parameters so, these many number of parameters associated with each pixel that can be applied here I mean that needs to be learned. So, there should be $k \times d$ parameters that should be learned at this output layer.

So, as we mentioned here that number of channels at the output layer which is say k. That should be equal to number of classes that is a kind of restriction that we need to follow here because this is meant for this semantic labeling of the pixels. And for performing the optimization, here we use a loss function and since we are classifying each pixel cross entropy loss function is a natural choice in this case.

So, this is how we can use a fully convolutional neural network and that can be used for labeling every pixel of the at the output from this network. And we can perform semantic segmentation using this network. So, let us now see that how this network, what are the different kind of variations of this architecture?

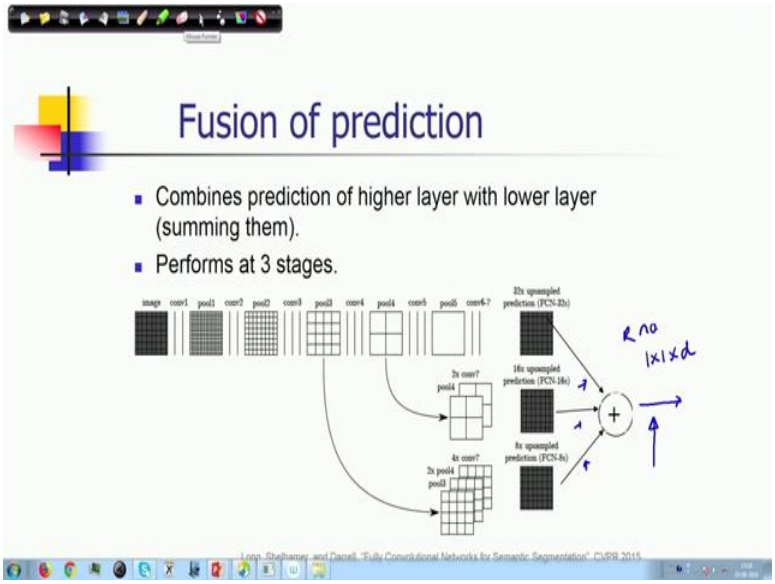
(Refer Slide Time: 26:08)



So, let us consider this particular work that is also performing semantic segmentation. And as you can see, what it is doing? It is in the process of computations it is performing the corresponding downsampling operations and also convolution operations. And also then after that it is also going through the up sampling operations at certain stages. And it has been brought to the same size of the input image and then it performs a pixel wise prediction as we have considered.

While doing up sampling, you it has considered that the initialization of the up sampled filter could be done by using billinear interpolation filters weights. That is a very standard interpolation filter whose weights are already predetermined because of the billinear operations there. So, after initialization you perform the optimization operations then those weights we will get updated. So, the next we will see what kind of operations so, this particular network is doing.

(Refer Slide Time: 27:37)



So, it combines the what it does it not only takes a prediction from the last layer, it also considers predictions of some previous layers operations. Let us consider this particular example say while down sampling the images at different stages of conclusion layers, then it down samples say always it down samples by a factor of 2. So, you have a down sampled by factor of 2 here, say in this case by a factor of 4; that means, you required up sampling by factor of 2 in this case up sampling by factor of 4 to bring it to the size of the input. So, this is factor of 8 and this is factor of 16 and say this is factor of 32.

So when you are trying to bring to the same size of the input what we would like to do here, we would like to perform an up sampling of by factor of 32. And we call on this up sampled output before the classification operations that is 32 X up sampled a prediction or FCN 32 s. So, prediction can be done from here using as I mentioned by using the k number of 1×1 cross the number of channels.

So, number of channels is not mentioned here suppose there are d number of channels at this layers at the final layer d number of channels. So, we have to predict using $1\times1\times d$ filters and for k semantic classes you have to use k number of such filters. So, we can perform prediction simply from this layer itself using this operations. But what it has been found that actually this performance could be improved.

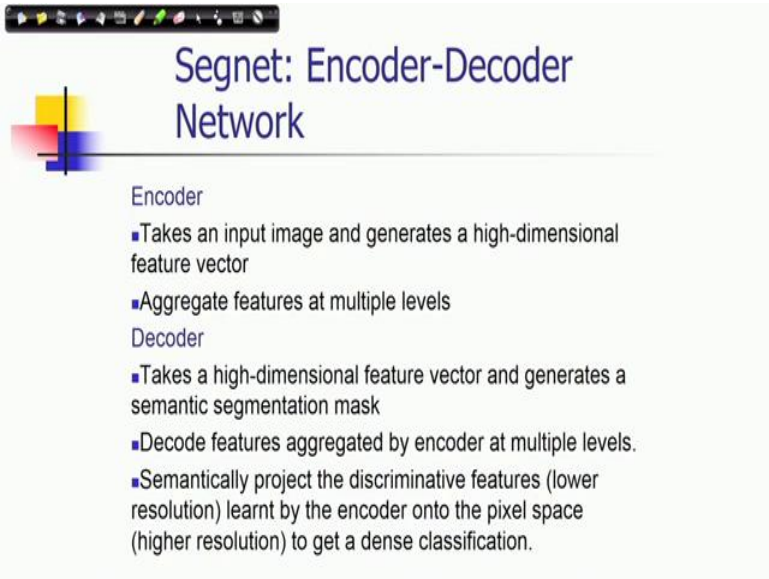
If we use also the information from the down the lower layers for example, from here if I perform a $16\times$ up sampling and then which means from this layer it is $2\times$ conv7 that is

what it means? But it is it means that you are doing $16\times$ up sampling. And then you again you also consider predicting using k number of filters at this stage. Similarly you are performing $4\times$ up sampling.

So, all those values all those values are what we are doing we are not predicting here. We are simply adding them or we are adding the corresponding values and then performing the prediction using $1\times1\times d$ filters k number of $1\times1\times d$ filters. So, if I consider this these operations it could be done in various ways. Probably in this work what it has been done, it has been applied independently for k times and then added them then the softmax softmax principle has been used.

So, you are learning these weights at all these stages and finally, the lost function is computed after fusing those values using soft max principle. And that would give variations and it has been found that that has improved the performance of this network for predict for performing semantic lebeling of the pixels. So, this is one kind of architecture, one kind of a strategy that has been followed. There are other architectures also, other variations also architectures are most similar.

(Refer Slide Time: 31:45)



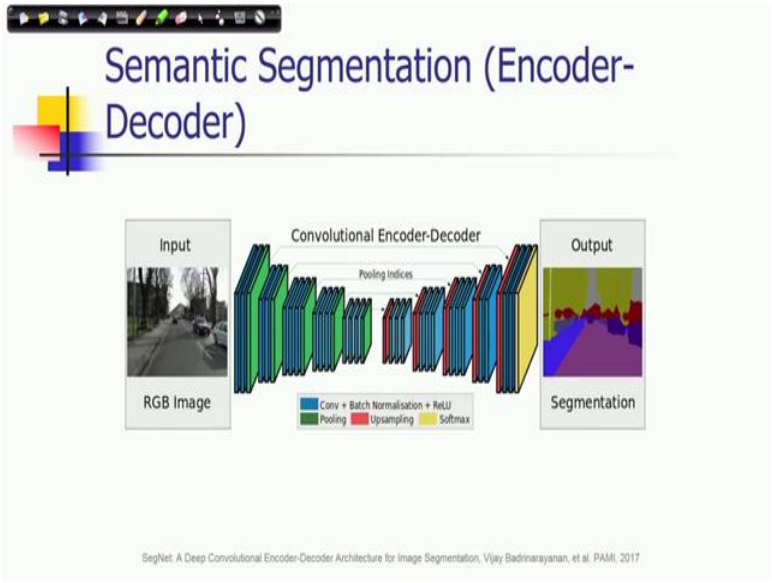
This architecture is called encoder decoder network and it is called segnet. So, what it does at the encoder level, it takes an input image and it generates a high dimensional feature vector. And also through this process it aggregates features at multiple levels.

Whereas, at the decoder it takes a high dimensional feature vector and generate a semantic segmentation mask.

So, in this process we will see instead of a the difference is that instead of up sampling using up sampling filters. In this network the un pooling operation has been used as we discussed earlier how un-pooling could be done; because the idea here is that by doing unpooling you are preserving the location. So, the high responses as we observe during the encoding stages.

So, those are those information are passed to the decoders and that helps in improving the performance that has been observed. So, so, the decoder it decode features a aggregated by encoded at multiple levels that is what its a reverse operation. And as you can see that it projects the discriminated features, which are being learned by the encoder to get it dense classification in the pixel space or in the original image.

(Refer Slide Time: 33:23)



So you can see that what are the processing here. So, you have these are the encoder stages and at every in encoding stage; that means, these are the convolutional layer. And the there are actually 3 convolutional operations and then there is a pooling operation max pooling operations. So, after every max pooling naturally there is a down sampling operations so, after pooling operations.

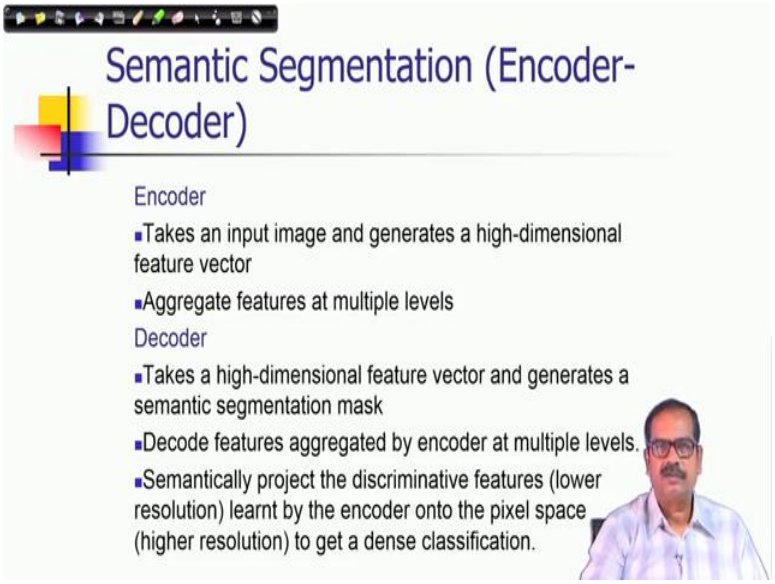
So, every pooling, the pooling indices are correspondingly passed to the corresponding decoding stage at the decoder stage. And they are in this case that is used for unpooling. So, finally, at the in the same way you get at the output at the final level which becomes the same size of the input. And you use the same 1×1 filters of that many number that many number of know levels that you would like to use for segmenting the image.

For example, in this case we could see there are around 1 2 3 4 5 6 say around 6 levels what are we shown here, but you know it may happen that there are many other levels, which are not reflected here because of the content of the image. So, we mentioned earlier that if there are k number of labels. So, at the output you should have $1\times1\times$ filters of length the number of channels what you used in the output that that many number of filters should be there.

This is how from the decoder we generate the segmentation output. However, there could be various decoding strategies that, we would like to see. So, when we are performing semantic segmentation what we are doing . there are two stages; first is a learning stage or that is you are trying to represent it as a feature and learning the features. And the next you are doing the up convolutions and to project it to the semantic domain.

So, there is a stage as you can see the up sampling of learn low resolution semantic feature maps and that is done using up convolutions which are. So, during up convolutions you are using by linear interpolation filter weight initialization of weights and then pixel wise you are predicting. So, the whole thing is being trained in this way. So, your input output specification is there and you are training the network through downsampling and upsampling using this kind of structure.

(Refer Slide Time: 36:24)



So, in the encoder, it takes an input image and generates a high dimensional feature vector. Encoder aggregates features at multiple levels and in the decoder it takes a high dimensional feature vector and generates a semantics segmentation mask. And decode features aggregated by encoder at multiple levels. Semantically it project the discriminative features and also that is learned by the encoder what I mentioned earlier to get a dense classification.

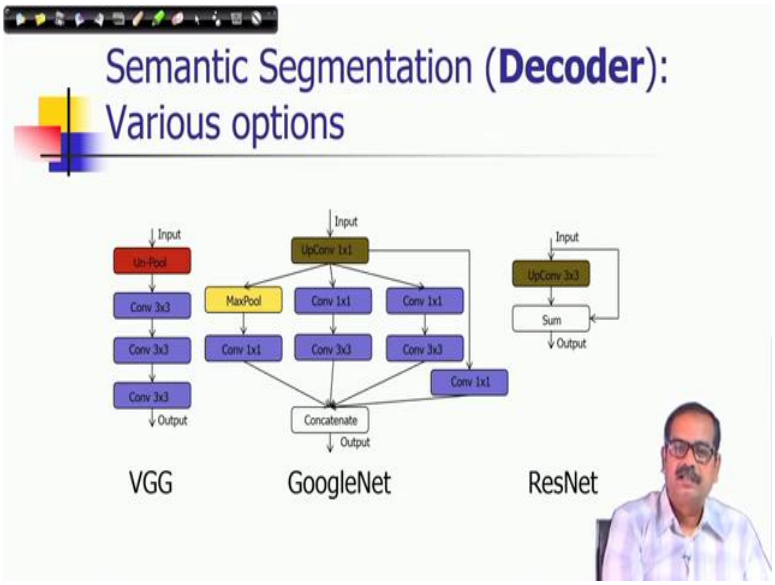
So, the basic thing here is that in this semantic segmentation, the encoder it takes the input image process in input image and it generates a high dimensional feature vectors. And it uses the features at multiple levels whereas, in decoder its input is that high dimensional feature, then it tries to project those features into the semantic domain. So, it tries to correlate those features with the labels and that is what it tries to get a dense classification out of this process.

So, this diagram shows that what could be the flow of computation. So all the convolutions layers and pooling layers are shown; so, in the left side you can see this is the encoder part and in the right side this is a decoder part. And you can see as I mentioned that pooling indices are to be provided for un-pooling operations in corresponding layer.

So, unpooling at this layer should be done using pooling indices of this one. So, that is why it is shown by these arrow. So, these are all pooling indices and this is a flow. So, it

is actually you can see there is no fully connected layer in this particular architecture. So, we call it also fully convolutional neural network architecture.

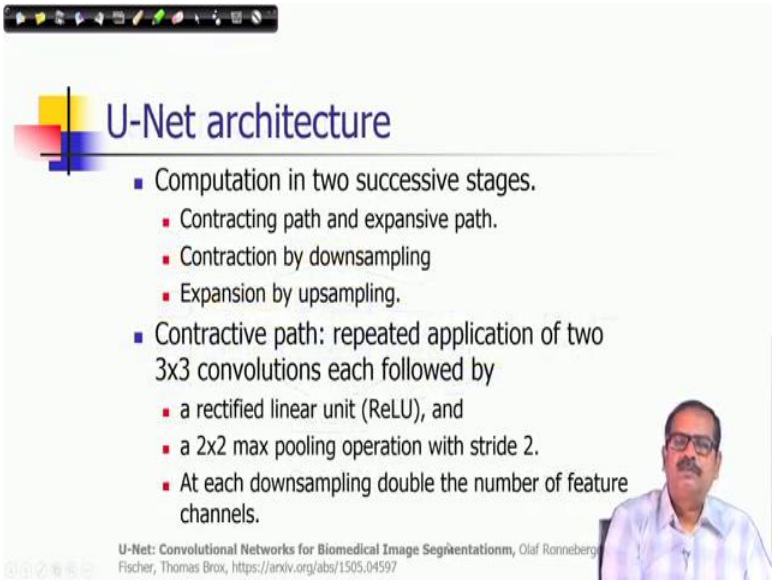
(Refer Slide Time: 38:23)



For decoding there could be various options also, like you know you can use the vgg network which has been shown which uses a convolutional neural network. So use the unpooling operations and then perform convolutions stages of convolutions already you can use Google net where the inception models are used. Once again unpooling has to be there and then as you know that Google net it concatenates the output of different filters sizes but it keeps the output size from each filter the same by using 0 padding.

And you can use this kind of decoder. You can use also resonate a structure, where you know you will be only learning the residual errors. So, several such options are several such architectures are configurations are possible.

(Refer Slide Time: 39:25)



There is one particular architecture, which is a very popular in this with respect to this segmentation and that is called U-Net architecture and this is also used similar concepts. So, let me take a break here and then we will discuss particularly this architecture. And also we will show some of these results from this architecture in the next lecture.

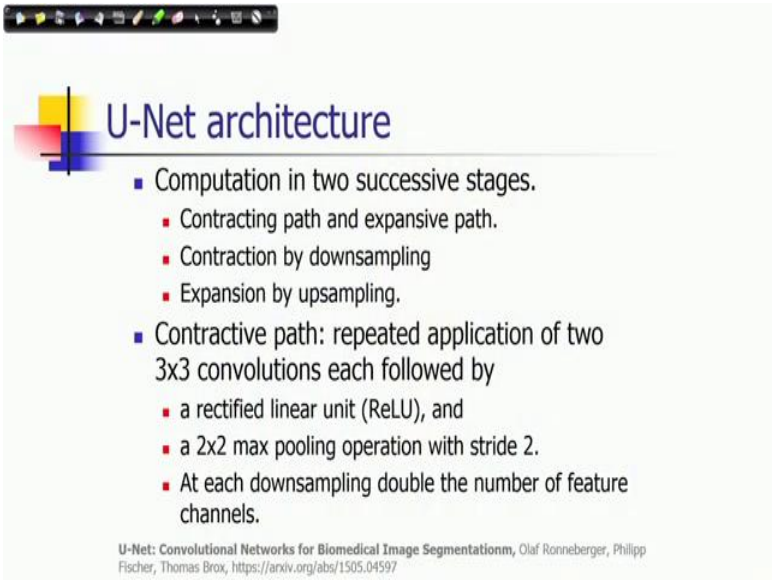
Thank you very much for listening to this lecture.

Computer Vision
Prof. Jayanta Mukhopadhyay
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

Lecture – 60
Deep Neural Architecture and Applications Part – VI

We are discussing about semantic segmentation using Deep Neural Architecture and in the last lecture I have refer to a very popular segmentation architecture which is called U-Net architecture.

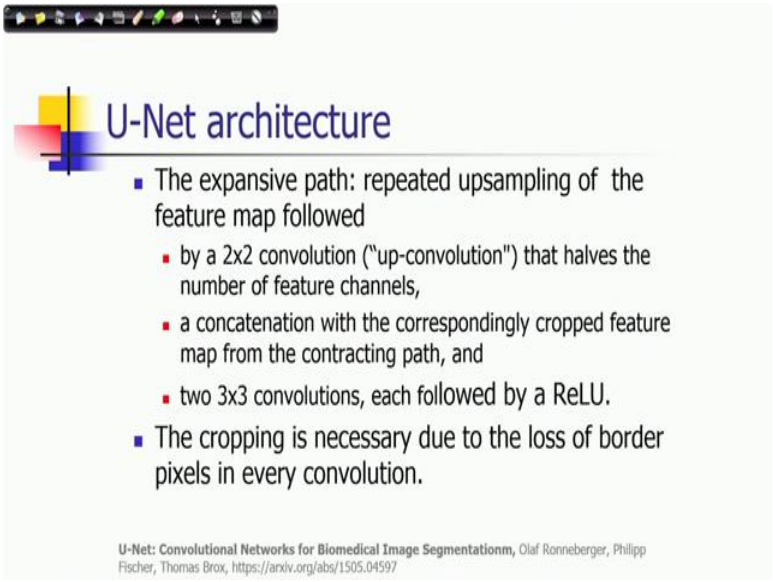
(Refer Slide Time: 00:31)



So, in this lecture I will give you a brief overview of that architecture. So, in U-Net architecture the competition is carried out in two successive stages. Like the previous architecture of semantic segmentation, there is a path for down sampling and up sampling. Here also we have the path for down sampling and up sampling, we call it as a contracting path and also an expansive path.

So, in the contractive path we have repeated application of two 3×3 convolutions, each of them is followed by a rectified linear unit and they need 2×2 max pooling operation with stride 2. And at each down sampling we also double the number of feature channels. That is how, this architecture is specified you can get the details of this architecture by the paper, what is shown here and you can get it from the internet.

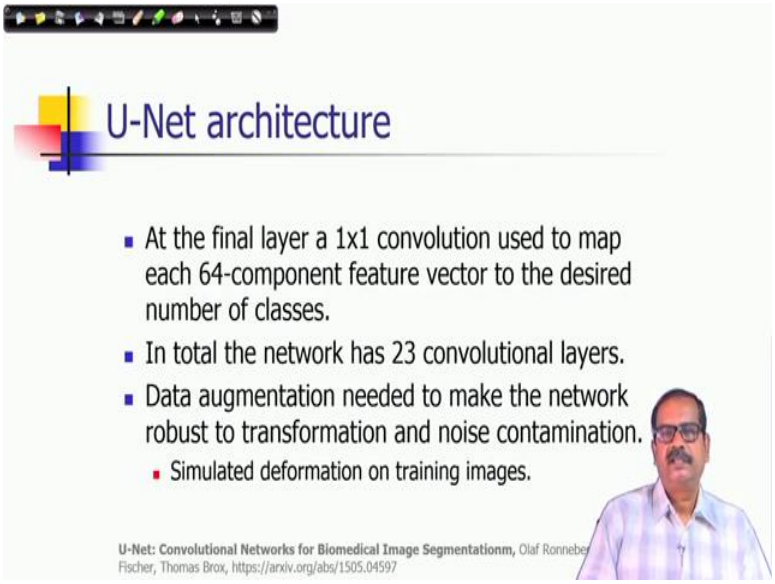
(Refer Slide Time: 01:22)



About the expansive path as I was mentioning that it is a repeated up sampling what is used for the feature map, and there is a 2×2 convolution or up convolution is used. And in the contractive path where the number of channels are doubled at every down sampling, in up sampling, that means, in the expansive path rather the feature channels are getting halved. And a concatenation also is there from the contractive paths output, we will show in the diagram.

So, in the expansive path we are using the up sampled features and also the features what is being computed in the contractive path those are concatenated. So, half of them are coming from there and then using them you are basically passing it to the next stage. So, it has three two 3×3 convolutions after doing up sampling and it goes through the stages. Finally, the size of the image or output remains becomes the same as the size of the input, and you require cropping to keep this constraint of size.

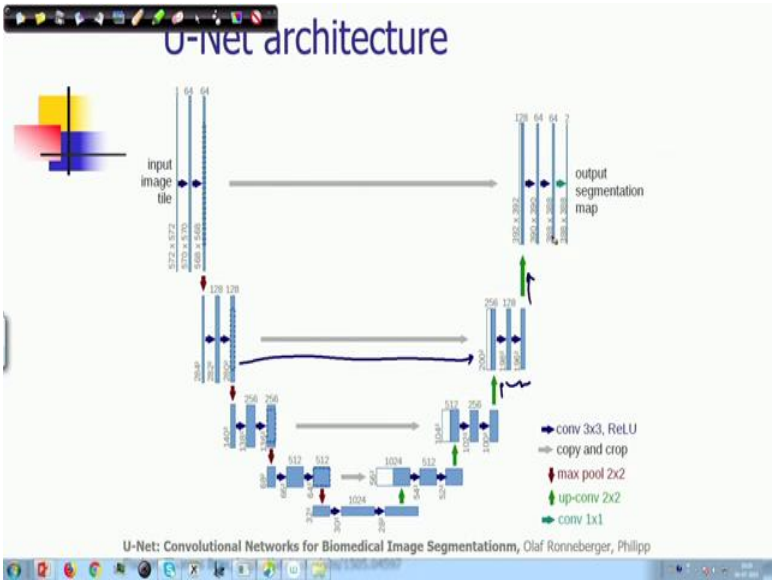
(Refer Slide Time: 02:32)



And at the final layer when you have generated all these pixels at the output, then for each pixel you classify those pixels into a number of classes. So, every pixel at the final layer, it is represented by a feature of 64 feature; 64 dimensional feature vector. And those 64 dimensional feature vectors becomes a input to every classifier.

So, in total there are 23 convolutional layers and in particular this work, they have it is used applied for medical image segmentation or segmentation of microscopic images and there are data augmentations which are used to increase the data pool, for example, they have used simulated deformations on training images.

(Refer Slide Time: 03:23)

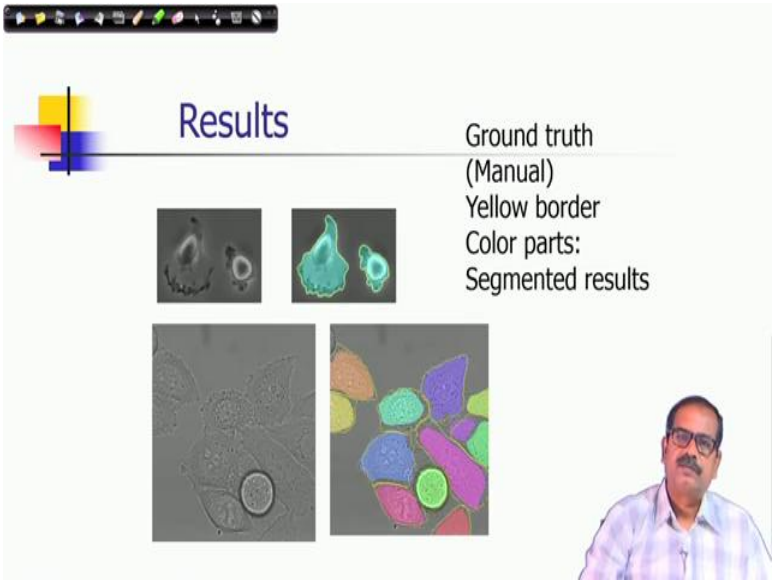


So, this is a schematic diagram which is taken from that paper itself. It is a U-Net architecture and you can see that how the contractive path it is down sampling images of different sizes and the number of channels are getting doubled there. And then in the expensive path, you have up sampling images.

And not only that as I was mentioning there is a concatenation of features. So, these features it is concatenated at this stage. So, half of this is coming from this stage and half of them is coming here and it become once again, this becomes a input to the next two convolution layers. Then again it is up sampled and it is performed in the same fashion.

So, this is the operation, what is being done in the U-Net architecture and you using the input or specification you train it, and then once the training is over then you can use it for the purpose of semantic segmentation.

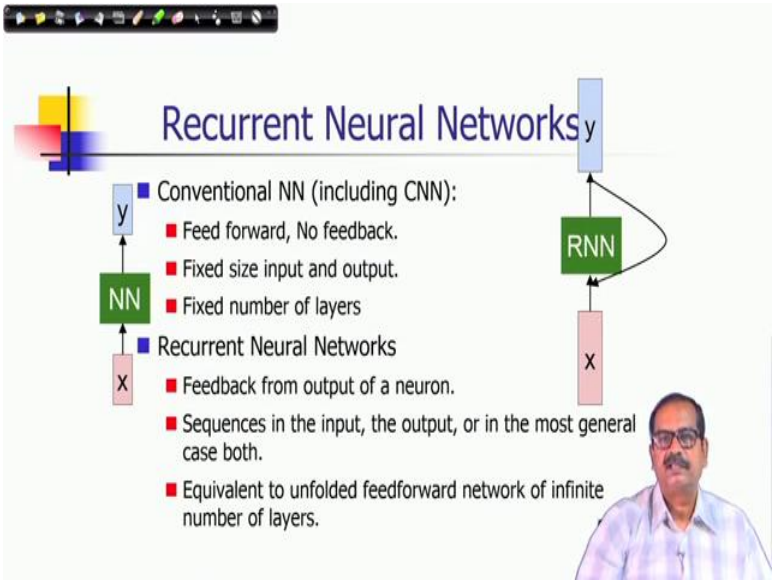
(Refer Slide Time: 04:23)



So, some of the results which are there in that paper I am just producing them. So, you can see that these are microscopic images. The top one is showing the segmentation of objects that these are the cells and they are the yellow border is the manually segmented that is the ground truth and the colour part is a result. So, it is very close we can see.

The bottom rows and the pairs of images are shown, original image and also segmented image. It is quite interesting and which is shown that it is very nicely it captures the corresponding segments and which collaborates with the ground truth also very close to it.

(Refer Slide Time: 05:11)



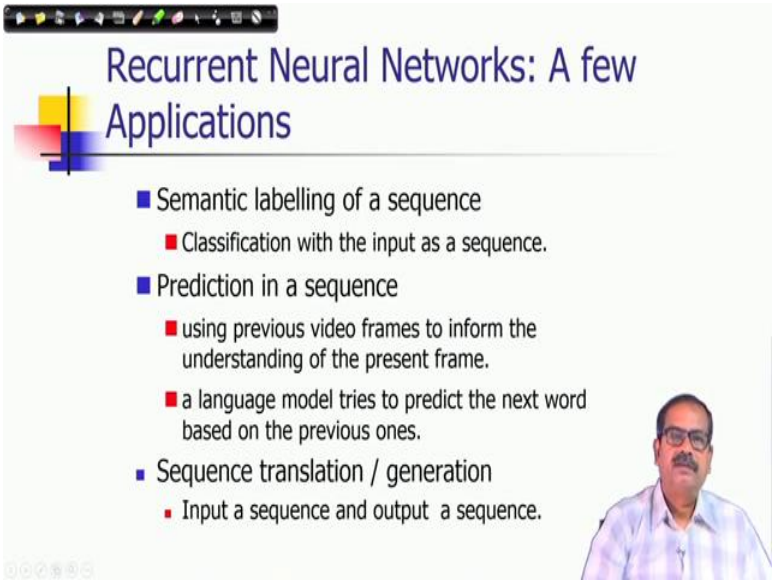
So, next another kind of architecture we will be discussing and we will be considering another types of problems which could be solved using neural architecture, these networks are called Recurrent Neural Networks.

So, let us first understand the difference between recurrent neural networks. For example, the conventional neural network, including CNN there is no feedback, it has been shown in the left side that NN block the green NN block, where the input is x and output is y and there is no connections from the output to the towards the input. Whereas, in recurrent neural networks you have a feedback, you have a connections from the output part to the input block.

So, in the conventional neural network it is a feed forward and there is no feedback. Whereas, in the recurrent neural networks you have feedback from output of a neuron. A conventional neural network it handles fixed size input and output, in the recurrent neural network input could be a variable size. It is a sequences in the input the output or in the most general case both.

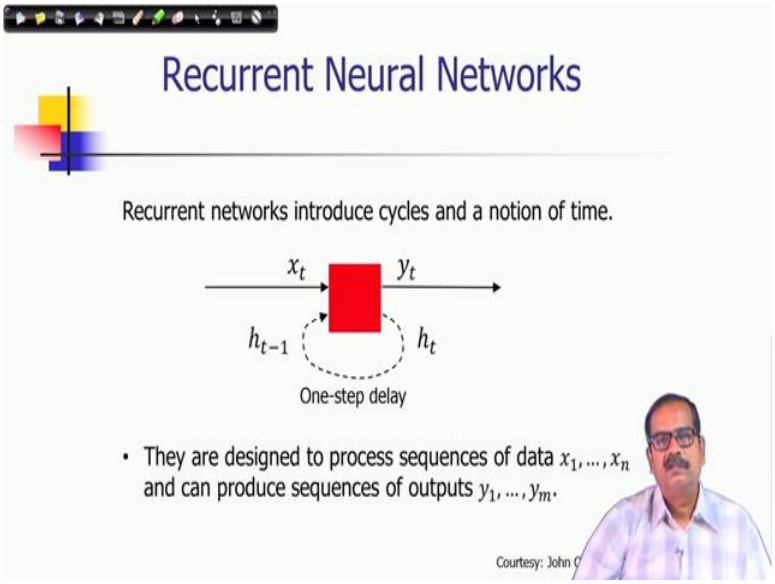
Conventional neural network it is a fixed number of layers and since, there is a feedback actually it can simulate an infinite number of feed forward network, it can unfold into an infinite number of layers. So, this is the characteristics of recurrent neural networks.

(Refer Slide Time: 06:47)



Some of the applications of recurrent neural networks, let us quickly visit some of those applications, like semantic labeling of a sequence. So, it can classify the input as a sequence, then prediction in a sequence. So, using previous a video frames to inform the understanding of the present frame one kind of example or you can consider the language model, it tries to predict the next word, based on the previous ones. And sequence translation and generation where input is a sequence and output is also a sequence.

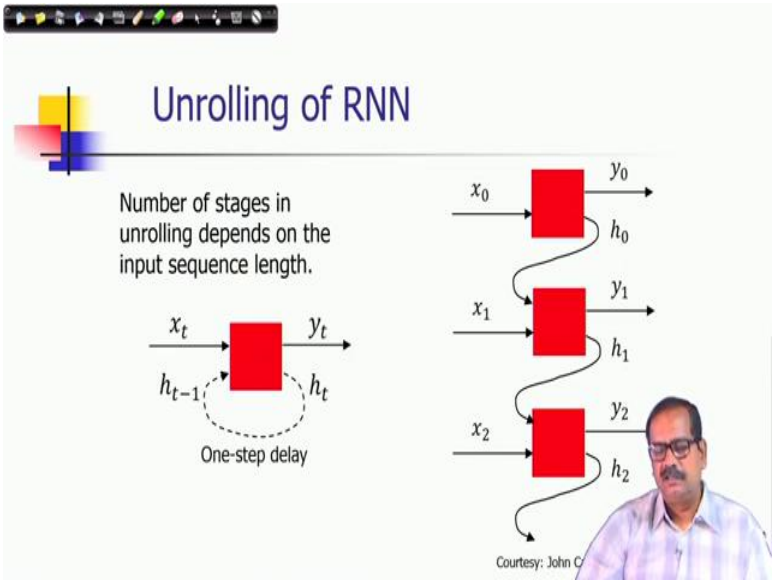
(Refer Slide Time: 07:25)



So, the basic feature of the recurrent neural network is that it introduced cycles and notion of time. So, consider this particular diagram and here the red square is representing a recurrent neural network node. So, input at time t, x_t and then there is output y_t, but the intermediate output from the network which has been also given as a feedback. So, you can see that input is not only x_t even the previous time instances output is also input to this block h_{t-1}.

So, as if there is a one step delay and this is how it works. So, this is used to design two process sequences of data and it can produce sequences of outputs. Because of this feature, that you are introducing a notion of sequence or time and it is kind of a delay in the processing. So, we can consider when you are giving input, input can come as a sequence and when you are producing output, output we will also a sequence.

(Refer Slide Time: 08:47)



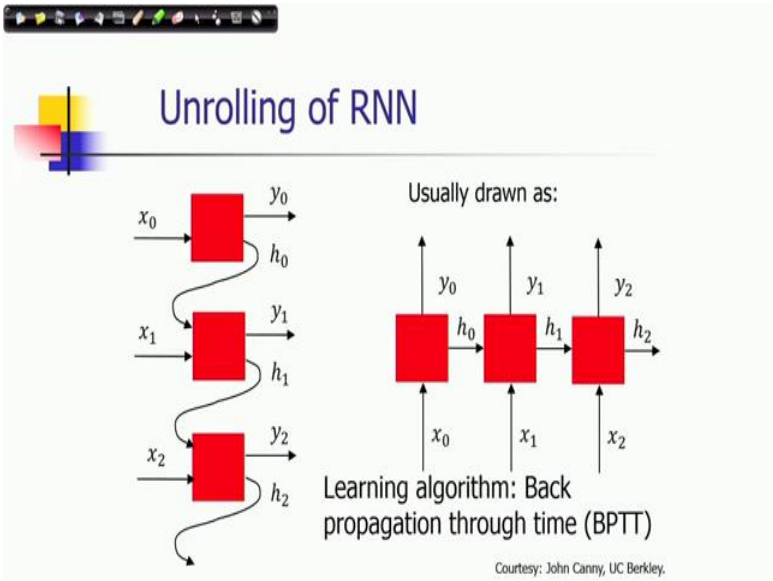
So, to analyze a recurrent neural network, we have to unroll or unfold the neural network. And number of stages of unrolling it depends upon the length of the input sequence.

So, you consider this that suppose you have three; length of sequences three x_0, x_1, x_2; so, x_0 is the past. So, the difference you note when we consider so, digital filter as if the input is coming from the left side and it is being shifted towards right. But here, the past is always shown at the in the figure in the top side and all the present are following

successively. So, as you proceed in the time domain because it is unfolding over the time you know instances.

So, you are always the temporarily present sample or current sample will be shown at the bottom or at the layer, which is successive layers always the temporarily following inputs will be coming in the sequence. So, if there are say for example, the sequence length is 3. So, you can consider there are three such unfolding.

(Refer Slide Time: 10:12)

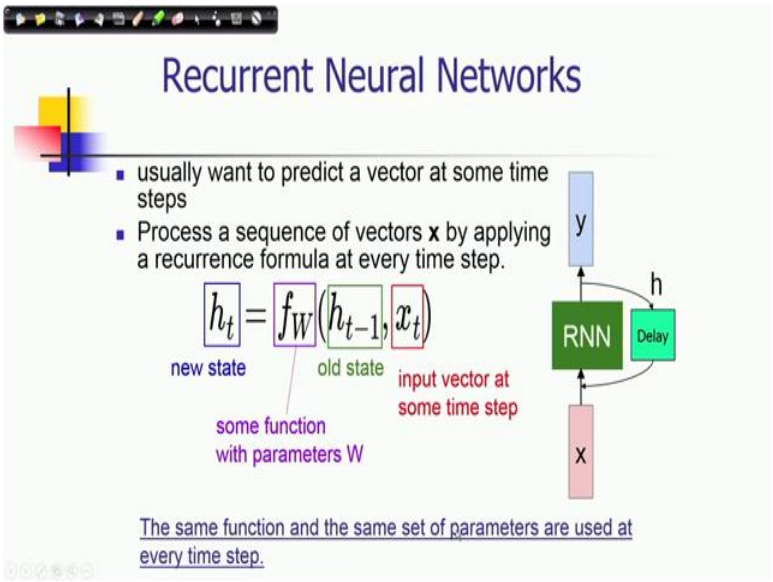


You can draw it also in this fashion; that means, you provide input x_0 in the first stage get output y_0. Also it generates an intermediate output h_0, which could be equal to y_0 or which could be some function of y_0. And they that becomes the input to the next RNN, a next stage next block of neural block it is unfolding, which is the same RNN string, but at that time instead it is operating with input x_1 and also from the intermediate input h_0 generates y_1.

Next time stamp it will be input x_2, then h_1 both are input, that is intermediate output of the previous time instance is h_1 and it produces output y_2. Also generates an intimated output h_2, since there is no other input and we may not observe. So, maybe now your sequence has been generated as y_0, y_1, y_2 from x_0, x_1, x_2. That is a kind of

explanation of the working of RNN. So, the learning algorithm what we use, if I unfold this the same back propagation algorithm can be used.

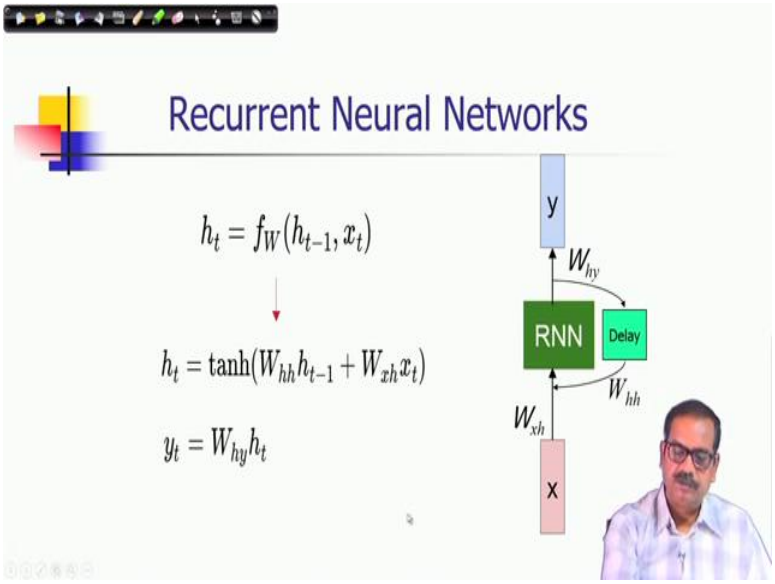
(Refer Slide Time: 11:24)



So, this is trying to elaborate that particular aspect, that relationship between the intermediate output and also the past input or present input and past intermediate output. So, you can see that y is function of h_t and h_t is function of h_{t-1}, x_t of that RNN block where the w is the parameter.

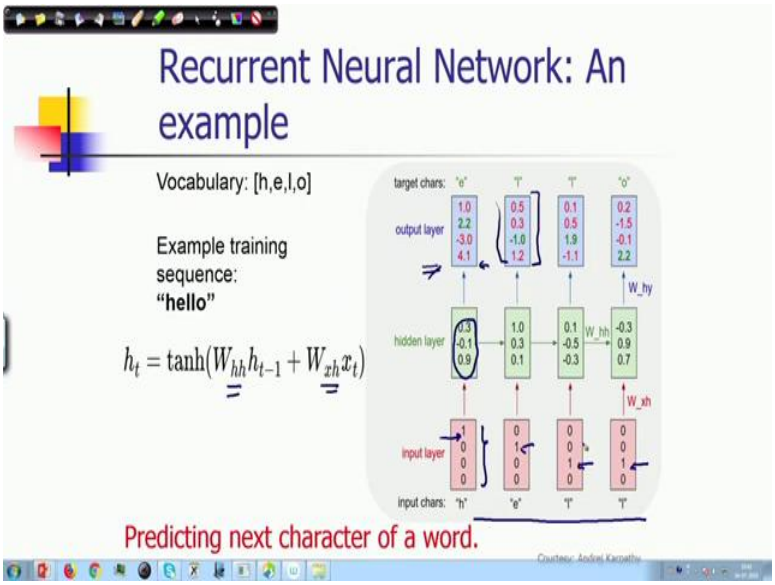
So, h_{t-1} is the old state, x_t is the input victor at sometimes time and W is a function with parameter w and h_t is the new state. So, the same function and the same set of parameters are used at every time step. And y_t can be considered as a function of h_t it is a linear function, there is a W_{hy} is the weights connecting the intermediate output of the neuron to the final output.

(Refer Slide Time: 12:26)



So, one of the form of h_t could be that it is tan hyperbolic function aAnd it has been shown by this weight set. So, W_{hh} is a matrix, which is producing the vector h_{t-1}, which is producing this particular form and $W_{xh}x_t$ is another set of inputs, and relationship within y_t and h_t is it is $W_{hy}h_t$ linear combinations from h_t.

(Refer Slide Time: 12:57)



So, one particular example is shown here, say in this case it is a example has been drawn from the natural language processing or string processing. So, what we are trying to

predict, we are trying to predict the next character of a word. So, there are training sequences.

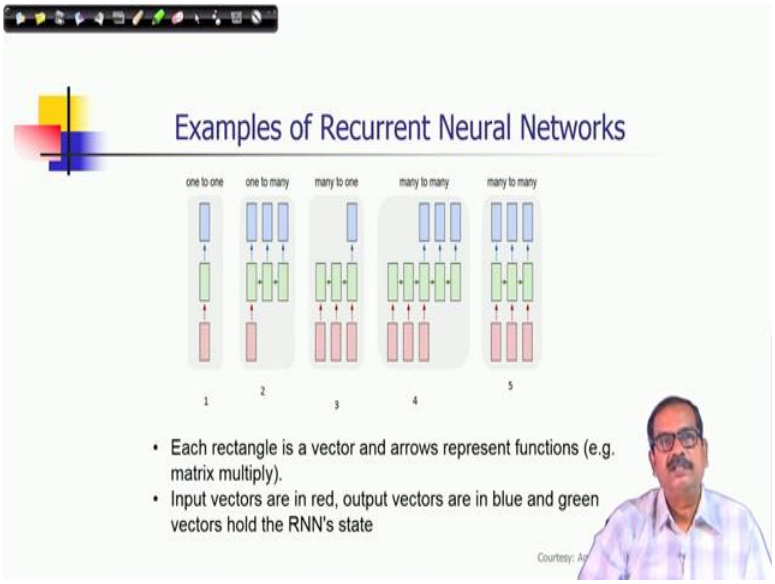
So, one may say hello is a training sequence and as you can see this RNN input. It can unfold the RNN into these seems there are the length of the string is five and there are prediction is four. So, we consider unfolding of RNN till fourth stages of input character and this is how the input is proceeded with temporal timestamp and this is the structure. And so, these are the weights at the stage and this is the weight for h_{t-1}, it is an one-dimensional input it is shown here.

Now, hidden layer has a three input sorry, hidden layer has a three node three vectors. So, accordingly you should have the weight vector W_{hh} and also $W_{xh}x_t$ that should come here and then you are using tan hyperbolic layer. So, there are three such nodes here and from there your output which is also considered as output of vector four dimensional vector.

Now, you are using here hot encoding; so, you can see the encoding which is used here. So, per h we are using a four dimensional hot encoding here. So, this position one shows h, this is e, this is l, this is l and your outputs which are being produced here yeah. The specified ground truths should have been here it should have been $\begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$, but instead you are producing this output. So, the last function will be determined in this fashion.

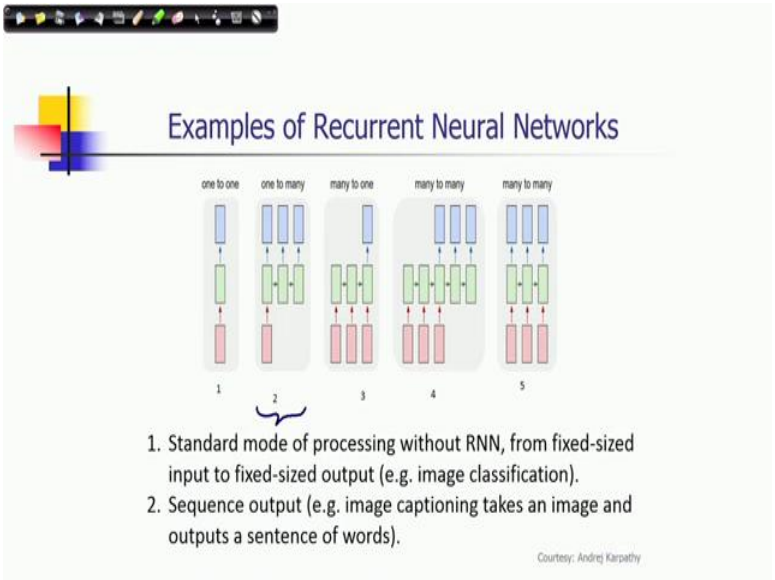
Similarly, this is a production of l, I mean it should have been l, but it produces this output and otherwise you know the ground truth should have been $\begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$. So, in this way the last function set to be defined.

(Refer Slide Time: 15:19)



These are some examples, that what are possible configurations and how different kinds of neural networks could we use, different architectures could be used for solving different types of problem. Here, you can see that we have given some numbers and each rectangle is a vector and arrows represent functions and input vectors are shown in red and output vectors are in blue.

(Refer Slide Time: 15:47)

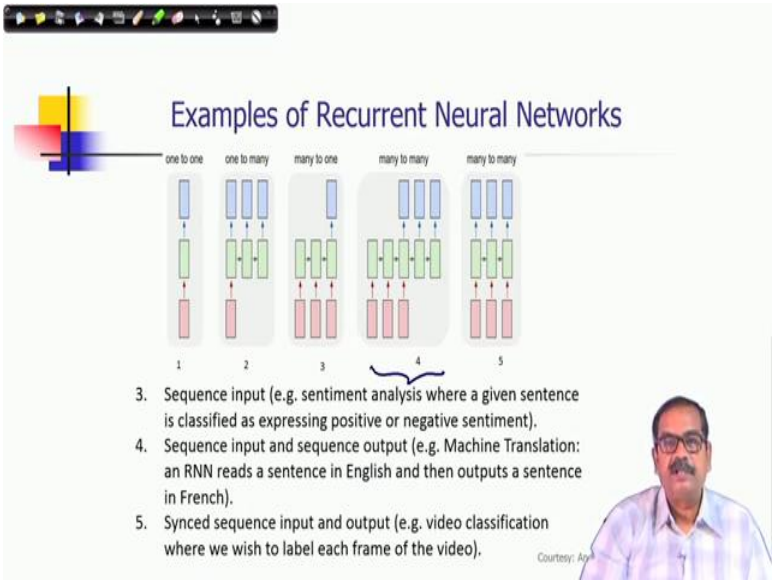


So, let me just explain. So, suppose these are different configurations first one is one to one configuration which means input is a fixed vector and output is also fixed size output.

So, example is an image classification and this is a conventional neural network processing, you do not require RNN for this kind of processing. Where as a second one which means this particular structure the input is a fixed size input or output is a sequence.

Now, here you require a RNN and so, it is a second of output the example could be the image captioning, which takes an image and outputs the sentence of word.

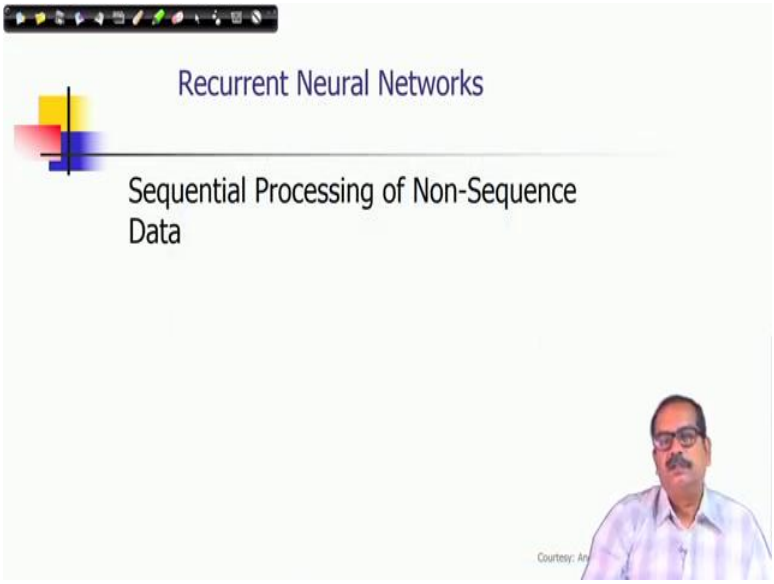
(Refer Slide Time: 16:38)



The third one is, so this is a sequence output is fixed. So, typical example could be that say sentiment analysis, where a given sentences classified as expressing positive or negative statement sentiment. Fourth applications, so these are fourth one where input is a sequence also output is a sequence and this is an example could be the machine translation. It reads a sentence in English and then outputs the sentence in French for example, a typical example of a problem, here also RNN should be use.

So, only for the first one it is a conventional network for all these kinds of sequence processing, we need to use RNN. Consider the fifth one you have the synchronous, no sequence input output sequence and it could be the video classification for labelling each frame of the video.

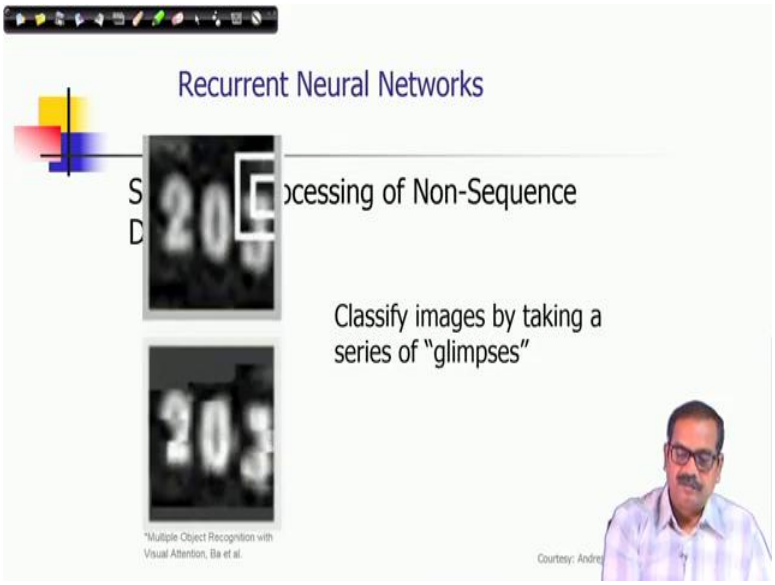
(Refer Slide Time: 17:46)



So, these are some examples of RNN, use of RNN in different applications. Sometimes you can convert even a pixel input as a sequence by following certain predetermined order of scanning the input data.

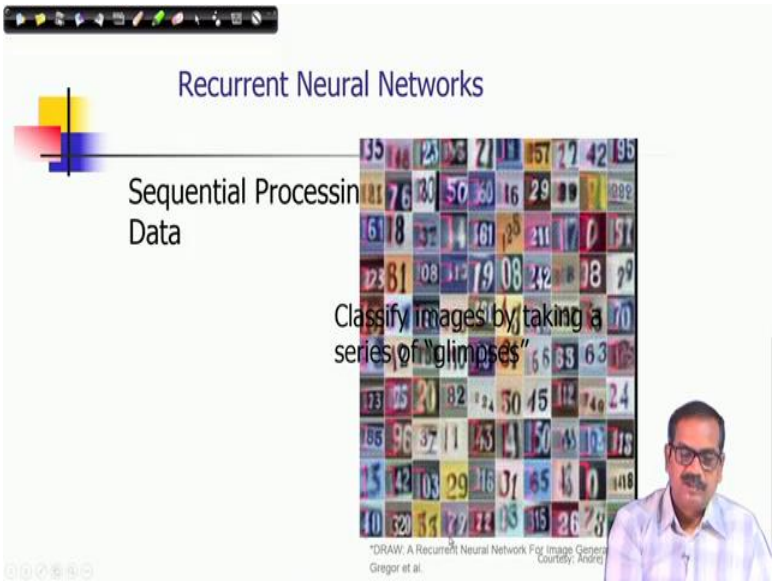
So, we call it sequential processing of non-sequence data; for example, for image we can scan from left to right and top to bottom. Now that itself will provide you some sequence of regions blocks. Now, those could be input to RNN and we can process it for various purposes.

(Refer Slide Time: 18:20)



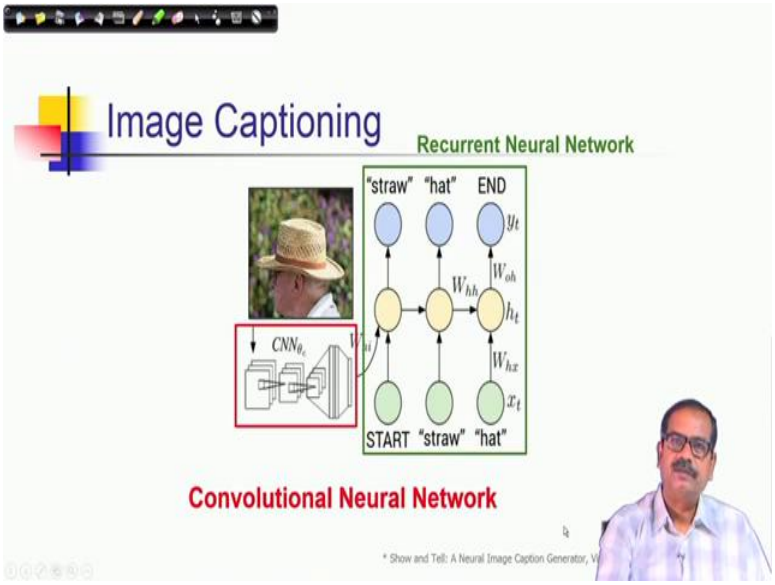
So, this is just this is showing a one kind of processing. So, the blocks are moving in a sequence and we would like to classify images by taking a series of glimpses.

(Refer Slide Time: 18:33)



Another example classify images by taking series of glimpses

(Refer Slide Time: 18:38)

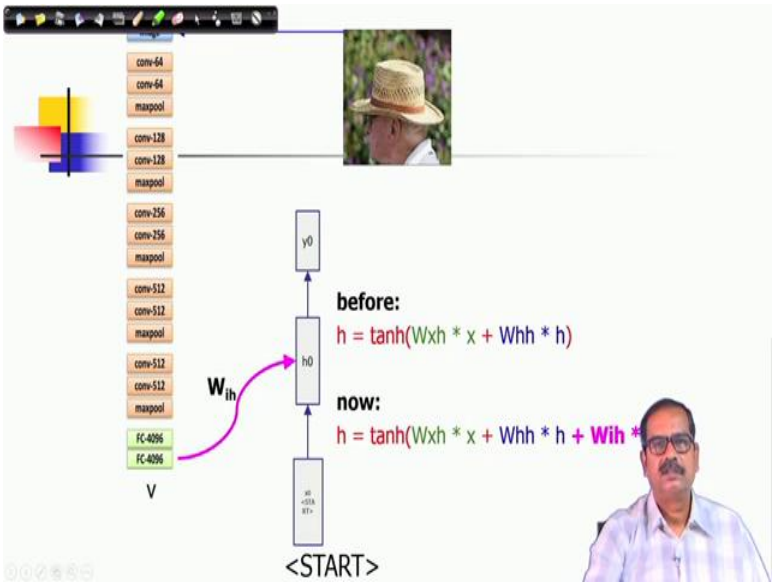


So, the application of this kind of one such very popular application is image captioning, where image regions we also can scan. But in this case what it does that it represents the image as a feature vector using a CNN and that is input to an RNN and the descriptions one by one the descriptions of the output. Once again the number of the length of the

output sequence depending we decide the length of the unfolding of RNN that has to be there.

So, for example, in this particular figure the image could be described as the straw hat and that is the sentence; so, it has to start and it has an end. So, with that kind of specifications you are training the RNN in this way. So, you use the convolution neural network to describe the feature and then, followed by recurrent neural network to extract the caption.

(Refer Slide Time: 19:42)

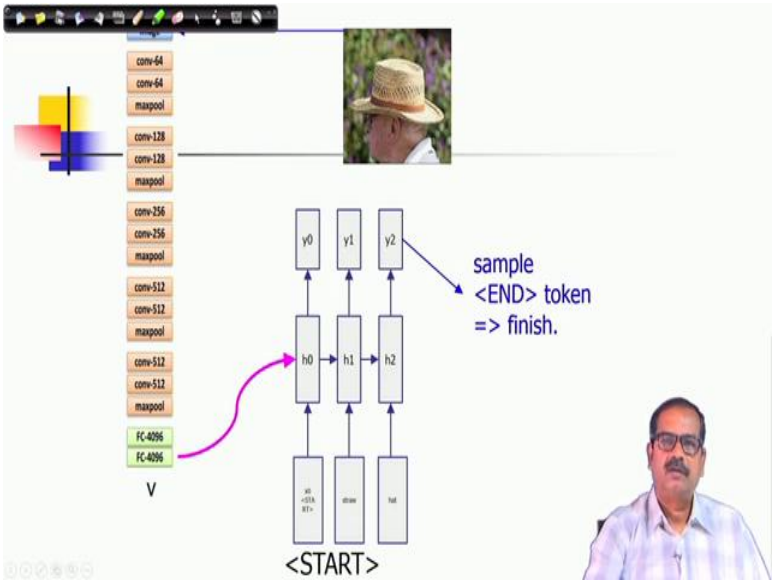


So, it just shows the workflow that you use a convolution neural network, where image is an input and it goes through several layers of convolution neural network and finally, you represent it is a feature vector. In fact, the last layers of fully connected layers they are mostly used for classification, you can remove that layer and you can use only the representation of the last layer of the fully connected layer FCporchiporchi

4096; that means, the feature dimension is 4096 at the last layer in this case.

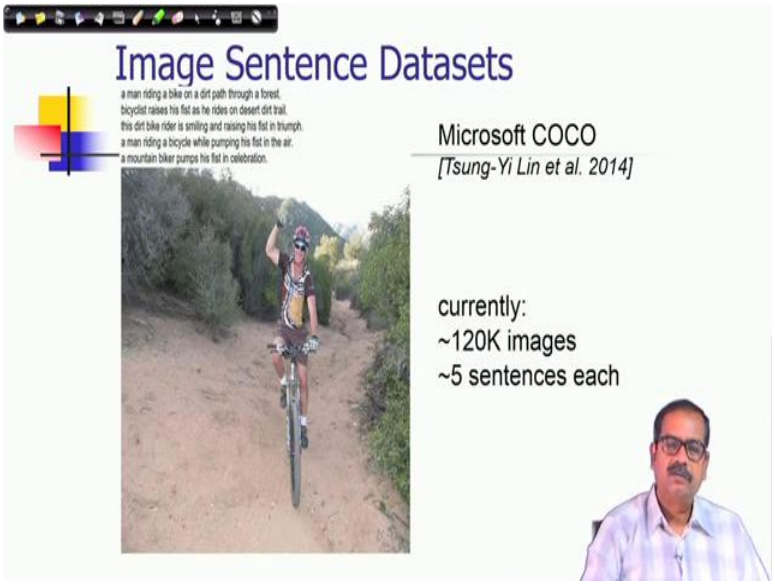
So, that could be the input to your RNN and also the input another input is a start symbol from where you are starting the sentence reconstruction, it reconstructed the sample why not, that itself is the input towards your intermediate sample.

(Refer Slide Time: 20:39)



So, these are sample and it goes like this. So, in the same (Refer Time: 20:44) way RNN operates and then it can generates a sentence. So, as you train on a number of images it depending upon the image content it tries to predict those levels which are being trained in a network.

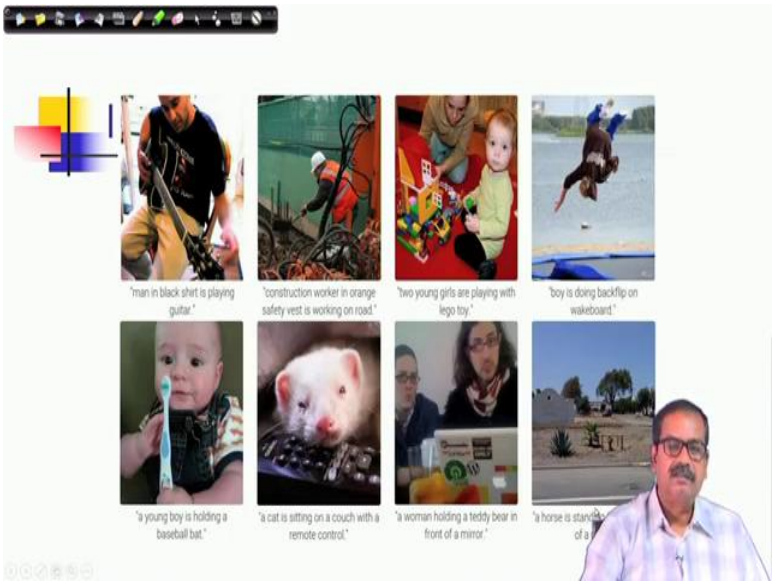
(Refer Slide Time: 20:59)



There is a data set which is called Microsoft COCO and it has say 120 K images and for each image there are 5 sentences and used it for training and there is an example. So, you

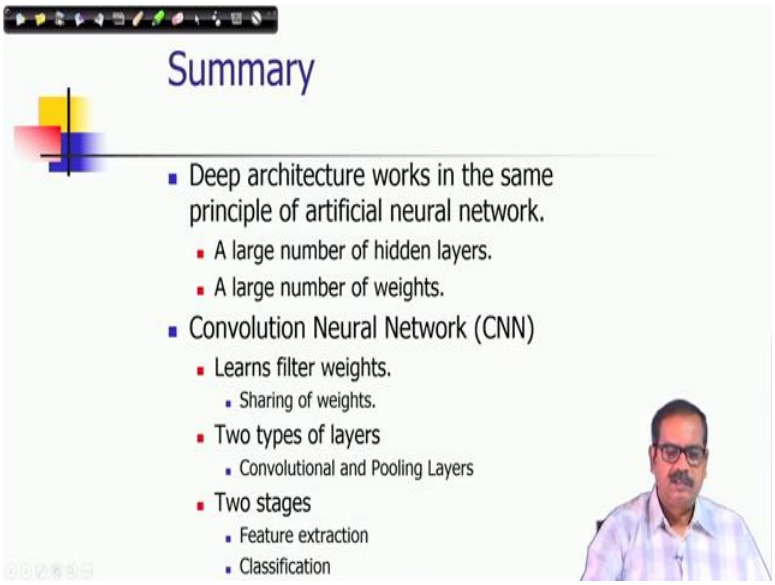
provide this as training set to the network and then you can generate different kind of image captions.

(Refer Slide Time: 21:18)



So, some examples are shown here consider the first row top row and the leftmost image. Where, you have the man in black shirt is playing guitar and a say other one is shown as construction worker in orange safety (Refer Time: 21:39) he is working on road. So, these are a very interesting results and presently a lot of work is going on video summarization, image captioning and many other things using this kind of network.

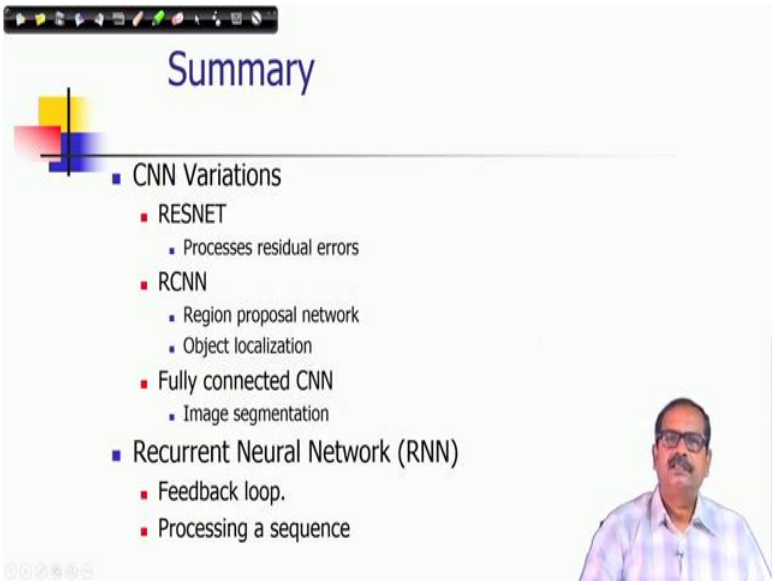
(Refer Slide Time: 21:55)



So, let me summarize the topics and whatever we have covered under this deep neural architecture and its applications. So, we have seen that deep architecture works in the same principle of artificial neural network, but it has a large number of hidden layers and it has a large number of weights.

One a very popular deep neural architecture is a convolution neural network, because it can learn filter weights and it can share those weights and there are two types of layers usually in this process. One is convolution layer another is this is pooling layers for the feature description, and there are two stages which are involved feature extraction and classification. So, in the classification stage usually we used fully connected neural networks.

(Refer Slide Time: 22:50)



So, there are variations of convolution neural networks say RESNET it processes residual errors then RCNN it has a region proposal network and it localizes object and also it could be fully connected CNN where, it uses image segmentation. And then there is recurrent neural network where it is a different kinds of neural architecture, it is quite different from convolution neural network, because it has a feedback loop it is not a feed forward network.

And recurrent neural networks are mainly used for processing a sequence. So, with this let me stop here. And this is the end of this particular topic and this lecture.

Thank you very much for listening to my lecture.

THIS BOOK
IS NOT FOR
SALE
NOR COMMERCIAL USE



(044) 2257 5905/08
nptel.ac.in
swayam.gov.in